

*Communauté Économique et Monétaire
de l'Afrique Centrale
(CEMAC)*



Cameroon AI Enthousiaste



*Institut Sous-régional de Statistique
et d'Économie Appliquée*

Organisation Internationale

BP : 294 Yaoundé - Tél : 222 220 134

*Direction du Développement et de la
Recherche*

ONG camerounaise

BP : 83000 Toulon, France

*Mémoire professionnel de fin d'étude présenté en vue de l'obtention du
diplôme d'Ingénieur Statisticien Économiste(ISE)*

OPTION : Data Science et Marketing

CONCEPTION D'UN SYSTEME DE RECOMMANDATION HYBRIDE POUR LE MATCHING EMPLOI-COMPETENCES AU CAMEROUN PAR INTEGRATION DES LLMS ET DES GRAPHES DE CONNAISSANCES

Rédigé par :

NGOULOU NGOUBILI Irch Defluviaire

Élève Ingénieur Statisticien Économiste cycle long – 5^e année

Sous l'encadrement de :

Dr. FOUTSE Yuehgoh

Présidente & Directrice exécutive adjointe, PhD Computer Science & Data Scientist

Année académique 2025-2026

Dédicace

À mes parents... (Max 1 page)

Remerciements

Sommaire

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	vi
Liste des tableaux	viii
Liste des figures	x
Avant-propos	xi
Résumé	xiii
Abstract	xiv
Introduction générale	1
I Cadre théorique et conceptuel	6
1 Fondements théoriques et revue de la littérature sur l’IA générative et les systèmes de recommandation	7
1.1 Concepts clés mobilisés dans le mémoire	7
1.2 Fondements économiques et problématique d’appariement	10

1.3	Systèmes de recommandation et person-job fit	11
1.4	IA générative et connaissances	13
1.5	RAG, GraphRAG et orchestration agentique	15
1.6	Évaluation, acquis de la littérature et positionnement du mémoire . . .	17
2	Source de données et Approche méthodologique	22
2.1	Cadre méthodologique et sources de données	22
2.2	Modélisation sémantique et structuration des connaissances	29
2.3	Orchestration agentique et génération contrôlée	33
2.4	Évaluation prévue, limites et synthèse méthodologique	35
II	Présentation des résultats	37
3	Implémentation du système de recommandation	38
3.1	Architecture opérationnelle et données exploitées	38
3.2	Implémentation du modèle d’embeddings et indexation pgvector	50
3.3	Implémentation de pgvector comme brique de retrieval	59
3.4	Implémentation du graphe Neo4j comme base de connaissances	66
3.5	Implémentation du moteur hybride de recommandation	73
3.6	Orchestration LangGraph, génération contrôlée et interfaces	79
4	Évaluation de la performance du système	84
4.1	Objectif et logique générale de l’évaluation	84
4.2	Données et protocole expérimental	85
4.3	Évaluation du modèle d’embeddings	86
4.4	Ablation pgvector seul contre pgvector + graphe	87

Sommaire

4.5	Analyse du reranking par le graphe	89
4.6	Calibration bayésienne des pondérations du score hybride	90
4.7	Performance opérationnelle et stabilité	93
4.8	Interprétation par rapport aux hypothèses	93
4.9	Lecture des sorties graphiques de performance	94
4.10	Limites de l'évaluation	95
4.11	Synthèse du chapitre	96
5	Discussion et recommandations	97
	Conclusion générale	I
	Bibliographie	I
	Bibliographie	III
A	Annexes	VIII

Liste des sigles et abréviations

ANN :	Approximate Nearest Neighbors
API :	Application Programming Interface
BERT :	Bidirectional Encoder Representations from Transformers
BI :	Business Intelligence
CEMAC :	Communauté Économique et Monétaire de l'Afrique Centrale
CV :	Curriculum Vitae
DCG :	Discounted Cumulative Gain
ESCO :	European Skills, Competences, Qualifications and Occupations
FAISS :	Facebook AI Similarity Search
FNE :	Fonds National de l'Emploi
FRED :	Federal Reserve Economic Data
GAT :	Graph Attention Network
GCN :	Graph Convolutional Network
GNN :	Graph Neural Network
GPT :	Generative Pre-trained Transformer
GPU :	Graphics Processing Unit
HNSW :	Hierarchical Navigable Small World
IA :	Intelligence Artificielle
IDCG :	Ideal Discounted Cumulative Gain
ISE :	Ingénieur Statisticien Économiste
ISSEA :	Institut Sous-régional de Statistique et d'Économie Appliquée
IVF :	Inverted File Index
KG :	Knowledge Graph
LLM :	Large Language Model
LoRA :	Low-Rank Adaptation
LSTM :	Long Short-Term Memory
MAP :	Mean Average Precision
MEPC :	Nomenclature Camerounaise des Métiers, Emplois et Professions
MRR :	Mean Reciprocal Rank
NCF :	Nomenclature Camerounaise des Formations
NCM :	Nomenclature Camerounaise des Métiers
NDCG :	Normalized Discounted Cumulative Gain
NLP :	Natural Language Processing
OIT :	Organisation Internationale du Travail
ONG :	Organisation Non Gouvernementale
QLoRA :	Quantized Low-Rank Adaptation
RAG :	Retrieval-Augmented Generation
RNN :	Recurrent Neural Network
RWKV :	Receptance Weighted Key Value
SEO :	Search Engine Optimization
SGPT :	Sentence Generative Pre-trained Transformer
SQL :	Structured Query Language
SR :	Système de Recommandation

Liste des tableaux

2.1	Correspondance entre objectifs spécifiques et choix méthodologiques .	26
2.2	Sources de données mobilisées et utilisation méthodologique	27
3.1	Pipeline général implémenté : étapes, entrées et sorties	40
3.2	Séparation des couches techniques du système	41
3.3	Volumes réellement exploités par les briques de recommandation	44
3.4	Lecture d'ingénierie de la qualité des champs utiles	46
3.5	Synthèse des traitements de nettoyage utiles au moteur	50
3.6	Protocole de construction des paires de fine-tuning	52
3.7	Hyperparamètres du fine-tuning SentenceTransformer	54
3.8	Comparaison directe entre le baseline et le modèle fine-tuné	56
3.9	Résultats de benchmark des représentations	57
3.10	Distribution des embeddings par type d'entité	60
3.11	Structure opérationnelle des tables pgvector	60
3.12	Diagnostic statistique du chunking documentaire	62
3.13	Indicateurs techniques pgvector	63
3.14	Index pgvector et index complémentaires	64
3.15	Indicateurs structuraux du graphe Neo4j	69
3.16	Principaux types de noeuds et de relations	70

3.17	Hétérogénéité des degrés par famille de noeuds	71
3.18	Chemins métiers exploitables pour l'explication	73
3.19	Composantes du score hybride implémenté	74
3.20	Règles de verdict du moteur hybride	76
3.21	Distribution statistique des verdicts sur les sorties de reranking	77
3.22	Résumé statistique des scores et des rangs	78
3.23	Exemple de reranking hybride pour un candidat	79
3.24	Diagnostic d'implémentation de la génération contrôlée	80
4.1	Benchmark des modèles d'embeddings sur 473 requêtes de test	86
4.2	Comparaison retrieval : pgvector seul versus pgvector + graphe	88
4.3	Résultat de l'optimisation bayésienne des pondérations du score hybride	91
A.1	Labels de nœuds et relations structurantes du graphe	IX
A.2	Volume d'entités indexées dans la base vectorielle pgvector	X

Table des figures

1.1	Évolution simplifiée des modèles de langage et des représentations textuelles	13
1.2	Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG	16
2.1	Architecture méthodologique du workflow agentique de recommandation emploi-compétences	24
3.1	Pipeline opérationnel effectivement implémenté	39
3.2	Interface Streamlit du système de recommandation	42
3.3	Diagnostic API ou état des services	43
3.4	Répartition des offres par source après nettoyage	45
3.5	Localisation nettoyée des offres	47
3.6	Compétences fréquentes et disponibilité descriptive	48
3.7	Trace d'entraînement du modèle SentenceTransformer	53
3.8	Évolution de la perte, pertes finales par split, métriques de validation et learning rate	55
3.9	Comparaison des modèles d'embeddings : qualité de classement et latence	57
3.10	Projection PCA et distribution des similarités cosinus	58
3.11	Indicateurs d'indexation pgvector	59
3.12	Tables et index pgvector dans PostgreSQL	65
3.13	Schéma logique du graphe de connaissances	66

3.14	Visualisation du graphe dans Neo4j Browser	67
3.15	Composition du graphe par types de noeuds et relations	68
3.16	Degrés et noeuds centraux du graphe	70
3.17	Matrice des liaisons entre familles de noeuds	72
3.18	Workflow LangGraph dans l’environnement de développement	80
3.19	Effet du reranking graphe sur le retrieval	81
3.20	Exemple de réponse générée avec sources et traces	82
3.21	Synchronisation entre pgvector et Neo4j	82
4.1	Comparaison des modèles d’embeddings sur le jeu de test	87
4.2	Effet de l’enrichissement graphe sur les métriques de retrieval	88
4.3	Distribution des déplacements de rang après reranking par le graphe	89
4.4	Évolution du NDCG@10 au cours des essais Optuna	91
4.5	Pondérations retenues après optimisation NDCG@10	92
4.6	Comparaison du NDCG@10 avant et après calibration	92
4.7	Stabilité opérationnelle observée pendant l’évaluation d’ablation	95

Avant-propos

L'Institut Sous-régional de Statistique et d'Économie Appliquée (ISSEA), institution spécialisée de la CEMAC créée en 1984, a pour mission principale la formation de cadres supérieurs en statistique, économie et data science, capables de répondre aux enjeux contemporains de décision fondée sur les données. Dans ce cadre, l'ISSEA propose plusieurs filières de formation, parmi lesquelles figure celle des Ingénieurs Statisticiens Économistes (ISE), alliant rigueur mathématique, modélisation économétrique et outils modernes d'analyse de données (Data Science).

Arrivés à un niveau avancé de leur formation, les élèves Ingénieurs Statisticiens Économistes sont tenus d'effectuer un stage professionnel d'une durée minimale de trois (03) mois. Ce stage constitue une phase essentielle du cursus, permettant de confronter les connaissances théoriques acquises à des problématiques réelles, tout en favorisant l'acquisition de compétences opérationnelles en environnement professionnel. Il est sanctionné par la rédaction d'un mémoire professionnel, soutenu devant un jury.

C'est dans ce cadre que nous avons effectué un stage du 9 mars 2026 au 30 juin 2026 au sein de KmerAI, où nous avons été intégrés à la Direction du Développement et de la Recherche. Cette immersion nous a permis de travailler sur une problématique appliquée située au croisement de l'analyse de données, de l'intelligence artificielle générative, du traitement automatique du langage naturel et de l'aide à la décision : l'amélioration du matching entre offres d'emploi, profils de candidats et compétences recherchées sur le marché camerounais.

Le présent mémoire s'inscrit dans cette dynamique et porte sur le thème : « Conception d'un système de recommandation hybride pour le matching emploi-compétences au Cameroun par intégration des LLMs et des graphes de connaissances ». Il vise principalement à concevoir une architecture capable de structurer les données d'offres et de profils, d'exploiter les référentiels métiers et formations, puis de produire des recommandations plus explicables que les approches fondées uniquement sur des mots-clés. Plus spécifiquement, il s'agit de normaliser les données disponibles, d'aligner les métiers et compétences avec les référentiels ESCO, MEPC et NCF, d'adapter un modèle d'embeddings au domaine emploi-compétences, de construire un graphe de connaissances sous Neo4j et de poser les bases d'un moteur de recommandation hybride intégrant similarité sémantique, relations de graphe et analyse du *skill gap*.

Ce travail répond à un double enjeu. Sur le plan scientifique et méthodologique, il cherche à montrer comment les modèles de représentation sémantique, les bases

vectorielles et les graphes de connaissances peuvent être combinés pour traiter un problème réel d'appariement sur le marché du travail. Sur le plan opérationnel, il propose une démarche reproductible susceptible d'aider KmerAI à développer des outils de recommandation, d'orientation et de montée en compétences adaptés aux besoins des candidats, des recruteurs et des acteurs de la formation.

Résumé

Mots-clés : Économétrie, Statistique, Développement...

Abstract

INTRODUCTION GÉNÉRALE

Contexte et justification

Le fonctionnement du marché de l'emploi constitue un déterminant central de la performance économique et sociale d'un pays. Au Cameroun, ce marché reste marqué par des déséquilibres structurels persistants, notamment l'inadéquation entre les compétences disponibles et les besoins des entreprises, la forte informalité et les difficultés d'insertion professionnelle des jeunes. Les données disponibles montrent que le chômage des jeunes demeure une réalité importante, avec un taux estimé à 6,6% en 2024 et 6,5% en 2025 selon les estimations modélisées de l'OIT relayées par la Banque Mondiale et la base FRED. Par ailleurs, plusieurs travaux sur le Cameroun soulignent que le problème ne tient pas uniquement au manque d'emplois, mais aussi à un mismatch entre formation, compétences et exigences du marché.

Dans ce contexte, la question du matching emploi-compétences devient centrale. En théorie, un marché du travail efficient devrait permettre d'associer rapidement les profils disposant des compétences requises aux postes vacants correspondants. En pratique, plusieurs frictions informationnelles persistent : asymétrie d'information entre recruteurs et candidats, faible structuration des profils de compétences, hétérogénéité des offres d'emploi et faiblesse des systèmes d'intermédiation comme le FNE et certains agences de recrutement. Ces dysfonctionnements conduisent à une situation paradoxale où coexistent des postes non pourvus et des chercheurs d'emploi, traduisant une inefficacité allocative du marché du travail.

Par ailleurs, l'essor des technologies numériques et de l'intelligence artificielle ouvre des perspectives nouvelles pour améliorer l'appariement entre offres et profils. Dans cette dynamique, KmerAI se positionne comme une plateforme camerounaise dédiée à la promotion de l'intelligence artificielle, à la vulgarisation de ses usages et à l'accompagnement des étudiants, chercheurs et entreprises dans l'adoption de solutions innovantes adaptées au contexte local. Ses missions consistent notamment à former aux bases de l'IA, à offrir un espace de partage d'expériences et à encourager la colla-

boration autour de cas d'usage appliqués à des secteurs stratégiques tels que la santé, l'agriculture, le transport, l'éducation et le marketing.

Cette orientation est particulièrement pertinente pour le marché de l'emploi camerounais, où les données liées aux offres, aux CV et aux compétences restent souvent dispersées et peu exploitées. Une approche fondée sur l'intelligence artificielle peut permettre d'analyser plus efficacement ces données hétérogènes afin de proposer des recommandations d'emploi plus pertinentes, plus rapides et plus personnalisées.

Problématique

Le marché de l'emploi camerounais est marqué par une forte asymétrie d'information, une hétérogénéité des profils professionnels et des offres d'emploi, ainsi qu'une faible capacité des outils classiques à évaluer les écarts réels de compétences. Les systèmes fondés sur des mots-clés ou des filtres statiques ne parviennent pas à détecter les compétences implicites, les équivalences sémantiques et les trajectoires de montée en compétences.

Par ailleurs, si des référentiels structurants existent notamment la Nomenclature Camerounaise des Métiers (NCM 2013), le référentiel européen ESCO et la Nomenclature Camerounaise des Formations (NCF), leur intégration opérationnelle dans les plateformes et outils d'appariement reste encore limitée. Ils sont peu exploités pour aligner les intitulés locaux de métiers sur des standards internationaux, harmoniser les compétences attendues, qualifier les niveaux et domaines de formation, et rendre les correspondances profil-offre plus transparentes et comparables. Il en résulte un décalage entre la richesse potentielle de ces référentiels et leur usage effectif dans les systèmes d'information dédiés à l'emploi et à l'orientation.

Dans ce contexte, la question centrale n'est plus seulement d'apparier un profil à une offre, mais de concevoir un système capable (i) d'exploiter conjointement les référentiels NCM, ESCO et NCF pour structurer les métiers, les compétences et les formations, (ii) de mesurer finement le *skill gap* entre un candidat et un poste cible, et (iii) de proposer un parcours de montée en compétences compatible avec le niveau, le domaine de formation et les trajectoires de qualification du candidat. La question principale de recherche peut ainsi être formulée comme suit :

Comment concevoir un système de recommandation hybride, fondé sur les LLMs, les graphes de connaissances, les embeddings vectoriels et le filtrage collaboratif, capable de mesurer le skill gap entre un profil et une offre d'emploi, puis de recommander un parcours de montée en compétences cohérent avec la nomenclature des formations, le niveau du candidat et le métier visé pour les membres de la communauté KmerAi ?

Cette question générale se décline en plusieurs questions de recherche sous-jacentes :

- **Q1** : Comment identifier et formaliser, dans le contexte camerounais, les compétences, métiers, formations et relations pertinentes pour un matching emploi compétences fiable et opérationnel à partir des référentiels NCM, ESCO et NCF ?
- **Q2** : Dans quelle mesure l'intégration d'un graphe de connaissances améliore-t-elle la qualité et la précision du matching par rapport aux approches purement vectorielles ?

Objectifs de l'étude

L'objectif général de cette étude est de concevoir et d'évaluer un système de recommandation hybride emploi-compétences capable d'aider à la décision dans le recrutement et la montée en compétence. Le système vise à rapprocher les profils de candidats et les offres d'emploi en combinant la recherche sémantique, les référentiels métiers et formations, le graphe de connaissances et une logique explicable de *skill gap*. Il ne s'agit donc pas seulement de retrouver les offres textuellement proches d'un profil, mais de produire un verdict opérationnel : identifier les offres immédiatement accessibles, distinguer celles qui nécessitent une montée en compétence, signaler les profils à conserver en vivier et expliciter les compétences à développer.

Plus spécifiquement, il sera question de :

- normaliser et aligner les offres d'emploi et profils candidats avec les référentiels ESCO, MEPC et NCF afin de produire des variables structurées intitulé de poste, compétences, niveau NCF, secteur, localisation exploitables par le système de matching ;
- adapter par *fine-tuning* contrastif un modèle *SentenceTransformer* au domaine emploi-compétences à partir de paires offre-profil, puis indexer les représentations produites dans une base vectorielle pgvector pour le retrieval sémantique ;

- construire un graphe de connaissances sous Neo4j modélisant les relations entre candidats, offres, compétences, métiers, secteurs et niveaux NCF, et concevoir un score hybride combinant similarité sémantique, couverture de compétences, compatibilité NCF et alignement sectoriel ;
- orchestrer, via un workflow *LangGraph* multi-étapes, l'ensemble du pipeline de recommandation retrieval sémantique, enrichissement graphe, analyse de *skill gap*, scoring hybride, critique et replanification afin de produire des verdicts interprétables (profil prêt, montée en compétence, vivier, hors cible) accompagnés d'une trajectoire de formation, et d'évaluer la qualité du retrieval par les métriques Recall@K, NDCG@K, MRR@K et MAP@K.

Intérêt de l'étude

L'intérêt d'une telle approche est renforcé par les limites des méthodes classiques de recrutement, souvent fondées sur des critères statiques ou sur des mots-clés, qui ne captent pas toujours la richesse des parcours et des compétences. En combinant différentes logiques de recommandation, un système hybride peut mieux prendre en compte les dimensions explicites et implicites des profils candidats, tout en améliorant la qualité des correspondances proposées. Cela rejoint la mission de KmerAI, qui vise à favoriser l'appropriation locale de l'IA pour résoudre des problèmes concrets du développement camerounais.

Hypothèses de recherche

Afin de répondre à la problématique, cette étude repose sur les hypothèses suivantes.

- **H1** : L'intégration conjointe de la NCM, d'ESCO et de la NCF dans un référentiel unifié métiers-compétences-formations améliore la structuration et les performances d'appariement profil-offre, par rapport à une approche sans ces référentiels ;
- **H2** : La modélisation du marché de l'emploi sous forme de graphe de connaissances (Neo4j), incluant explicitement les niveaux et domaines de formation de la NCF, améliore la qualité du matching par rapport à une approche purement vectorielle ;

- **H3** : Le *fine-tuning* d'un modèle de représentation sémantique sur le domaine emploi-compétences améliore la qualité des embeddings et la similarité sémantique entre profils et offres, par rapport au même modèle non adapté ;
- **H4** : La combinaison d'un RAG vectoriel, d'un GraphRAG et d'un score de recommandation hybride améliore la pertinence et l'explicabilité des recommandations de *skill gap* et de parcours de montée en compétences, notamment lorsque ces parcours sont contraints par la nomenclature des formations.

Méthodologie de recherche

La démarche adoptée s'inscrit dans un cadre d'ingénierie des données combinant structuration des connaissances et modélisation hybride. Elle repose sur la collecte et la normalisation des données issues d'offres d'emploi sur les différents sites web et de profils, alignées sur la NCM 2013 afin d'assurer la cohérence sémantique. Un graphe de connaissances est ensuite construit sous Neo4j en intégrant les référentiels ESCO et NCM, à partir d'un processus d'extraction automatique de triplets à partir des données textuelles. Parallèlement, les descriptions sont vectorisées et indexées dans un espace sémantique permettant la mesure de similarité. Le moteur de recommandation combine ces différentes sources d'information à travers un score hybride intégrant similarité structurelle, similarité sémantique et signal collaboratif. Le système permet ainsi d'identifier les écarts de compétences entre profils et offres, et de générer des recommandations sous forme de trajectoires d'apprentissage. L'évaluation repose sur des métriques standard telles que la précision@K, le rappel et le NDCG, ainsi que sur l'analyse de la cohérence des recommandations générées.

Plan du travail

Le mémoire est structuré en trois parties. La première présente le cadre théorique relatif aux systèmes de recommandation, aux modèles de langage et aux graphes de connaissances. La deuxième détaille l'approche méthodologique, incluant la modélisation du graphe et la conception du système hybride. La troisième analyse les résultats obtenus et discute les performances du modèle ainsi que ses implications dans le contexte du marché du travail camerounais.

Première partie

Cadre théorique et conceptuel

Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation

Ce chapitre situe le mémoire dans la littérature sur l'appariement emploi-compétences, les systèmes de recommandation, l'intelligence artificielle générative, les graphes de connaissances et la génération augmentée par récupération. Il ne vise pas à reprendre l'ensemble des raisonnements des ouvrages ou articles consultés. Il sélectionne les résultats qui éclairent directement le thème du stage : concevoir un système capable de recommander des offres, d'identifier des écarts de compétences et de produire des explications fondées sur des données structurées et non structurées.

La revue est guidée par une hypothèse centrale : le matching emploi-compétences n'est pas un simple problème de similarité textuelle. Il combine des frictions du marché du travail, une forte variabilité du vocabulaire, des contraintes de qualification, des relations métier-compétence-formation et un besoin d'explicabilité. Cette nature composite explique pourquoi la littérature pertinente relève à la fois de l'économie du travail, de la recommandation hybride, de la recherche sémantique, des graphes de connaissances et de la génération contrôlée.

1.1 Concepts clés mobilisés dans le mémoire

Avant d'entrer dans la revue de littérature, il est nécessaire de stabiliser les concepts utilisés dans la suite du mémoire. Cette clarification évite une confusion fréquente : employer indifféremment IA générative, LLM, RAG, GraphRAG, moteur de recommandation et chatbot. Dans ce travail, ces termes désignent des fonctions complémentaires, mais non interchangeables.

Matching emploi-compétences.

Le matching emploi-compétences désigne le processus qui consiste à rapprocher un profil candidat d'une offre, d'un métier ou d'une trajectoire de formation à partir de caractéristiques observables : compétences, niveau de formation, expérience, secteur, localisation et objectif professionnel. Il est traité ici comme un problème d'appariement économique, mais aussi comme un problème de représentation des connaissances.

Système de recommandation.

Un système de recommandation est un dispositif algorithmique qui classe des éléments selon leur pertinence probable pour un utilisateur ou un cas d'usage [1]. Dans ce mémoire, les éléments recommandés peuvent être des offres d'emploi, des compétences, des métiers ou des pistes de formation. La pertinence ne renvoie donc pas seulement à une préférence ; elle renvoie aussi à une compatibilité professionnelle explicable.

Skill gap.

Le *skill gap*, ou écart de compétences, correspond à la différence entre les compétences déjà détenues par un candidat et celles attendues pour une offre ou un métier cible. Il permet de dépasser une logique binaire compatible/non compatible : un candidat peut être proche d'un poste tout en nécessitant une montée en compétences.

Embedding et recherche sémantique.

Un embedding est une représentation vectorielle d'un texte, d'un profil, d'une offre ou d'une compétence. La recherche sémantique utilise cette représentation pour retrouver des objets proches en sens, même lorsque les formulations diffèrent. Elle répond à un problème récurrent des données d'emploi : des expressions comme « data analyst », « analyste de données » et « reporting statistique » peuvent désigner des zones de compétences voisines.

Base vectorielle pgvector.

pgvector désigne l'extension PostgreSQL utilisée pour stocker et rechercher des embeddings [2]. Dans ce mémoire, elle sert de couche de recherche rapide pour retrouver les offres, métiers, compétences ou documents proches d'une requête. Son rôle

est de présélectionner des candidats à la recommandation. Elle ne suffit cependant pas à décider seule, car un bon score vectoriel ne garantit pas que le niveau NCF, l'expérience ou les compétences obligatoires soient compatibles.

Graphe de connaissances Neo4j.

Un graphe de connaissances représente des entités et leurs relations : candidat, offre, compétence, métier, secteur, niveau, formation et référentiel [3]. Dans ce mémoire, Neo4j sert à représenter les relations explicites entre ces entités. Il permet de répondre à des questions que la seule similarité vectorielle ne traite pas correctement : quelles compétences une offre exige-t-elle ? quelles compétences le candidat possède-t-il ? quel niveau de formation est demandé ? quels chemins relient un métier à une formation ?

IA générative, LLM et prompting.

L'intelligence artificielle générative désigne les modèles capables de produire du contenu, notamment du texte. Un LLM est un grand modèle de langage entraîné sur de grands corpus textuels et capable de générer, reformuler ou synthétiser des réponses. Dans ce mémoire, le LLM n'est pas le moteur de vérité du système. Il intervient comme générateur contrôlé : il reçoit un contexte construit à partir de pgvector, Neo4j et des documents, puis rédige une réponse en français. Le prompting désigne l'ensemble des consignes données au modèle pour encadrer son comportement, limiter les inventions et structurer la réponse.

RAG, GraphRAG et Agentic RAG.

La RAG combine récupération d'information et génération : le système recherche d'abord des éléments pertinents, puis les transmet au LLM pour produire une réponse [4]. Le GraphRAG étend cette logique en ajoutant des faits et chemins issus d'un graphe de connaissances. L'Agentic RAG ajoute une couche d'orchestration : l'agent analyse l'intention, choisit les outils, récupère les données, construit le contexte, génère la réponse et peut demander une révision si le contexte est insuffisant. Dans ce mémoire, cette orchestration est assurée par LangGraph.

Critic et traçabilité.

Le critic est un mécanisme de contrôle qui examine si la réponse générée semble suffisamment couverte par le contexte récupéré. Il ne remplace pas une validation humaine, mais il réduit le risque d'une réponse fluide et insuffisamment fondée. La traçabilité désigne la capacité à afficher les outils mobilisés, les sources consultées et le chemin suivi par le workflow. Elle est indispensable dans un système d'aide à la décision, car une recommandation professionnelle doit être vérifiable.

1.2 Fondements économiques et problématique d'appariement

1.2.1 Marché du travail, recherche d'emploi et frictions

Le problème traité dans ce mémoire relève d'abord de l'économie du travail. Les modèles de recherche et d'appariement montrent que le chômage peut coexister avec des postes vacants lorsque la rencontre entre travailleurs et entreprises est coûteuse, lente ou imparfaitement informée [5, 6]. Cette lecture est importante : un système de recommandation emploi-compétences n'est pas seulement un outil informatique ; c'est un dispositif d'intermédiation qui cherche à réduire les coûts de recherche et à améliorer la qualité des mises en relation.

La littérature sur l'asymétrie d'information renforce ce diagnostic. Akerlof montre que l'incertitude sur la qualité peut dégrader les échanges lorsque les acteurs ne disposent pas de signaux fiables [7]. Sur le marché de l'emploi, cette asymétrie apparaît des deux côtés : le recruteur ne voit pas toujours les compétences réelles du candidat, et le candidat ne connaît pas toujours les exigences effectives du poste. La théorie du signalement de Spence explique alors le rôle des diplômes, certifications et expériences comme signaux observables [8]. Mais ces signaux restent incomplets lorsque les métiers évoluent rapidement, notamment dans les domaines numériques où les compétences techniques sont souvent décrites par des formulations instables.

1.2.2 Capital humain, tâches et inadéquation des compétences

La théorie du capital humain rappelle que la formation et l'expérience sont des investissements productifs [9]. Toutefois, la possession d'un capital humain ne garantit

pas l'appariement. Encore faut-il que les compétences acquises soient visibles, comparables et alignées sur les compétences demandées. Autor montre que le progrès technique transforme surtout les tâches composant les métiers, et non uniquement les intitulés d'emploi [10]. Pour un système de recommandation, cela impose de descendre au niveau des compétences et des tâches plutôt que de s'arrêter aux titres de poste.

L'apport de cette littérature est de montrer que l'inadéquation emploi-compétences relève autant de la circulation imparfaite de l'information que d'un manque de formation. Sa limite, pour notre objet, est qu'elle ne fournit pas directement les outils de traitement des données nécessaires pour extraire, normaliser, comparer et expliquer les compétences. La section suivante examine donc les systèmes de recommandation comme réponse algorithmique à ce problème d'appariement.

1.3 Systèmes de recommandation et person-job fit

1.3.1 Approches par contenu, collaboratives et hybrides

Les systèmes de recommandation cherchent à classer des items selon leur pertinence probable pour un utilisateur. La littérature distingue classiquement les approches fondées sur le contenu, le filtrage collaboratif et les approches hybrides [11, 1]. Dans le contexte de ce mémoire, l'utilisateur peut être un candidat, un conseiller emploi ou une plateforme ; les items peuvent être des offres, compétences, métiers ou formations.

Les approches fondées sur le contenu utilisent les caractéristiques explicites des items et des profils. Elles sont adaptées au matching emploi-compétences parce que les offres et les candidats possèdent des descriptions textuelles, des secteurs, des niveaux et des compétences. Pazzani et Billsus montrent que ces systèmes exploitent les attributs des contenus pour produire des recommandations personnalisées [12]. Leur avantage est l'interprétabilité ; leur limite est la dépendance au vocabulaire observé.

Le filtrage collaboratif exploite les interactions passées entre utilisateurs et items. Les approches de factorisation matricielle ont fortement structuré ce domaine en ap-

prenant des facteurs latents à partir des matrices d'interactions [13]. Les modèles neuronaux ont ensuite généralisé cette logique, par exemple avec le Neural Collaborative Filtering [14]. Ces méthodes sont puissantes lorsque l'on dispose d'un historique dense : clics, candidatures, recrutements, évaluations ou parcours de formation. Dans le cas étudié ici, cette hypothèse est fragile, car les données d'interaction restent limitées et de nombreux profils se trouvent en situation de démarrage à froid.

Les approches hybrides combinent plusieurs sources de signal pour compenser les limites propres à chaque famille. Burke montre que l'hybridation peut améliorer la robustesse en associant contenu, collaboration, connaissances et règles métier [15]. Cette conclusion prépare directement le choix d'un système hybride : les signaux textuels, relationnels et métier doivent être séparés puis combinés, car chacun ne capture qu'une partie de la compatibilité.

1.3.2 Spécificité du person-job fit

La recommandation d'emploi se distingue de la recommandation de films ou de produits. Une offre n'est pas seulement un item susceptible de plaire ; elle impose des contraintes de qualification, d'expérience, de localisation, de disponibilité et de compétences. La littérature récente sur le *person-job fit* insiste sur cette double sélection : le candidat doit être intéressé par l'offre, mais l'employeur doit aussi pouvoir considérer le profil comme compatible [16]. La recommandation devient donc un problème bilatéral.

Cette spécificité change l'interprétation des scores. Dans le commerce électronique, une recommandation peut viser la probabilité de clic ou d'achat. Dans l'emploi, une recommandation doit distinguer au moins trois situations : le candidat est immédiatement compatible ; le candidat est proche mais nécessite une montée en compétences ; le candidat est trop éloigné du poste cible. La notion de *skill gap* devient alors centrale.

Les travaux récents mobilisant les graphes et les LLM dans les ressources humaines vont dans ce sens. HRGraph propose par exemple d'exploiter des graphes de connaissances enrichis par LLM pour la recommandation d'emploi [17]. D'autres travaux s'intéressent à la prédiction des compétences et à la construction de graphes de talents

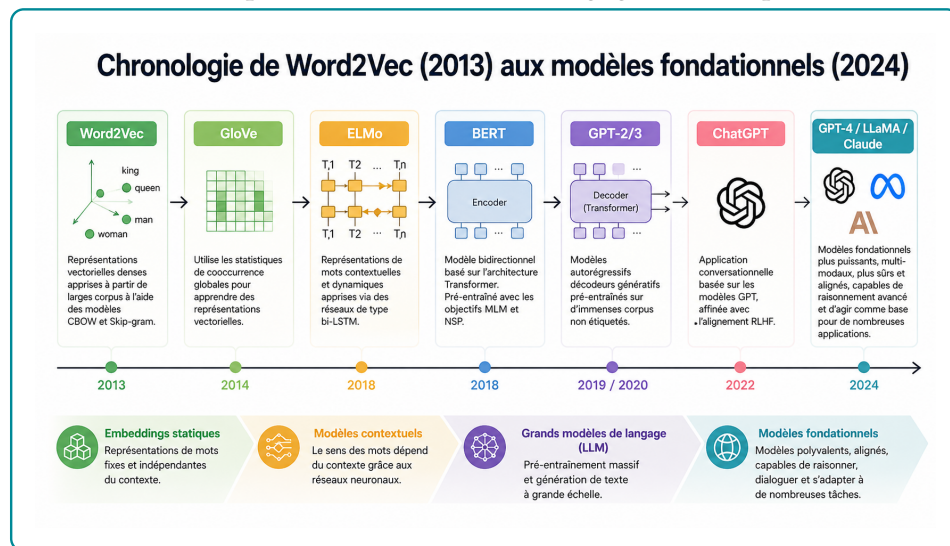
pour les ressources humaines [18]. Ces contributions confirment l'intérêt des graphes pour représenter les relations entre compétences, métiers et profils, mais elles ne résolvent pas automatiquement la question de l'ancrage local : nomenclatures camerounaises, données réelles de plateforme, contraintes de formation et explication opérationnelle.

1.4 IA générative et connaissances

1.4.1 Grands modèles de langage et génération contrôlée

Les grands modèles de langage ont transformé le traitement automatique du langage naturel en permettant des tâches de génération, synthèse, reformulation et raisonnement assisté. Leur rupture technique vient principalement de l'architecture Transformer proposée par Vaswani et al., fondée sur le mécanisme d'attention [19]. Avant cette rupture, les représentations textuelles étaient passées des sacs de mots aux embeddings denses comme Word2Vec, qui apprennent des vecteurs capturant des régularités sémantiques et syntaxiques [20]. Les mécanismes d'attention introduits en traduction neuronale ont ensuite permis aux modèles de pondérer dynamiquement les parties pertinentes d'une séquence [21].

Figure 1.1 – Évolution simplifiée des modèles de langage et des représentations textuelles



Source : Figure du dossier du projet, synthèse pédagogique de l'évolution des modèles de langage

Deux grandes familles de modèles sont utiles pour ce mémoire. La première regroupe les modèles de représentation, comme BERT et Sentence-BERT. BERT apprend des représentations contextuelles bidirectionnelles adaptées à la compréhension du texte [22]. Sentence-BERT adapte cette logique pour produire des embeddings de phrases comparables efficacement par similarité cosinus [23]. La seconde famille regroupe les modèles génératifs, comme GPT, T5 ou les modèles instruction-tuned. GPT-3 a montré qu'un modèle de grande taille pouvait effectuer des tâches variées à partir de quelques exemples dans le prompt [24]. T5 a proposé une formulation unifiée des tâches NLP comme transformation texte-vers-texte [25]. L'apprentissage par retour humain a ensuite amélioré la capacité des modèles à suivre des instructions [26].

L'acquis de cette littérature est considérable : les LLM savent reformuler, synthétiser et produire des explications lisibles. Sa limite, pour notre problématique, tient au statut de la vérité : un modèle génératif peut produire une réponse plausible sans être fondée sur les données locales. Dans un système d'emploi, cette limite impose une génération contrôlée par des sources récupérées.

1.4.2 Embeddings, recherche sémantique et bases vectorielles

Après la clarification conceptuelle, l'enjeu est de savoir dans quelles conditions les représentations vectorielles sont fiables pour la recherche d'information. Les benchmarks montrent que les modèles d'embeddings doivent être évalués sur des tâches variées, car un bon modèle généraliste peut être insuffisant dans un domaine spécialisé [27, 28]. Cette observation justifie l'adaptation d'un modèle SentenceTransformer au vocabulaire emploi-compétences.

La recherche sémantique résout une partie du problème de vocabulaire, mais elle reste une approximation. Deux textes proches peuvent cacher une incompatibilité : niveau insuffisant, compétence obligatoire absente, localisation incompatible ou métier trop éloigné. La similarité vectorielle doit donc être traitée comme un signal de présélection, non comme une décision finale.

La littérature sur la recherche approximative de plus proches voisins montre par ailleurs que le passage à l'échelle exige des structures d'indexation efficaces. HNSW fournit un cadre robuste pour la recherche vectorielle approximative [29], tandis

que FAISS a popularisé la recherche vectorielle à grande échelle [30]. Dans ce mémoire, l'utilisation de PostgreSQL avec pgvector répond à une logique pragmatique : conserver les embeddings dans une base relationnelle exploitable, interrogeable et intégrable au reste du pipeline [2].

1.4.3 Graphes de connaissances et recommandation relationnelle

Hogan et al. montrent que les graphes de connaissances servent à structurer des informations hétérogènes, à rendre explicites des relations sémantiques et à soutenir des tâches de recherche ou de raisonnement [3]. Dans le domaine emploi-compétences, cette représentation complète la recherche vectorielle : elle rend visibles les relations entre candidats, offres, compétences, métiers, formations et niveaux.

La recommandation par graphe a également connu des avancées importantes. Les graph neural networks ont fourni des outils pour propager de l'information dans des graphes [31, 32]. LightGCN a montré que, dans la recommandation collaborative, une architecture simplifiée de propagation sur graphe pouvait être très efficace [33]. Ces travaux établissent l'intérêt de la structure relationnelle pour la recommandation.

Cependant, le présent mémoire ne cherche pas principalement à entraîner un GNN. Le choix méthodologique est différent : utiliser un graphe de connaissances comme support d'explication, de contraintes et de calcul du skill gap. Ce choix est plus adapté au contexte disponible. Les données d'interaction ne sont pas assez riches pour justifier un modèle collaboratif profond, tandis que les relations métier-compétence-formation sont directement interprétables.

1.5 RAG, GraphRAG et orchestration agentique

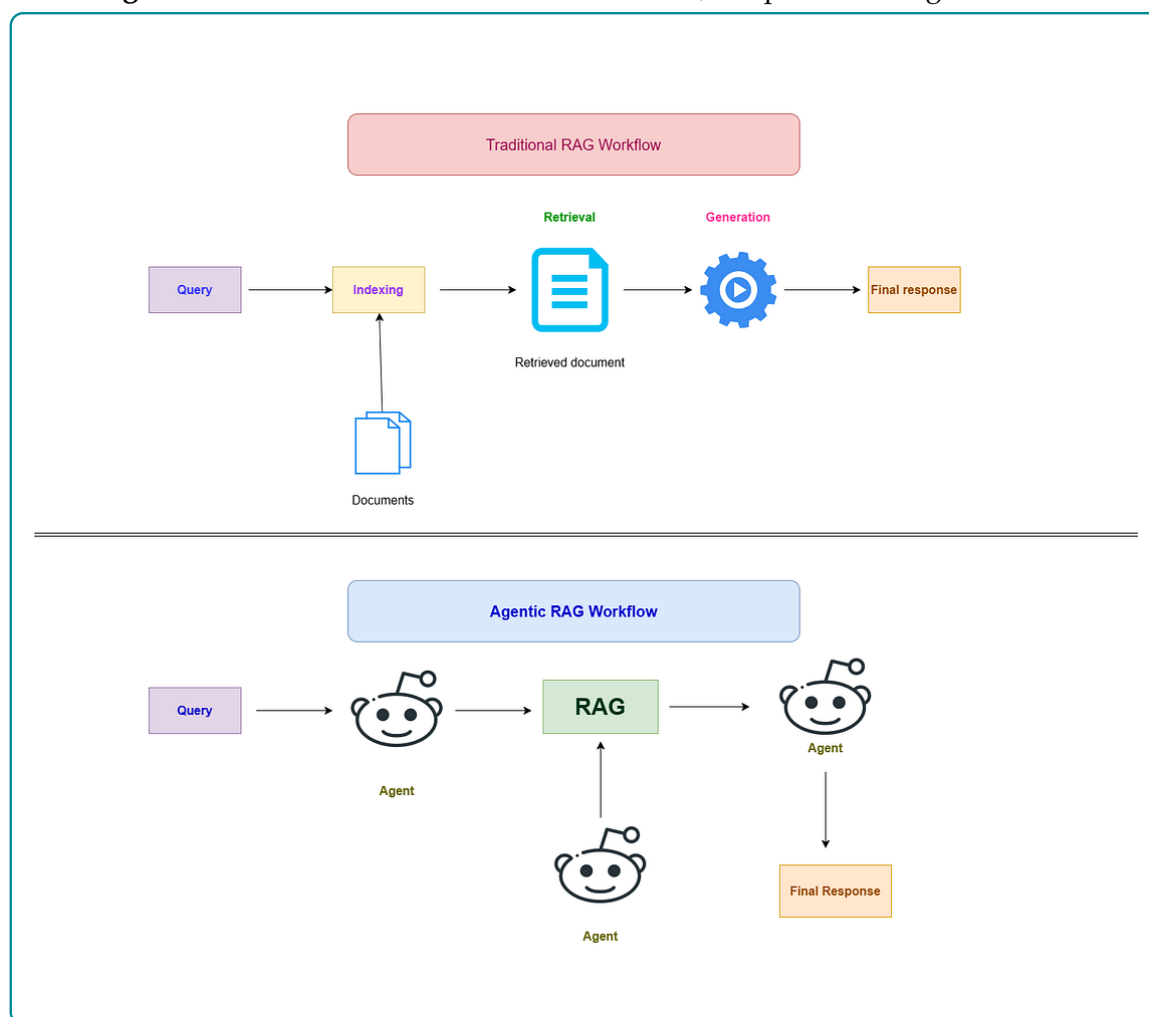
1.5.1 De la RAG au GraphRAG

La génération augmentée par récupération, ou RAG, combine une étape de recherche d'information et une étape de génération. Lewis et al. montrent que l'accès à une mémoire externe améliore les tâches intensives en connaissances, car le modèle ne

dépend plus uniquement de ses paramètres internes [4]. Les synthèses récentes sur la RAG insistent sur les mêmes enjeux : qualité de la récupération, actualisation des connaissances, évaluation de la factualité et articulation entre retriever et générateur [34].

Dans une RAG classique, le système récupère des documents proches d'une requête, puis les transmet au LLM. Cette architecture est utile, mais elle reste centrée sur des passages textuels. Pour le matching emploi-compétences, le contexte doit aussi intégrer des relations : compétences possédées, compétences requises, niveau de formation, métier cible, secteur et formation possible. Le GraphRAG étend alors la récupération vers des informations structurées par graphe.

Figure 1.2 – Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG



Source : Figure du dossier du projet, inspirée des architectures Agentic RAG et des travaux sur agents outillés

La figure 1.2 illustre cette évolution. Une RAG simple suit une chaîne relativement fixe : requête, récupération, génération. Une architecture GraphRAG ajoute une couche relationnelle. Une architecture agentique introduit une capacité de planification et de contrôle : le système peut choisir un outil, observer le résultat, élargir le contexte et réviser la réponse.

1.5.2 Agentic RAG, critic et traçabilité

Les travaux ReAct montrent l'intérêt d'alterner raisonnement et action outillée dans les modèles de langage [35]. Self-RAG va plus loin en introduisant une logique d'auto-réflexion pour décider quand récupérer de l'information et comment critiquer la réponse [36]. Pour ce mémoire, ces travaux ne doivent pas être interprétés comme une autorisation à laisser un agent décider librement. Ils justifient plutôt une orchestration contrôlée : classer l'intention, choisir les sources, récupérer les preuves, construire le contexte, générer, puis vérifier si la réponse est suffisamment fondée.

Le manque de la littérature est ici opérationnel. Beaucoup de travaux décrivent des architectures générales, mais la fiabilité dépend de détails concrets : qualité des chunks, schéma du graphe, robustesse des requêtes, traçabilité des sources, gestion des cas sans résultat et critères de révision. C'est pourquoi l'architecture retenue ajoute un critic et des traces d'exécution. Le LLM produit la réponse finale, mais les données récupérées et les outils appelés doivent rester observables.

1.6 Évaluation, acquis de la littérature et positionnement du mémoire

1.6.1 Évaluer la performance et la factualité

L'évaluation d'un système emploi-compétences ne peut pas se limiter à la fluidité de la réponse générée. Elle doit d'abord mesurer la qualité de la récupération d'information : le système retrouve-t-il les offres, compétences ou documents réellement pertinents ? La littérature en recherche d'information utilise pour cela des métriques

de classement comme Precision@k, Recall@k, MRR et NDCG [27, 28]. Ces métriques supposent l'existence d'un ensemble de résultats pertinents, noté $Rel(q)$, pour une requête q , et d'une liste classée de résultats retournés par le système.

La **Precision@k** mesure la proportion de résultats pertinents parmi les k premiers résultats retournés. Si $R_k(q)$ désigne l'ensemble des k premiers éléments proposés pour la requête q , alors :

$$Precision@k(q) = \frac{|R_k(q) \cap Rel(q)|}{k}. \quad (1.1)$$

Cette métrique répond à la question suivante : parmi les premières recommandations affichées, combien sont réellement utiles ? Dans un système emploi-compétences, une Precision@5 élevée signifie que les cinq premières offres proposées contiennent peu de résultats non pertinents. Sa limite est qu'elle ne mesure pas si le système a retrouvé tous les bons résultats disponibles.

Le **Recall@k** mesure au contraire la proportion des éléments pertinents retrouvés dans les k premiers résultats :

$$Recall@k(q) = \frac{|R_k(q) \cap Rel(q)|}{|Rel(q)|}. \quad (1.2)$$

Cette métrique répond à la question : parmi toutes les offres ou compétences pertinentes, quelle part le système a-t-il réussi à faire remonter ? Elle est importante pour ne pas éliminer trop tôt des offres utiles. En recommandation emploi, un faible Recall@k peut signifier que le moteur rate des opportunités pertinentes pour le candidat.

Le **MRR** (*Mean Reciprocal Rank*) évalue la position du premier résultat pertinent. Pour une requête q , si $rank_q$ est le rang du premier résultat pertinent, alors le score associé est :

$$RR(q) = \frac{1}{rank_q}. \quad (1.3)$$

Sur un ensemble de N requêtes, le MRR est :

$$MRR = \frac{1}{N} \sum_{q=1}^N \frac{1}{rank_q}. \quad (1.4)$$

Cette métrique est utile lorsque l'on veut savoir si le système place rapidement au moins une bonne réponse. Dans une interface de recommandation, cela correspond à une logique pratique : l'utilisateur regarde souvent les premiers résultats avant de parcourir toute la liste.

Le **NDCG@k** (*Normalized Discounted Cumulative Gain*) tient compte non seulement de la présence des résultats pertinents, mais aussi de leur ordre et éventuellement de leur degré de pertinence. Si rel_i désigne le niveau de pertinence du résultat placé au rang i , alors :

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}. \quad (1.5)$$

Le score est ensuite normalisé par le meilleur classement possible, noté $IDCG@k$:

$$NDCG@k = \frac{DCG@k}{IDCG@k}. \quad (1.6)$$

Cette métrique est particulièrement adaptée lorsque tous les résultats pertinents ne se valent pas. Par exemple, une offre parfaitement alignée avec les compétences du candidat devrait être mieux classée qu'une offre seulement partiellement compatible.

Dans une architecture RAG, l'évaluation ne s'arrête pas au classement des résultats. Elle doit aussi mesurer la qualité de la réponse générée à partir du contexte récupéré. RAGAS distingue notamment la fidélité au contexte, la pertinence de la réponse, la qualité de la récupération et l'usage des sources [37]. La **fidélité** vérifie si les affirmations de la réponse sont supportées par les éléments récupérés. La **pertinence de réponse** vérifie si la réponse traite effectivement la question posée. La **couverture des sources** vérifie si le contexte contient suffisamment d'informations pour répondre. Enfin, la **traçabilité** vérifie si les preuves ou citations sont identifiables.

Ces critères sont moins simples à formaliser que Precision@k ou Recall@k, car ils portent sur du langage naturel. Des approches comme G-Eval utilisent des LLM

comme juges pour approximer l'évaluation humaine [38]. Cette pratique doit rester prudente : un juge automatique peut aider à comparer des variantes de système, mais il ne remplace pas une validation métier. Dans ce mémoire, l'évaluation doit donc combiner des métriques de retrieval pour pgvector et Neo4j, des critères de factualité pour la réponse générée, et une interprétation professionnelle de l'utilité des recommandations.

1.6.2 Acquis, manques et contribution du mémoire

La littérature fournit plusieurs acquis solides. Les modèles d'appariement du marché du travail expliquent pourquoi les frictions informationnelles justifient des dispositifs d'intermédiation. Les systèmes de recommandation montrent que l'hybridation est souvent nécessaire lorsque les données sont hétérogènes. Les Transformers et les embeddings permettent de dépasser la recherche lexicale. Les graphes de connaissances apportent une structure relationnelle utile pour expliquer les recommandations. La RAG et l'Agentic RAG permettent enfin de produire des réponses en langage naturel à partir de contextes récupérés.

Mais ces acquis laissent des manques importants. Premièrement, une proximité sémantique n'est pas une preuve de compatibilité professionnelle. Deuxièmement, les modèles collaboratifs exigent des historiques d'interaction souvent absents dans les plateformes émergentes. Troisièmement, un LLM peut formuler une réponse convaincante sans être fidèle aux données. Quatrièmement, les graphes RH proposés dans la littérature ne sont pas automatiquement adaptés aux nomenclatures camerounaises ni aux données locales de KmerAI.

Le domaine de recherche du mémoire se situe donc à l'intersection de la recommandation hybride, des graphes de connaissances et de la génération contrôlée. La contribution n'est pas de proposer un nouveau Transformer ou un nouvel algorithme de factorisation matricielle. Elle consiste à concevoir et évaluer une architecture intégrée pour le matching emploi-compétences, dans laquelle chaque brique remplit une fonction distincte : présélection sémantique, contrôle relationnel, calcul d'écart de compétences, génération contextualisée et vérification de la réponse.

Cette position est volontairement prudente. Le système ne prétend pas remplacer

l'expertise humaine du conseiller emploi ou du recruteur. Il vise à rendre l'information plus exploitable : quelles offres sont proches du profil, quelles compétences expliquent la recommandation, quels écarts subsistent, et quelles pistes de formation peuvent réduire ces écarts. C'est cette articulation entre performance algorithmique, explicabilité et utilité professionnelle qui guide les choix méthodologiques et l'implémentation présentés dans les chapitres suivants.

Source de données et Approche méthodologique

2.1 Cadre méthodologique et sources de données

2.1.1 Positionnement méthodologique

La présente étude adopte une démarche d'ingénierie appliquée orientée vers la conception, l'intégration et l'évaluation d'un artefact numérique d'aide à la décision. L'objet étudié n'est donc pas uniquement un modèle statistique pris isolément, mais un système complet de recommandation emploi-compétences combinant données structurées, représentations vectorielles, graphe de connaissances, règles métier et génération augmentée par récupération. Ce positionnement est cohérent avec la problématique du mémoire : améliorer l'appariement entre candidats, offres d'emploi, compétences, métiers et trajectoires de formation dans un contexte où l'information est hétérogène, incomplète et dispersée.

La logique méthodologique retenue repose sur quatre principes. Le premier est la *traçabilité* : chaque recommandation doit pouvoir être rattachée à des données observables, à un référentiel ou à une relation explicite du graphe. Le deuxième est l'*hybridation* : aucune source d'information ne suffit seule à résoudre le problème. La similarité sémantique capte les proximités de langage, le graphe encode les relations métier, les règles contrôlent les contraintes de niveau et le LLM sert à formuler une réponse intelligible. Le troisième principe est l'*explicabilité opérationnelle* : le système ne doit pas seulement retourner une liste d'offres, mais justifier le rapprochement, signaler les compétences communes, identifier les écarts et proposer une trajectoire de montée en compétences. Le quatrième principe est le *contrôle génératif* : les réponses produites par le LLM doivent être contraintes par le contexte récupéré afin de réduire le risque d'hallucination, conformément à la logique du RAG [4, 37].

Sur le plan de l'orchestration, le mémoire retient explicitement *LangGraph* comme cadre méthodologique de pilotage du système agentique [39]. Ce choix signifie que le raisonnement applicatif n'est pas laissé à une boucle conversationnelle informelle : il est modélisé comme un graphe d'états, composé de noeuds spécialisés, de transitions contrôlées et d'un état partagé. *LangGraph* est donc utilisé pour organiser le routage entre les briques du système, conserver les traces des outils mobilisés, permettre une révision lorsque le contexte est insuffisant et rendre le workflow auditable.

Ce chapitre précise donc les choix de conception du système. Les résultats d'exécution, les performances mesurées et les exemples de réponses sont volontairement réservés aux chapitres suivants. La méthodologie décrit ici comment les données sont préparées, comment les représentations sont construites, comment les bases pgvector et Neo4j sont articulées, comment le workflow agentique est organisé et comment l'évaluation est conçue.

La figure 2.1 présente l'architecture générale retenue pour la solution. Elle doit être lue comme une carte méthodologique du système plutôt que comme un simple schéma technique. Le point d'entrée est la requête de l'utilisateur. Cette requête n'est pas transmise directement au modèle génératif ; elle est d'abord interprétée par une couche d'orchestration qui détermine le type de besoin exprimé. Cette étape est centrale, car une demande de recommandation candidat-offre, une question sur un référentiel, une interrogation relationnelle sur les compétences ou une question globale ne mobilisent pas les mêmes sources ni les mêmes traitements.

Le workflow distingue ainsi plusieurs familles de traitements. La recommandation candidat s'appuie sur une combinaison de recherche vectorielle, de recherche textuelle et de signaux issus du graphe afin de produire un classement explicable. La recherche référentielle privilégie les fragments documentaires et les métadonnées associées, car l'enjeu est alors de retrouver une information sourcée. Les questions relationnelles exploitent davantage Neo4j, où les liens entre métiers, compétences, niveaux, secteurs et formations permettent de répondre à des questions que la similarité textuelle seule ne suffit pas à résoudre. Les questions globales mobilisent une synthèse plus large, fondée sur des résumés, des communautés ou des agrégations issues de plusieurs sources.

L'intérêt méthodologique de cette architecture est donc l'hybridation contrôlée. PostgreSQL avec pgvector joue le rôle de mémoire sémantique pour retrouver des objets

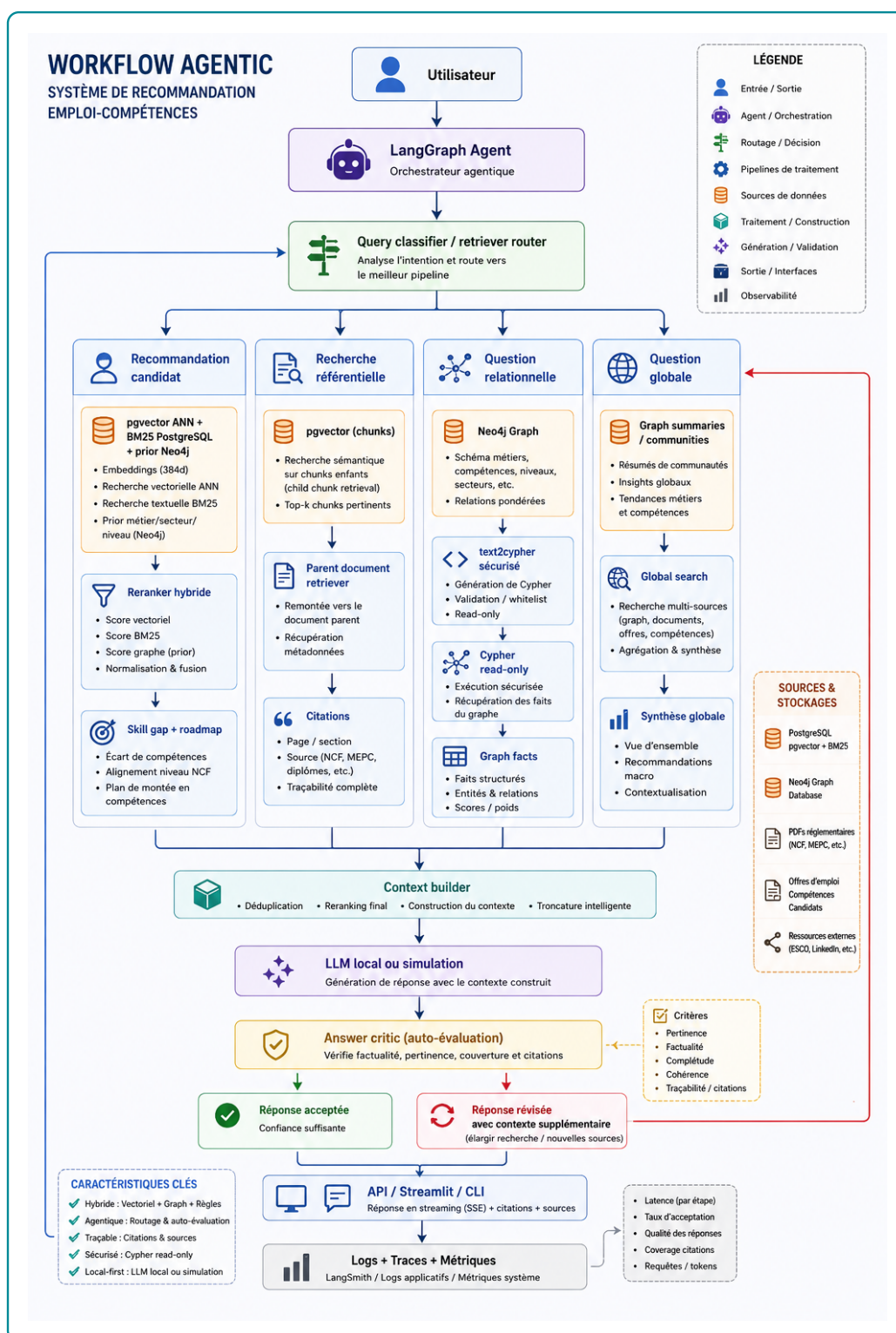


Figure 2.1 – Architecture méthodologique du workflow agentique de recommandation emploi-compétences

Source : Auteur

proches dans l'espace des embeddings. Neo4j joue le rôle de mémoire relationnelle pour vérifier, expliquer et pondérer les liens métier-compétence-formation. Les documents référentiels apportent une base de citation et de traçabilité. Le constructeur de contexte rassemble ensuite ces résultats dans un format exploitable par le LLM, en évitant de lui confier seul la décision de pertinence. La génération intervient donc en aval, après récupération et structuration du contexte.

Enfin, la partie inférieure du schéma rappelle que le système ne s'arrête pas à la production d'une réponse. Un module de critique évalue si la réponse est suffisamment pertinente, factuelle, complète et traçable. Si le contexte est insuffisant, la boucle peut relancer une recherche élargie avant de produire une réponse révisée. Ce choix méthodologique est important pour le mémoire : l'architecture ne cherche pas seulement à générer un texte convaincant, mais à produire une recommandation fondée sur des sources identifiables, des relations vérifiables et des critères métier explicites. Les sections suivantes détaillent chacune de ces briques : préparation des données, construction des embeddings, stockage vectoriel, graphe de connaissances, moteur hybride, orchestration agentique et protocole d'évaluation.

Table 2.1 – Correspondance entre objectifs spécifiques et choix méthodologiques

Objectif spécifique	Choix méthodologique retenu
Normaliser les offres, candidats et référentiels	Construction d’une couche de données standardisée : identifiants stables, champs nettoyés, niveaux de formation, secteurs, compétences et textes d’embedding.
Adapter la recherche sémantique au domaine emploi-compétences	Fine-tuning contrastif d’un modèle SentenceTransformer, puis indexation des embeddings dans PostgreSQL avec pgvector pour le retrieval top-K [23, 2].
Représenter les relations entre métiers, compétences et profils	Construction d’un graphe de connaissances Neo4j reliant candidats, offres, compétences, métiers, secteurs, localisations et référentiels [3, 40].
Produire une recommandation explicable	Score hybride combinant similarité vectorielle, couverture de compétences, compatibilité de niveau et alignement sectoriel, complété par une analyse de <i>skill gap</i> .
Orchestrer la réponse utilisateur	Workflow LangGraph routant la requête vers les outils adaptés : pgvector, Neo4j, recherche documentaire, Text2Cypher, critic et génération finale.
Evaluer la qualité du système	Protocole séparant l’évaluation du retrieval, du graphe, du classement et de la réponse générée.

Source : Auteur

2.1.2 Sources de données et leur rôle dans le système

Le système mobilise deux catégories de données. La première correspond aux données opérationnelles du marché du travail : offres d’emploi et profils de candidats. Ces données décrivent respectivement la demande de travail et l’offre de travail disponible. La seconde catégorie correspond aux référentiels de structuration : ESCO, MEPC et NCF. Ces référentiels permettent d’éviter une approche purement textuelle et de rattacher les intitulés, compétences et niveaux de formation à des nomenclatures interprétables.

Les offres d’emploi apportent les informations relatives aux postes disponibles : intitulé, employeur, secteur, localisation, type de contrat, expérience attendue, niveau d’étude, compétences mentionnées et description détaillée. Les profils candidats dé-

crivent les caractéristiques individuelles : diplôme, niveau d'étude, spécialité, expérience, compétences, mobilité et objectif professionnel. Ces deux sources sont naturellement bruitées : les intitulés ne sont pas normalisés, les compétences peuvent être formulées librement et certaines variables sont incomplètes. La méthodologie doit donc transformer ces données en objets comparables.

ESCO fournit une nomenclature internationale des métiers et compétences. Son intérêt est de disposer d'un vocabulaire structuré et de relations métier-compétence déjà établies. La MEPC permet d'ancrer le système dans la nomenclature camerounaise des métiers. La NCF fournit une structure nationale des niveaux et domaines de formation. L'intégration conjointe de ces sources permet de relier le langage des annonces, les profils des candidats et les classifications institutionnelles.

Table 2.2 – Sources de données mobilisées et utilisation méthodologique

Source	Nature	Utilisation dans la méthode
Offres d'emploi	Données opérationnelles collectées par KmerAI	Normalisation, extraction de compétences, génération de <code>text_to_embed</code> , indexation vectorielle, création des noeuds d'offres et calcul des scores de matching.
Profils candidats	Base locale de demandeurs	Normalisation du niveau, du diplôme, de la mobilité et du métier visé; création des embeddings candidats; analyse de compatibilité avec les offres.
ESCO	Référentiel métiers-compétences	Source de métiers, compétences, groupes ISCO et relations métier-compétence, utilisée pour structurer le graphe et enrichir les correspondances.
MEPC	Nomenclature camerounaise des métiers	Ancrage national des métiers, structuration hiérarchique et alignement partiel avec les groupes internationaux.
NCF	Nomenclature camerounaise des formations	Codification des niveaux et domaines de formation, utilisée pour la compatibilité candidat-offre et l'analyse de montée en compétences.
PDF réglementaires	Documents officiels fragmentés	Constitution de <code>DocChunk</code> pour la recherche référentielle et la citation des sources.

Source : Auteur

2.1.3 Préparation et normalisation des données

La préparation des données constitue la première couche méthodologique du système. Son objectif est de produire des tables propres, stables et exploitables par les deux moteurs de recherche : le moteur vectoriel et le moteur graphe. Cette étape ne se limite pas au nettoyage cosmétique des textes. Elle détermine la qualité des relations construites, la pertinence des embeddings et la fiabilité des recommandations.

Pour les offres d'emploi, la normalisation consiste d'abord à réduire les doublons et les variations de forme. Une annonce peut apparaître plusieurs fois avec des différences mineures de titre, d'employeur ou de localisation. Le traitement doit donc stabiliser les identifiants et conserver les informations utiles sans multiplier artificiellement les observations. Les champs textuels sont ensuite harmonisés : suppression des caractères parasites, standardisation des espaces, séparation des champs multi-valeurs et conservation des informations de contexte. Les compétences mentionnées sont transformées en listes exploitables, tandis que les localisations et secteurs sont séparés en valeurs principales et valeurs secondaires.

Pour les candidats, la normalisation porte sur le diplôme, le niveau d'étude, la spécialité, l'expérience, les compétences, le secteur visé et la mobilité géographique. Une attention particulière est portée au niveau de formation. Lorsque l'information disponible le permet, le niveau est rapproché de la NCF. Ce rapprochement est indispensable parce qu'une recommandation emploi-compétences ne peut pas se fonder uniquement sur la proximité textuelle : une offre peut être sémantiquement proche d'un profil tout en exigeant un niveau de qualification supérieur.

Chaque offre et chaque candidat reçoit enfin un champ `text_to_embed`. Ce champ synthétise les éléments utiles pour la recherche sémantique : intitulé, compétences, secteur, niveau, description et objectif. Le choix d'un texte synthétique répond à une contrainte pratique : les modèles d'embeddings travaillent sur des séquences textuelles, mais les données métier sont partiellement structurées. Le champ `text_to_embed` sert donc d'interface entre les variables métiers et l'espace vectoriel.

2.1.4 Alignement des référentiels métiers et formations

L'alignement des référentiels répond à un besoin central : rendre comparables des données issues de sources différentes. Les annonces utilisent des formulations libres, ESCO propose une taxonomie internationale, la MEPC suit une logique nationale de classification des métiers et la NCF structure les formations. La méthode adoptée est prudente : elle vise un alignement exploitable pour la recommandation, sans prétendre établir une équivalence parfaite entre toutes les nomenclatures.

La MEPC est structurée selon ses niveaux hiérarchiques : grands groupes, sous-groupes et groupes de base. La NCF est organisée autour des niveaux de formation, grands domaines, domaines spécialisés et domaines détaillés. Cette structuration permet de conserver l'information institutionnelle au lieu de la réduire à de simples libellés. Les tables obtenues alimentent ensuite le graphe Neo4j et la base vectorielle.

L'alignement avec ESCO s'appuie sur les codes et les proximités de libellés lorsque ces informations sont disponibles. Cette stratégie est méthodologiquement raisonnable, mais elle doit être interprétée avec prudence. Un même groupe de classification peut regrouper plusieurs réalités professionnelles. L'alignement sert donc à améliorer la couverture et l'interprétabilité, non à produire une vérité définitive sur l'équivalence entre métiers.

2.2 Modélisation sémantique et structuration des connaissances

2.2.1 Choix du modèle d'embeddings et fine-tuning

Le système utilise un modèle SentenceTransformer pour représenter les offres, candidats, métiers, compétences et fragments documentaires dans un espace vectoriel. Ce choix est adapté au problème, car le matching emploi-compétences repose largement sur la similarité sémantique entre textes courts ou semi-structurés : intitulés de postes, compétences, descriptions d'annonces, objectifs professionnels et libellés de référentiels. Les SentenceTransformers ont été conçus pour produire des embeddings directement comparables par similarité cosinus, ce qui les rend adaptés aux tâches de recherche sémantique [23].

Le fine-tuning retenu n'est pas un fine-tuning génératif. Il ne vise pas à apprendre au modèle à rédiger des réponses, mais à adapter l'espace vectoriel au vocabulaire local du domaine emploi-compétences. La logique est contrastive : rapprocher les paires pertinentes et éloigner les paires moins pertinentes. Par exemple, une offre de « data analyst » doit être rapprochée d'un profil mentionnant analyse de données, statistique, Python, SQL ou visualisation, même si les formulations exactes diffèrent. À l'inverse, un profil orienté développement web ne doit pas être considéré comme équivalent à une offre de comptabilité uniquement parce que certaines expressions génériques se ressemblent.

Les paires d'apprentissage sont construites à partir des données préparées. Elles peuvent mobiliser les relations offre-compétence, métier-compétence, candidat-métier et offre-candidat lorsque les signaux disponibles sont suffisamment fiables. Cette étape est critique : un fine-tuning apprend les régularités de ses exemples. Des paires mal construites peuvent dégrader l'espace vectoriel au lieu de l'améliorer. La méthodologie retient donc une construction contrôlée des exemples, puis une évaluation séparée du modèle de base et du modèle adapté.

2.2.2 Indexation vectorielle dans PostgreSQL et pgvector

Les embeddings produits sont stockés dans PostgreSQL avec l'extension pgvector. Ce choix permet de conserver dans un même environnement des représentations vectorielles et des métadonnées relationnelles. pgvector permet l'indexation et la recherche de voisins proches dans PostgreSQL, ce qui facilite l'intégration avec les autres tables du système [2]. Pour accélérer les recherches top-K, la méthode s'appuie sur une logique d'indexation approximative des voisins proches, cohérente avec les travaux sur HNSW [29].

La base vectorielle joue deux rôles. Le premier est le retrieval sémantique : retrouver les offres, métiers, compétences ou documents proches d'une requête. Le second est le préfiltrage pour la recommandation : produire un ensemble de candidats plausibles avant d'appliquer des contraintes plus structurées. Ce préfiltrage est nécessaire, car le graphe de connaissances apporte de l'explicabilité, mais il n'est pas toujours le meilleur premier filtre lorsque la requête est formulée librement.

Chaque résultat vectoriel doit conserver ses métadonnées : identifiant, type d'entité, libellé, source, score de similarité et texte associé. Sans ces métadonnées, la génération finale serait incapable de citer ses sources et le système perdrait sa traçabilité.

2.2.3 Construction du graphe de connaissances sous Neo4j

Le graphe de connaissances représente la couche relationnelle du système. Son rôle est de modéliser explicitement les relations entre les candidats, les offres, les compétences, les métiers, les secteurs, les localisations et les référentiels. Cette représentation est justifiée par la nature même du problème : le matching emploi-compétences n'est pas seulement une question de proximité textuelle, mais une question de relations. Une offre requiert des compétences, un candidat possède des compétences, un métier est classé dans une nomenclature, un niveau de formation conditionne l'accès à certains postes, et une trajectoire de montée en compétences dépend des écarts identifiés.

La méthodologie retient des noeuds correspondant aux entités du système. Les relations décrivent les liens fonctionnels : une offre **REQUIERT** une compétence, un candidat **POSSEDE** une compétence, une offre est rattachée à un secteur ou à une localisation, un métier est classé dans un groupe, et un fragment documentaire provient d'un document référentiel.

Ce graphe sert à trois opérations méthodologiques. Premièrement, il vérifie les correspondances proposées par la recherche vectorielle. Deuxièmement, il explique les recommandations en identifiant les compétences communes et manquantes. Troisièmement, il permet des requêtes relationnelles que la recherche vectorielle gère mal, par exemple : quelles compétences sont associées à un métier donné ? quelles offres exigent une compétence précise ? quelles compétences manquent à un candidat pour viser une famille de métiers ?

2.2.4 Score hybride et logique de skill gap

Le score de recommandation combine plusieurs signaux complémentaires. La similarité sémantique mesure la proximité entre le texte du profil et celui de l'offre. La

couverture de compétences mesure la proportion des compétences requises par l'offre qui sont déjà présentes chez le candidat. La compatibilité de niveau vérifie si le niveau de formation du candidat est cohérent avec le niveau attendu par l'offre. L'alignement sectoriel mesure la cohérence entre le secteur ou le domaine du candidat et celui de l'offre.

La forme générale du score est la suivante :

$$S_{hyb}(c, o) = \alpha S_{sem}(c, o) + \beta S_{graph}(c, o) + \gamma C_{ncf}(c, o) + \delta A_{sect}(c, o),$$

où c désigne un candidat, o une offre, S_{sem} la similarité sémantique, S_{graph} la couverture de compétences issue du graphe, C_{ncf} la compatibilité de niveau de formation et A_{sect} l'alignement sectoriel. Cette formulation traduit un choix méthodologique important : le système ne délègue pas toute la décision au LLM. Le LLM intervient pour synthétiser et expliquer, mais le classement repose sur des signaux calculables et auditaibles.

Les pondérations ne doivent pas être fixées uniquement par intuition. La méthodologie retient donc une calibration statistique par optimisation bayésienne avec Optuna, en utilisant le *Tree-structured Parzen Estimator* comme stratégie de recherche [41]. Dans l'expérience logicielle retenue, le score calibré est volontairement ramené à deux signaux observables dans les sorties d'évaluation : le score sémantique issu de pgvector et le score Neo4j issu de l'enrichissement graphe. L'objectif retenu est le NDCG@10, car cette métrique évalue à la fois la présence des offres pertinentes dans le top 10 et leur ordre dans la liste. Pour chaque essai, l'algorithme propose des poids positifs, les normalise afin que leur somme soit égale à 1, recalcule le score hybride, rerange les offres par candidat, puis mesure le NDCG@10 moyen. La solution retenue est celle qui maximise cette métrique sur l'échantillon d'évaluation.

Cette calibration est volontairement séparée de la génération LLM. Elle concerne seulement le moteur de classement. Elle inclut également un garde-fou : les poids proposés par Optuna sont comparés à des solutions de frontière, par exemple un score purement sémantique ou un score purement fondé sur les compétences. Cette précaution évite de retenir artificiellement un score hybride si les données montrent qu'une composante unique classe mieux les offres selon le critère NDCG@10. La calibration est donc utilisée comme un test empirique des pondérations, non comme

une justification automatique du caractère hybride du système.

L'analyse de *skill gap* complète le score. Elle distingue les compétences acquises, les compétences requises et les compétences manquantes. Cette distinction permet de produire un verdict opérationnel : candidat immédiatement compatible, candidat compatible après montée en compétence, candidat à conserver en vivier ou profil hors cible. Ce verdict est plus utile qu'un simple score numérique, car il transforme la recommandation en aide à la décision.

2.3 Orchestration agentique et génération contrôlée

2.3.1 Architecture Agentic RAG et orchestration LangGraph

L'orchestration du système repose sur un workflow agentique implémentable avec *LangGraph*. Méthodologiquement, *LangGraph* est retenu parce qu'il permet de représenter l'agent comme un graphe d'états plutôt que comme une simple succession de prompts [39]. La requête utilisateur est d'abord analysée afin d'identifier son intention : diagnostic, recommandation candidat, skill gap, recherche référentielle, question relationnelle sur le graphe, orientation métier ou recherche générale. Le système sélectionne ensuite les outils adaptés. Cette étape de routage est essentielle : interroger pgvector, Neo4j ou les documents réglementaires ne répond pas au même besoin.

Le workflow suit une chaîne contrôlée : `analyse_request`, `plan_tools`, `execute_tools`, `build_context`, `generate_final_answer`, puis critique éventuelle de la réponse. Le noeud d'analyse identifie le use-case. Le noeud de planification sélectionne les outils. Le noeud d'exécution appelle les bases et services nécessaires. Le noeud de construction de contexte organise les résultats récupérés. Le noeud de génération produit la réponse finale avec le LLM. Le critic vérifie ensuite si la réponse paraît suffisamment fidèle au contexte ; en cas de faiblesse, le workflow peut élargir le contexte avant de régénérer la réponse.

Ce choix s'inscrit dans la logique de l'Agentic RAG : le système n'est pas un simple prompt envoyé à un modèle, mais une orchestration d'outils spécialisés autour d'un état partagé [35, 36]. Dans le présent travail, l'agent n'est pas conçu comme une entité

autonome non contrôlée. Il est plutôt un routeur raisonné, limité à des outils définis, dont les sorties sont tracées.

2.3.2 Text2Cypher et rôle du LLM

Deux usages du LLM sont distingués. Le premier concerne l'interrogation du graphe en langage naturel. Pour les questions de type `graph_query` ou les recherches générales nécessitant un raisonnement relationnel, la question peut être transformée en requête Cypher par un module Text2Cypher. Le modèle retenu pour cette tâche est `neo4j/text2cypher-gemma-2-9b-it-finetuned-2024v1`. La génération de Cypher est encadrée par le schéma Neo4j : types de noeuds, propriétés et relations autorisées. Cette contrainte est indispensable, car un modèle génératif qui ne connaît pas le schéma réel peut produire des requêtes syntaxiquement plausibles mais inexécutables ou non pertinentes.

Le second usage du LLM concerne la génération de la réponse finale. Le système utilise l'API OpenRouter avec le modèle `openai/gpt-oss-20b:free`. Ce modèle ne doit pas inventer la réponse à partir de ses connaissances générales. Il reçoit un contexte construit par le workflow : résultats pgvector, lignes Neo4j, fragments documentaires et métadonnées de source. Le prompt lui demande de répondre en français, de structurer la réponse et de citer les informations utilisées.

Cette séparation des rôles est méthodologiquement importante. Text2Cypher sert à produire une requête structurée vers le graphe. Le LLM final sert à formuler une synthèse lisible. Dans les deux cas, le modèle est subordonné aux données disponibles et non l'inverse.

2.3.3 Interfaces et traçabilité

Le système prévoit trois modes d'accès : une interface Streamlit pour l'usage conversationnel, une API FastAPI pour l'intégration applicative et une CLI pour les tests de développement. Ces interfaces ne changent pas la logique de recommandation ; elles exposent le même moteur sous des formes différentes. La CLI facilite la vérification rapide des use-cases, Streamlit permet une démonstration interactive et FastAPI pré-

pare l'intégration dans un environnement applicatif comme la plateforme Kmer-AI, les agences de recrutement (FNE et l'ACPE).

La traçabilité est intégrée à plusieurs niveaux. Les traces du workflow indiquent les noeuds traversés et les outils appelés. Les résultats conservent les identifiants des entités, les sources documentaires, les scores et les chemins du graphe. La réponse finale doit citer les éléments utilisés. Cette exigence est décisive dans un système d'aide à l'orientation ou au recrutement : une recommandation sans preuve peut être séduisante, mais elle n'est pas défendable.

2.4 Évaluation prévue, limites et synthèse méthodologique

2.4.1 Protocole d'évaluation prévu

L'évaluation est conçue comme une évaluation en plusieurs étages. Le premier étage porte sur le retrieval vectoriel : le système retrouve-t-il les offres, compétences, métiers ou documents attendus dans les premiers résultats? Les métriques retenues sont notamment Precision@K, Recall@K, MRR@K, MAP@K et NDCG@K, couramment utilisées pour les tâches de recherche d'information [27, 28].

Le deuxième étage porte sur le graphe : les relations retournées sont-elles correctes, utiles et interprétables? Les tests doivent couvrir les requêtes métier-compétence, candidat-compétence, offre-compétence, compatibilité de niveau et recherche de chemins explicatifs. Le troisième étage porte sur le score hybride : il s'agit de comparer les classements produits par la recherche vectorielle seule, le graphe seul et la combinaison hybride. Le quatrième étage porte sur la réponse générée : fidélité au contexte, pertinence par rapport à la question, couverture des preuves, citations et utilité opérationnelle.

La méthode distingue donc clairement performance de récupération et qualité de génération. Une réponse bien rédigée ne prouve pas que le système a retrouvé les bonnes informations. Inversement, un bon retrieval ne garantit pas une bonne synthèse. Cette séparation protège l'analyse contre une erreur fréquente dans les systèmes RAG : juger le système uniquement à l'apparence de la réponse finale.

2.4.2 Limites méthodologiques et précautions d'interprétation

Plusieurs limites doivent être prises en compte dès la conception. Premièrement, les données d'offres et de candidats peuvent contenir des omissions, des doublons et des formulations ambiguës. Deuxièmement, l'alignement entre référentiels reste partiel : il améliore la structuration, mais ne garantit pas une équivalence parfaite entre catégories. Troisièmement, le graphe dépend de la qualité des relations extraites ou importées. Une relation manquante peut conduire à sous-estimer une compatibilité réelle. Quatrièmement, les LLM peuvent produire des formulations plausibles mais non soutenues par les données ; c'est pourquoi le contexte, les citations et le critic sont nécessaires.

Une autre limite est matérielle. L'utilisation locale ou semi-locale de modèles spécialisés, notamment pour Text2Cypher, peut être contrainte par la mémoire disponible. Cette contrainte influence les choix techniques : recours à une API pour la génération finale, chargement contrôlé du modèle Text2Cypher et possibilité de fallback vers des templates Cypher sécurisés. Ces choix ne sont pas des faiblesses méthodologiques en soi ; ils traduisent l'adaptation du système aux ressources disponibles.

2.4.3 Synthèse de l'approche méthodologique

La méthodologie proposée construit un système hybride plutôt qu'un modèle isolé. Elle part de données locales hétérogènes, les normalise, les rattache à des référentiels, les encode dans un espace vectoriel, les structure dans un graphe de connaissances, puis orchestre leur interrogation par un workflow agentique. Le rôle du LLM est strictement encadré : transformer certaines questions en requêtes graphe, puis formuler une réponse fondée sur les preuves récupérées.

Cette approche répond aux objectifs du mémoire. Elle permet de dépasser la recherche par mots-clés, d'exploiter les référentiels institutionnels, de produire des recommandations explicables et de relier chaque verdict à des compétences, des niveaux de formation et des sources identifiables. Les chapitres suivants présentent la réalisation opérationnelle de cette méthodologie, puis l'évaluation de ses performances.

Deuxième partie

Présentation des résultats

Implémentation du système de recommandation

Ce chapitre présente la matérialisation opérationnelle de l'architecture méthodologique décrite précédemment. Il ne reprend donc pas les définitions générales des embeddings, du RAG, du graphe de connaissances ou de l'orchestration agentique. Son objectif est différent : montrer ce qui a été effectivement construit, alimenté, indexé, relié et rendu exploitable dans le système de recommandation emploi-compétences.

La lecture du chapitre suit une logique d'ingénierie statistique. Chaque brique est examinée à partir de trois éléments : les artefacts produits, les indicateurs observés et les limites opérationnelles. Les résultats finaux de performance, le calibrage du *critic* et l'évaluation détaillée des réponses générées sont volontairement réservés au chapitre d'évaluation.

3.1 Architecture opérationnelle et données exploitées

3.1.1 Briques effectivement implémentées

La vue d'ensemble de l'implémentation doit être lue comme une chaîne d'exécution, et non comme une reprise de la figure méthodologique. Le système part de fichiers de données et aboutit à une application interrogeable. Entre les deux, chaque étape produit un artefact vérifiable : fichiers nettoyés, paires de fine-tuning, modèle sauvegardé, embeddings persistés, graphe Neo4j, moteur de recommandation, workflow LangGraph, API et interface utilisateur.

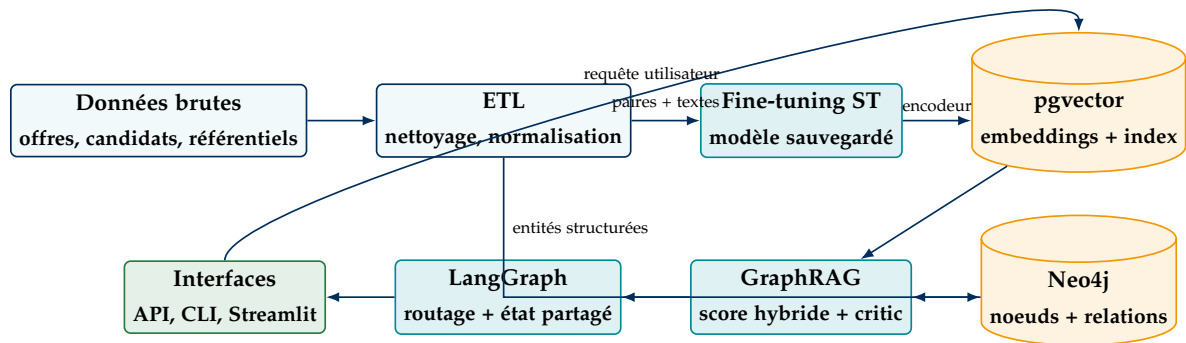


Figure 3.1 – Pipeline opérationnel effectivement implémenté

Source : Auteur, à partir des scripts, bases et sorties du projet

La figure 3.1 distingue les flux principaux. Les données nettoyées alimentent deux familles de composants : d’une part le modèle d’embeddings et la base vectorielle ; d’autre part le graphe de connaissances. Le moteur GraphRAG se situe à l’intersection de ces deux sources : il récupère des voisins sémantiques dans pgvector, exploite les relations Neo4j pour contextualiser les résultats, puis transmet un contexte contrôlé au workflow LangGraph. L’interface n’est donc pas un composant isolé ; elle est le point d’accès à une chaîne déjà construite.

Table 3.1 – Pipeline général implémenté : étapes, entrées et sorties

Ordre	Étape	Entrées principales	Sorties concrètes
1	ETL	Données brutes d’offres, candidats et référentiels	Fichiers normalisés, <code>text_to_embed</code> , <code>pairs_train/val/test.jsonl</code> , champs paires
2	Fine-tuning	Paires offre–description et modèle <code>all-MiniLM-L6-v2</code>	Modèle sauvegardé dans <code>models/st_finetuned/final</code> et métriques d’entraînement.
3	Indexation vectorielle	Entités nettoyées et encodeur fine-tuné	Table <code>embeddings</code> , table <code>doc_chunks</code> , index HNSW et index textuels PostgreSQL.
4	Graphe de connaissances	Offres, candidats, compétences, métiers, niveaux et référentiels	Noeuds Neo4j, relations métier-compétence-offre-candidat, contraintes et chemins explicatifs.
5	GraphRAG hybride	Résultats pgvector, chemins Neo4j et règles métier	Contexte candidat–offre, score hybride, analyse de <i>skill gap</i> , critic et Text2Cypher.
6	Orchestration LangGraph	Question utilisateur et registre d’outils	État partagé, routage, exécution des outils, génération finale et traces du workflow.
7	Exposition applicative	Workflow agentique et services locaux	API FastAPI, CLI de test et interface Streamlit.

Source : Auteur, à partir des scripts `src/` et `scripts/` du projet

Table 3.2 – Séparation des couches techniques du système

Couche	Artefacts	Responsabilité dans le système
Données	<code>data/processed</code> , <code>data/finetune</code>	Porter les observations nettoyées, les variables normalisées et les jeux de paires utilisés pour l'apprentissage.
Modèle <code>/final</code>	<code>models/st_finetuned</code> Transformer textes, offres, candidats, compétences et référentiels en vecteurs comparables.	
Index vectoriel	PostgreSQL <code>kmer_ai</code> , pg-vector	Stocker les embeddings, exécuter la recherche top-K et conserver les liens vers Neo4j.
Graphe	Neo4j	Représenter les relations explicites entre candidats, offres, compétences, métiers, niveaux, secteurs et documents.
Moteur hybride	<code>src/05_graphrag</code>	Combiner proximité vectorielle, relations du graphe, règles métier, skill gap et critic.
Orchestration	<code>src/08_agentic</code> <code>_graphrag/graph</code>	Router la requête, appeler les outils, construire le contexte, déclencher la génération et conserver les traces.
Interfaces	<code>src/06_api/main.py</code> <code>chatbot_app.py</code> , <code>cli.py</code>	Rendre le système utilisable via API, interface conversationnelle ou ligne de commande.
Outputs	<code>outputs/memoire</code> <code>_stats</code> , <code>outputs/evaluati</code>	Fournir les tableaux, graphiques et diagnostics utilisés dans le mémoire.

Source : Auteur, à partir de l'organisation du dépôt

Cette séparation est le point central de l'implémentation. Les données ne sont pas confondues avec le modèle ; le modèle ne remplace pas la base vectorielle ; pgvector ne remplace pas Neo4j ; le graphe ne génère pas la réponse ; LangGraph ne calcule pas seul la recommandation ; l'interface n'est qu'une couche d'accès. Le système est donc auditable par couche : on peut vérifier les fichiers produits par l'ETL, le modèle sauvegardé, les tables pgvector, les noeuds Neo4j, les outils GraphRAG, les traces LangGraph et les réponses exposées à l'utilisateur.

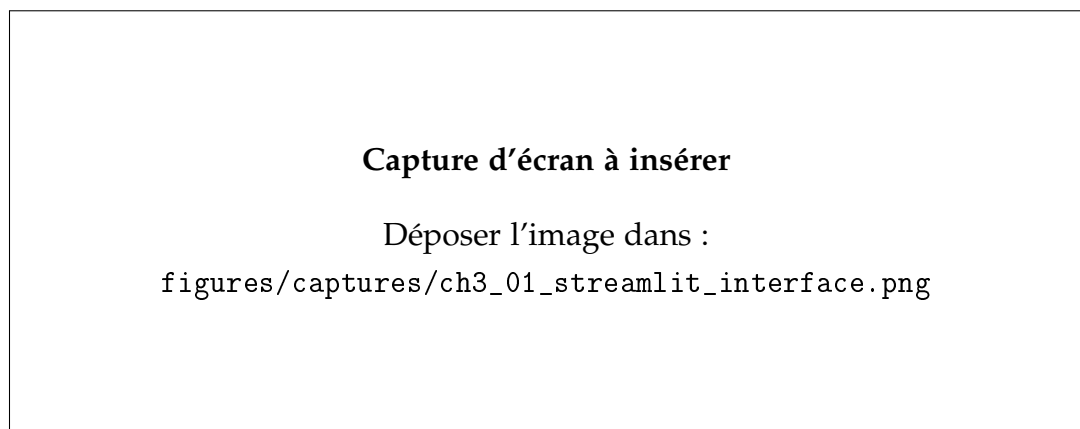


Figure 3.2 – Interface Streamlit du système de recommandation

Source : Capture d'écran de l'application locale

La capture de l'interface Streamlit doit être interprétée comme une preuve d'intégration applicative. Elle ne mesure pas la qualité statistique du système ; elle montre que les briques sont exposées dans une interface utilisable, avec une entrée utilisable, des résultats retournés et, idéalement, des traces du workflow. Pour le mémoire, cette capture doit donc compléter les métriques, pas les remplacer.

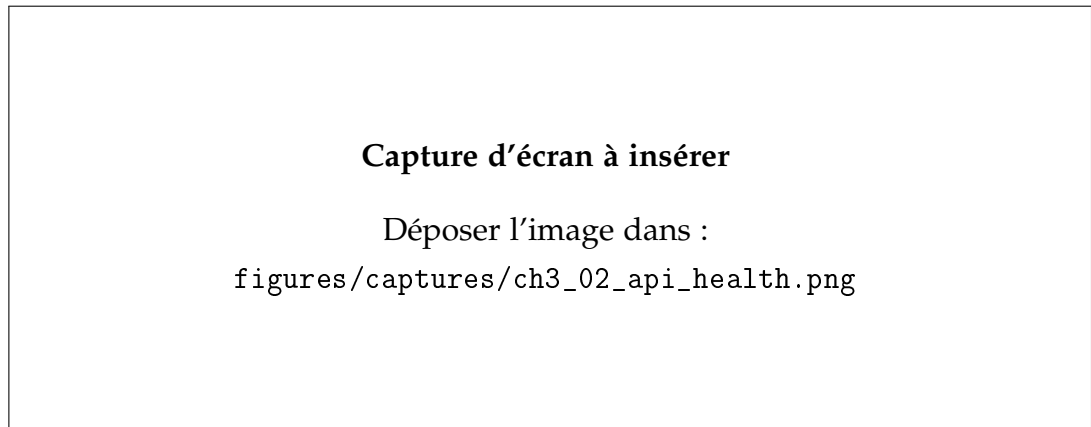


Figure 3.3 – Diagnostic API ou état des services

Source : Capture d'écran de FastAPI, terminal ou navigateur local

La capture du diagnostic API est utile pour documenter la disponibilité des services : modèle d'embeddings, pgvector, Neo4j et agent. Elle permet de passer d'un discours déclaratif à une preuve d'exécution locale. Si un service apparaît en erreur sur la capture, il faut l'assumer dans l'interprétation plutôt que masquer le problème.

3.1.2 Diagnostic du corpus après nettoyage

Le corpus utilisé dans l'implémentation ne doit pas être lu comme une simple base descriptive. Il constitue la matière première du moteur de recommandation : chaque variable conservée doit donc apporter un signal exploitable pour rapprocher une offre, un candidat, une compétence, un métier ou un référentiel. La question d'ingénierie posée ici est directe : le système dispose-t-il d'assez d'information structurée pour recommander correctement, et quelles limites doivent être compensées par l'architecture hybride ? Le tableau suivant synthétise uniquement les volumes qui conditionnent directement la faisabilité technique du système.

Table 3.3 – Volumes réellement exploités par les briques de recommandation

Objet exploité	Volume	Utilité pour le système
Offres normalisées	13 957	Base principale à recommander, indexée dans pgvector et reliée au graphe.
Candidats normalisés	41 298	Vivier à partir duquel sont calculés les rapprochements offre–profil.
Compétences indexées	13 939	Signal de matching, de filtrage et de justification du <i>skill gap</i> .
Secteurs nettoyés	1 039	Signal de contexte pour limiter les rapprochements absurdes entre domaines trop éloignés.
Métiers indexés	3 039	Niveau d’agrégation utile pour stabiliser les recommandations au-delà des intitulés d’offres.
Groupes métiers MEPC	209	Référentiel externe pour structurer les proximités métier.
Domaines détaillés NCF	201	Référentiel de formation utilisé pour contextualiser les niveaux et domaines.
Localisations représentées dans Neo4j	312	Signal géographique pour filtrage souple et explication des résultats.

Source : 01_description_vue_ensemble.csv, 06_pgvector_counts.csv et diagnostic Neo4j

Le premier résultat d’ingénierie est positif : les volumes sont suffisants pour construire un système hybride, car le corpus ne se limite pas aux offres. Il contient aussi des candidats, des compétences, des secteurs, des métiers, des localisations et des référentiels. En revanche, l’asymétrie entre 13 957 offres et 41 298 candidats impose de contrôler les effets de popularité : un secteur très représenté peut devenir dominant même lorsque la pertinence individuelle est moyenne. Cette limite justifie l’usage d’un reranking hybride plutôt qu’un classement purement vectoriel.

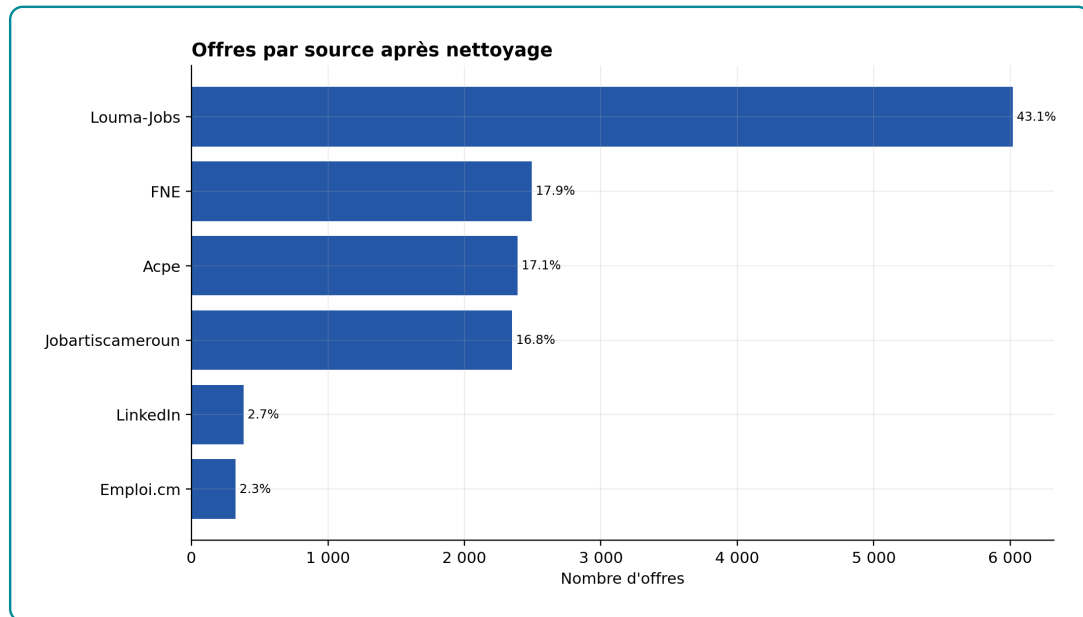


Figure 3.4 – Répartition des offres par source après nettoyage

Source : `outputs/memoire_stats/implementation_deep_current/01_offres_par_source.csv`

Les 13 957 offres proviennent de six sources : Louma-Jobs, FNE, ACPE, Jobartis Cameroun, LinkedIn et Emploi.cm. Louma-Jobs concentre 6 021 offres, soit 43,14 % du fichier. Le FNE représente 2 494 offres, ACPE 2 388, Jobartis Cameroun 2 350, LinkedIn 380 et Emploi.cm 324. Cette concentration n'est pas neutre : le modèle observe davantage le vocabulaire, les formats et les secteurs présents sur Louma-Jobs que ceux des autres plateformes. Le corpus est donc robuste pour construire un prototype opérationnel, mais il ne doit pas être présenté comme une photographie exhaustive du marché camerounais de l'emploi. La bonne lecture est la suivante : les sources sont assez nombreuses pour éviter une dépendance totale à une seule plateforme, mais assez déséquilibrées pour imposer une analyse par source dans l'évaluation.

3.1.3 Qualité des variables utiles au matching

La qualité d'un système de recommandation dépend moins du nombre brut de lignes que de la disponibilité des variables qui portent la décision : description textuelle, compétences, localisation, secteur, niveau et source. Le diagnostic ci-dessous montre les signaux fiables et ceux qui doivent être traités avec prudence.

Table 3.4 – Lecture d'ingénierie de la qualité des champs utiles

Champ	Indicateur observé	Conséquence pour la recommandation
Description d'offre	8 197 descriptions disponibles sur 13 957 offres, soit 58,73 %.	Le texte long seul est insuffisant. Les métadonnées et compétences doivent enrichir la représentation vectorielle.
Source	6 sources, dont Louma-Jobs à 43,14 %.	La source doit être surveillée comme facteur de biais de collecte et de style rédactionnel.
Localisation	11 149 offres rattachées au Cameroun et 2 808 à l'international.	Lorsque la ville manque, le pays est conservé comme signal géographique minimal ; le filtre ville doit rester souple.
Compétences	13 939 entités compétences indexées dans pgvector.	Le signal est riche, mais hétérogène : certaines valeurs relèvent du domaine ou du secteur plutôt que d'une compétence fine.
Secteur	1 039 secteurs nettoyés et 13 957 relations DANS_SECTEUR matérialisées dans Neo4j.	Le secteur est un bon signal d'explication et de reranking, surtout lorsque la description est absente ; sa forte granularité impose toutefois des regroupements prudents.
Niveau	41 254 relations A_NIVEAU présentes dans le graphe.	Le niveau est utilisable pour la cohérence offre-profil, mais il doit être pondéré avec les compétences et l'expérience.

Source : 03_description_par_source.csv, 02_zones_localisation.csv, 06_pgvector_counts.csv et diagnostic Neo4j

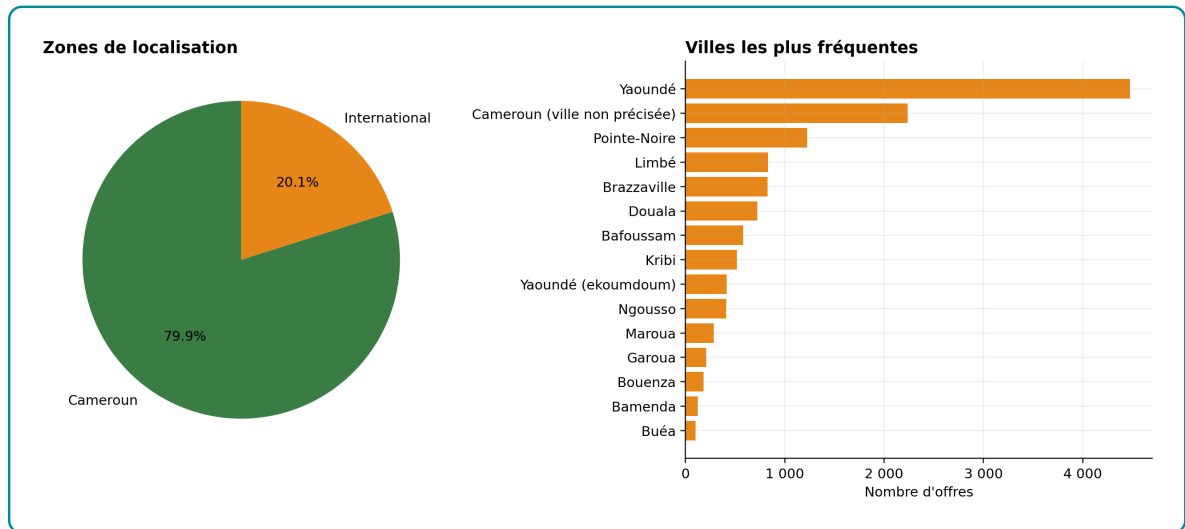


Figure 3.5 – Localisation nettoyée des offres

Source : `outputs/memoire_stats/implementation_deep_current/02_zones_localisation.csv` et `02_villes_localisation.csv`

La localisation confirme que le système dispose d'un signal géographique exploitable : 79,88 % des offres sont rattachées au Cameroun et 20,12 % à l'international. Les localités dominantes sont notamment Yaoundé, Douala, Bafoussam, Kribi, Maroua, Garoua, Bamenda et Buéa, avec aussi des localités plus fines comme Ngoussou ou Yaoundé-Ekoumdoum. Lorsque la ville n'est pas indiquée mais que le pays est renseigné, l'offre n'est plus classée comme localisation globalement inconnue : elle conserve une étiquette du type « Cameroun (ville non précisée) » ou « États-Unis (ville non précisée) ». Le nettoyage a donc privilégié une stratégie hiérarchique : exploiter d'abord la ville, sinon le pays, et ne pas inventer une ville absente du fichier brut.

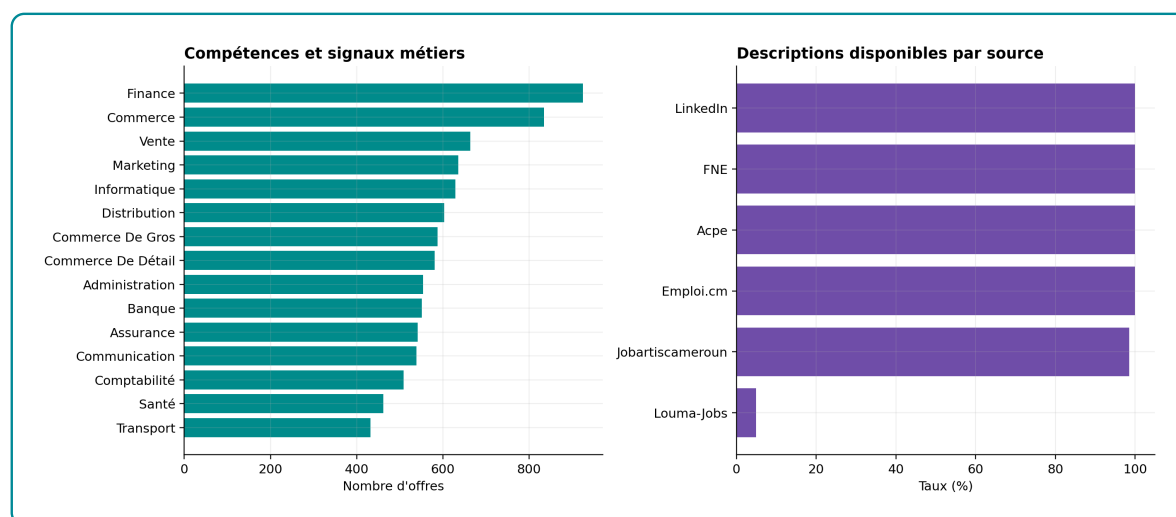


Figure 3.6 – Compétences fréquentes et disponibilité descriptive

Source : *outputs/memoire_stats/implementation_deep_current/03_compétences_frequentes.csv*
et *03_description_par_source.csv*

Les signaux les plus fréquents sont Finance, Commerce, Vente, Marketing, Informatique, Distribution, Banque, Assurance, Communication, Comptabilité, Santé et Transport. Cette structure est cohérente avec un système emploi-compétences, mais elle révèle un risque méthodologique : plusieurs libellés sont des domaines d'activité plutôt que des compétences observables. Dans l'implémentation, ces variables sont donc utilisées comme signaux d'appariement et de contexte, non comme preuve définitive d'une compétence maîtrisée par un candidat.

La disponibilité descriptive est le principal point faible du corpus. Les sources FNE, ACPE, LinkedIn et Emploi.cm disposent d'une description pour toutes leurs offres dans le fichier nettoyé ; Jobartis Cameroun atteint 98,55 %. La limite vient de Louma-Jobs : seulement 295 descriptions disponibles sur 6 021 offres, soit 4,90 %. Cette hétérogénéité explique un choix central de l'ETL : la représentation envoyée au modèle d'embeddings ne peut pas être uniquement le détail textuel de l'offre ; elle doit combiner métadonnées, compétences, secteur, localisation et description lorsque celle-ci existe.

3.1.4 Nettoyage réalisé et limites conservées

Le nettoyage appliqué au corpus répond à un principe conservateur : corriger les incohérences techniques sans créer une information qui n'existe pas. Les libellés ont été harmonisés pour limiter les effets de casse et de formats d'écriture. Pour la localisation, la règle retenue est hiérarchique : si la ville est disponible, elle est utilisée ; si la ville est absente mais que le pays est connu, le pays est conservé ; si aucune information géographique exploitable n'existe, l'observation reste non localisable. Cette règle évite de supprimer silencieusement les offres concernées, tout en signalant au moteur que le filtrage par ville doit être moins contraignant que le filtrage par pays.

La déduplication des offres suit également une règle stricte : une offre n'est considérée comme doublon que lorsque toute la ligne normalisée est identique sur l'ensemble des variables. Ce choix est rigoureux pour un corpus multi-sources, car deux offres peuvent partager un intitulé ou un employeur tout en différant par la ville, la source, la date, le niveau, le secteur ou la description. Une déduplication floue aurait risqué de supprimer des opportunités réellement distinctes.

Table 3.5 – Synthèse des traitements de nettoyage utiles au moteur

Traitement	Décision implémentée	Effet attendu
Casse et formats	Harmonisation des libellés pour réduire les variantes d'écriture.	Moins de fragmentation dans les compétences, secteurs, villes et sources.
Doublons	Validation uniquement si toute la ligne est identique sur toutes les variables.	Suppression prudente des répétitions exactes, sans effacer des offres proches mais distinctes.
Localisation	Séparation Cameroun, international et non précisée; conservation des localités fines disponibles.	Filtrage géographique possible sans perte des offres incomplètes.
Champs vides	Modalités explicites lorsque le champ reste exploitable comme absence d'information.	Le système peut pénaliser ou contextualiser le manque d'information au lieu d'ignorer l'offre.
Descriptions faibles	Enrichissement de la représentation par métadonnées et compétences.	Robustesse accrue pour les sources où le détail de l'offre est absent ou court.

Source : Pipeline ETL du projet et sorties de diagnostic du dossier
outputs/memoire_stats/implementation_deep_current

Les graphiques secondaires détaillant la structure sectorielle, les villes détaillées, les niveaux NCF, l'expérience, la structure des candidats, les longueurs de textes et les référentiels MEPC–NCF sont renvoyés en annexes. Dans ce chapitre, seuls les résultats qui changent une décision d'architecture sont conservés : enrichir les embeddings, utiliser pgvector pour le rappel rapide, exploiter Neo4j pour les contraintes métier, et garder un critic pour contrôler les réponses générées.

3.2 Implémentation du modèle d'embeddings et indexation pgvector

3.2.1 Choix du modèle baseline et construction des paires

Le modèle de base retenu est `sentence-transformers/all-MiniLM-L6-v2`. Ce choix est défendable pour une raison d'ingénierie : il produit des embeddings de 384 dimensions, donc moins coûteux à stocker et à rechercher que des modèles à 768 dimensions, tout en restant compatible avec une indexation HNSW dans pgvector. Dans ce projet, la contrainte est concrète : le même encodeur sert ensuite à vectoriser les candidats, les offres, les compétences, les métiers et les référentiels. Un modèle plus lourd peut améliorer certains cas sémantiques, mais il augmente la taille de stockage, la latence d'encodage et le coût de réindexation.

Ce modèle est acceptable comme point de départ parce qu'il fournit déjà une représentation dense générale des phrases. Il n'est toutefois pas suffisant comme modèle final. Un encodeur généraliste rapproche des textes proches en langage courant, mais il ne connaît pas nécessairement les proximités utiles au domaine emploi-compétences camerounais : intitulés locaux, secteurs, niveaux de formation, familles métiers, descriptions incomplètes et compétences parfois formulées comme domaines. Le fine-tuning vise donc à spécialiser l'espace vectoriel sans changer l'architecture de production.

Table 3.6 – Protocole de construction des paires de fine-tuning

Élément	Implémentation observée
Modèle de départ	sentence-transformers/all-MiniLM-L6-v2, embeddings denses de 384 dimensions.
Rôle de sentence1	Représentation structurée de l'offre : titre, secteur, contrat, niveau d'études, expérience, ville et compétences.
Rôle de sentence2	Détails nettoyés de l'annonce uniquement, c'est-à-dire le texte descriptif que le modèle doit rapprocher de la représentation structurée.
Filtrage informatif	Conservation des paires avec compétences disponibles, sentence1 d'au moins 40 caractères et sentence2 d'au moins 120 caractères; le champ ft_eligible est appliqué lorsqu'il existe.
Nombre total de paires	6 476 paires offre-description.
Split	4 542 paires en train, 981 en validation et 953 en test, avec stratification par secteur principal.
Fichiers produits	data/finetune/pairs_train.jsonl, pairs_val.jsonl, pairs_test.jsonl et pairs_metadata.json.

Source : src/01_etl/build_pairs.py et data/finetune/pairs_metadata.json

La structure sentence1/sentence2 est volontairement asymétrique. sentence1 représente la requête métier structurée que le système manipulera souvent en production : métadonnées, compétences et contexte de l'offre. sentence2 représente le texte descriptif à retrouver. Ce choix entraîne le modèle à rapprocher une description naturelle d'une formulation structurée, ce qui correspond mieux au besoin du système qu'un entraînement sur deux phrases descriptives ordinaires.

Le filtrage des offres informatives est nécessaire. Entraîner le modèle sur des paires dont la description est vide, trop courte ou sans compétences introduirait du bruit : le modèle apprendrait à rapprocher des textes pauvres au lieu de structurer le domaine. Le seuil de 120 caractères pour sentence2 et de 40 caractères pour sentence1 est donc un garde-fou d'apprentissage. Il réduit le volume utilisé pour le fine-tuning, mais améliore la qualité du signal.

Capture d'écran à insérer

Déposer l'image dans :

`figures/captures/ch3_03_finetuning_terminal.png`

Figure 3.7 – Trace d'entraînement du modèle SentenceTransformer

Source : Capture d'écran du terminal d'entraînement

La capture du terminal de fine-tuning doit montrer le chargement du modèle, le démarrage de l'entraînement, l'évolution de la loss et les paramètres utilisés. Elle joue un rôle de preuve d'exécution. L'interprétation reste toutefois statistique : une loss qui baisse ne suffit pas à valider le modèle ; il faut la compléter par des métriques de ranking et une comparaison avec des modèles de référence.

3.2.2 Courbes d'entraînement et comparaison des modèles

Table 3.7 – Hyperparamètres du fine-tuning SentenceTransformer

Hyperparamètre	Valeur
Nombre d'époques	5
Batch size	32
Négatifs implicites par exemple	31
Learning rate maximal	2×10^{-5}
Scheduler	Cosine avec warmup
Warmup	100 étapes configurées, limitées par les étapes disponibles de l'époque
Perte	<i>MultipleNegativesRankingLoss</i> avec similarité cosinus
Temps d'entraînement observé	15 160 s
Checkpoint retenu	Meilleur checkpoint sur validation autour de l'époque 4

Source : *src/02_finetune_st/train_sentence_transformer.py* et
models/st_finetuned/evaluation_metrics.json

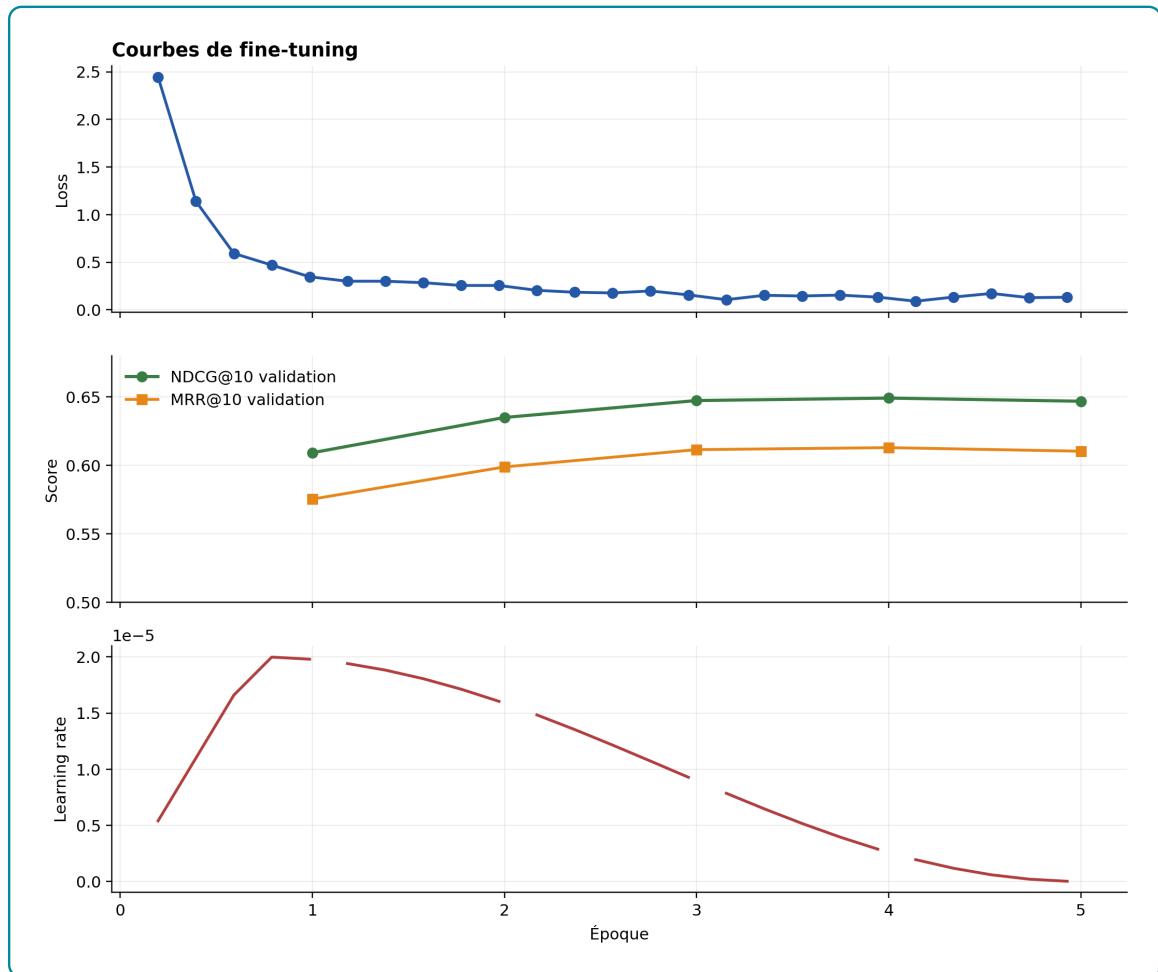


Figure 3.8 – Évolution de la perte, pertes finales par split, métriques de validation et learning rate

Source : `models/st_finetuned/evaluation_metrics.json` et
`outputs/memoire_stats/implementation_deep_current/04_finetuning_loss_par_split.csv`

L'entraînement dure cinq époques, avec un *learning rate* maximal de 2×10^{-5} , 100 étapes de *warmup*, un batch de 32 et une décroissance de type *WarmupCosine*. La perte d'entraînement chute fortement au début, de 2,445 à moins de 0,5 avant la fin de la première époque, puis se stabilise. Pour contrôler la généralisation, la même perte contrastive a été recalculée après entraînement sur un échantillon fixe de 64 paires par split : 0,171 sur train, 0,285 sur validation et 0,433 sur test. Ces valeurs ne remplacent pas les métriques de classement, car la *MultipleNegativesRankingLoss* dépend de la composition des batches ; elles indiquent surtout que les jeux non vus restent logiquement plus difficiles que le train.

La validation atteint un NDCG@10 maximal de 0,6492 à l'époque 4, avant un léger recul à 0,6469 à l'époque 5. Ce profil est cohérent avec un apprentissage rapide suivi d'un plateau. Il serait donc fragile de prolonger l'entraînement sans contrôle : au-delà d'un certain point, le gain marginal devient faible et le risque de sur-adaptation augmente.

Table 3.8 – Comparaison directe entre le baseline et le modèle fine-tuné

Métrique test	Baseline	Fine-tuné	Gain
NDCG@10	0,1173	0,4537	+0,3364
MRR@10	0,1012	0,3984	+0,2971
Recall@1	0,0730	0,3127	+0,2398
Recall@5	0,1367	0,5058	+0,3692
Recall@10	0,1688	0,6347	+0,4660
Precision@1	0,0730	0,3127	+0,2398

Source : models/st_finetuned/eval_comparatif/evaluation_comparatif.json

Cette comparaison est la preuve la plus directe de l'intérêt du fine-tuning : le modèle de base ne structure pas correctement le domaine dans ce protocole, alors que le modèle adapté récupère beaucoup plus souvent la description pertinente dans les premiers rangs. Le gain de recall@10 est particulièrement important, car l'application ne dépend pas d'une seule réponse immédiate ; elle récupère d'abord un ensemble court de candidats pertinents avant reranking hybride.

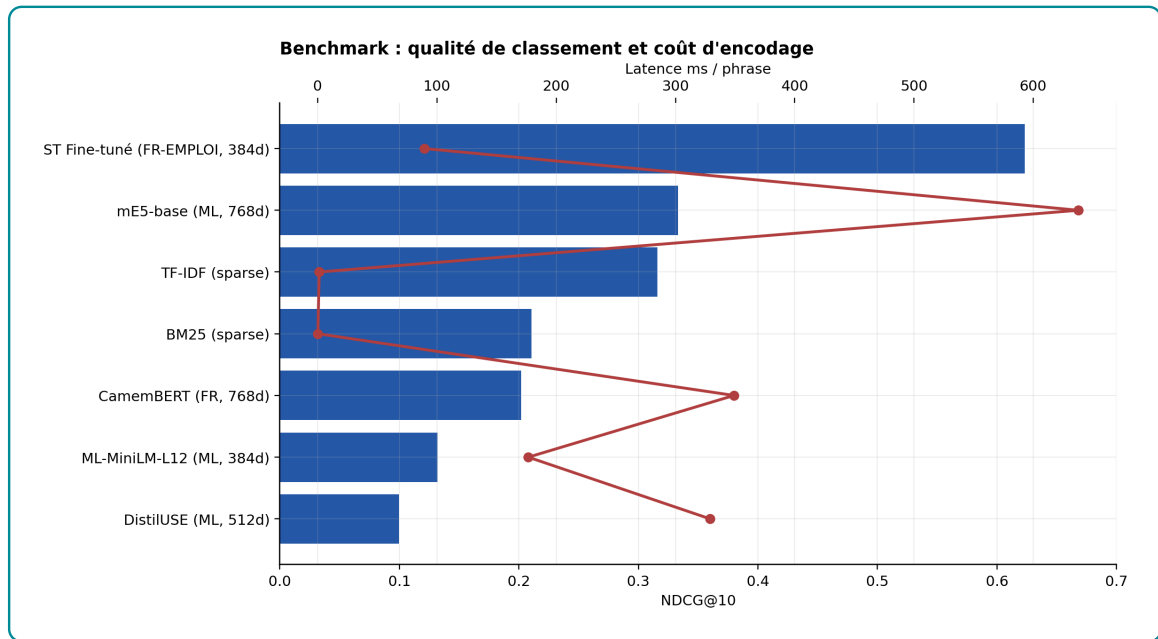


Figure 3.9 – Comparaison des modèles d'embeddings : qualité de classement et latence

Source : `outputs/evaluation/embedding_benchmark.csv`

Table 3.9 – Résultats de benchmark des représentations

Modèle	Dim.	NDCG@10	MRR@10	Latence ms
ST fine-tuné emploi-compétences	384	0,6232	0,5874	89,5
mE5-base multilingue	768	0,3333	0,3015	637,9
TF-IDF	29 169	0,3159	0,3016	1,3
BM25	–	0,2105	0,1771	0,1
CamemBERT sentence	768	0,2018	0,1755	349,1

Source :

`outputs/memoire_stats/implementation_deep_current/05_benchmark_modeles_embeddings.csv`

Le benchmark multi-modèles confirme le résultat. Le modèle fine-tuné atteint un NDCG@10 de 0,6232 et un MRR@10 de 0,5874, contre 0,3333 pour mE5-base et 0,3159 pour TF-IDF. L'intérêt du fine-tuning est donc empirique, pas seulement théorique. TF-IDF reste cependant utile pour les correspondances exactes, acronymes et intitulés rares. La bonne architecture n'oppose donc pas « deep learning » et lexical ; elle utilise le modèle fine-tuné comme signal sémantique principal et conserve les signaux lexicaux comme complément.

3.2.3 Interprétation de l'espace vectoriel

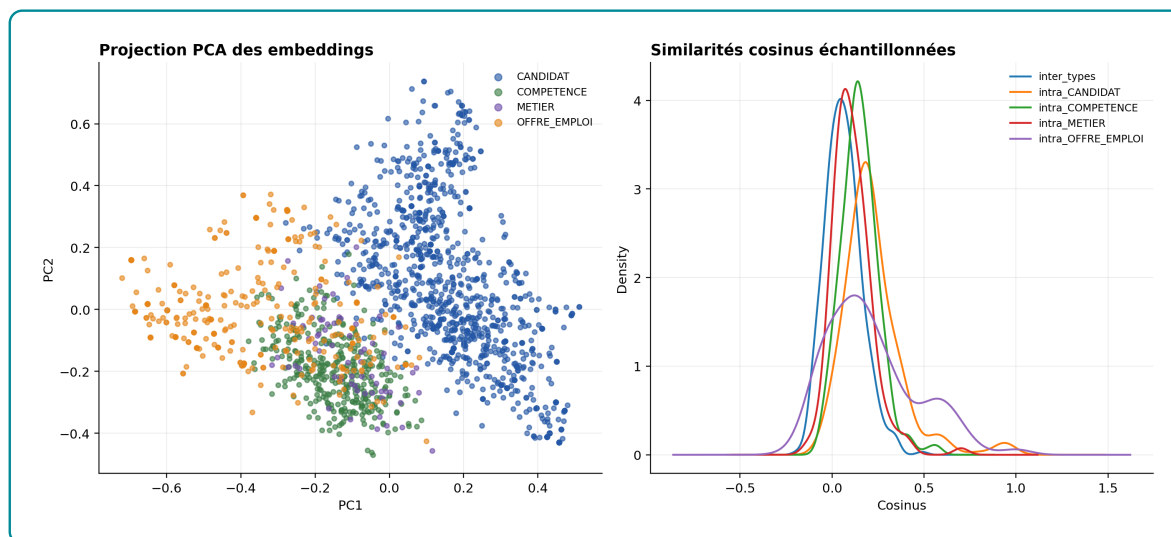


Figure 3.10 – Projection PCA et distribution des similarités cosinus

Source : Échantillon d'embeddings extrait de la table `embeddings`

La projection PCA ne doit pas être interprétée comme une carte exacte du sens ; elle compresse un espace de 384 dimensions en deux axes. Elle sert plutôt de diagnostic : vérifier que les embeddings présentent une structure exploitable et que les différents types d'entités ne sont pas totalement confondus. La distribution des similarités cosinus complète cette lecture en montrant si les proximités intra-types et inter-types sont cohérentes. L'enjeu n'est pas d'obtenir de beaux nuages de points, mais de vérifier que l'espace permet au moteur de retrouver des offres, candidats, compétences et métiers proches sans tout écraser dans une masse indifférenciée.

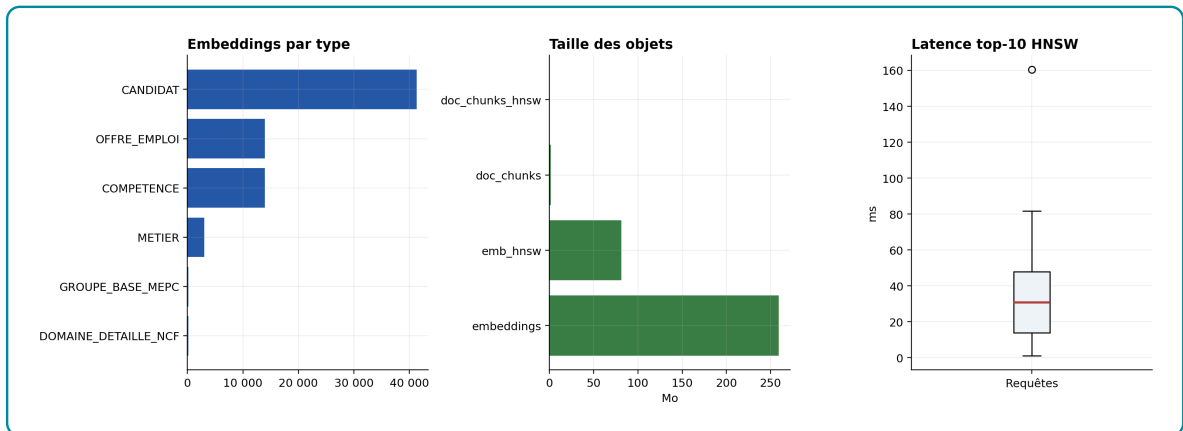


Figure 3.11 – Indicateurs d'indexation pgvector

Source : Base PostgreSQL *kmer_ai*, tables *embeddings* et *doc_chunks*

3.3 Implémentation de pgvector comme brique de retrieval

La base pgvector n'est pas utilisée comme un simple entrepôt de vecteurs. Elle constitue le premier étage du retrieval de l'Agentic RAG : transformer une requête en vecteur, récupérer un top- K d'entités proches, puis transmettre ces résultats au moteur hybride et au graphe Neo4j. La preuve attendue est donc statistique et opérationnelle : volume indexé, couverture des entités, dimension des vecteurs, index effectivement présents et latence de recherche.

Table 3.10 – Distribution des embeddings par type d’entité

Type d’entité	Vecteurs	Part	Couverture Neo4j
Candidats	41 298	56,85 %	100 %
Offres d’emploi	13 957	19,22 %	100 %
Compétences	13 939	19,19 %	100 %
Métiers	3 039	4,18 %	100 %
Groupes de base MEPC	209	0,29 %	100 %
Domaines détaillés NCF	201	0,28 %	100 %
Total entités structurées	72 643	100 %	100 %
Chunks documentaires	142	–	100 %

Source : `outputs/memoire_stats/implementation_deep_current/06_pgvector_counts.csv` et
`06_pgvector_doc_chunks.csv`

Cette distribution montre que pgvector couvre les deux côtés du matching. Les candidats représentent 56,85 % des embeddings et les offres 19,22 %. Les compétences et métiers forment le vocabulaire pivot qui permet de passer d’une similarité textuelle à une logique emploi-compétences. La couverture Neo4j de 100 % est déterminante : chaque résultat vectoriel peut être remplacé dans le graphe pour récupérer secteur, niveau, localisation, compétences reliées ou chemin explicatif.

Table 3.11 – Structure opérationnelle des tables pgvector

Table	Rôle	Champs et usages clés
embeddings	Recherche sémantique sur les entités structurées.	entity_kind, entity_id, model_id, embedding, label_fr, text_to_embed, neo4j_node_id.
doc_chunks	Recherche documentaire sur les référentiels.	chunk_id, source, section_title, subsection_title, chunk_text, chunk_strategy, ncf_code, mepc_code, embedding, neo4j_node_id.

Source : Schéma PostgreSQL inspecté via `06_pgvector_indexes.csv`

Le modèle d’encodage utilisé est le SentenceTransformer fine-tuné du projet, identifié dans la base par `all-MiniLM-L6-v2-ft-offres-cm`. Les vecteurs ont une dimen-

sion de 384, ce qui garde un compromis raisonnable entre précision de retrieval et coût d'indexation. Au total, la brique vectorielle contient 72 643 embeddings d'entités structurées et 142 embeddings documentaires, soit 72 785 vecteurs exploitables par le retrieval.

3.3.1 Stratégie de chunking des référentiels

Les chunks documentaires ne sont pas produits par un découpage arbitraire du texte. La stratégie implémentée dans `scripts/ingest_pdfs.py` est structurelle par défaut : le script détecte les titres de section à partir de la mise en forme et des motifs numérotés, puis chaque titre ouvre un chunk dont le contenu s'étend jusqu'au titre suivant. Les fragments trop courts, inférieurs à 80 caractères, sont fusionnés avec le fragment précédent afin d'éviter d'indexer des passages sans contexte. Une limite supérieure de 3 000 caractères contrôle la taille maximale du passage envoyé à l'encodeur. Cette stratégie est cohérente avec les documents exploités, car la NCF, le MEPC et le référentiel des diplômes sont des documents hiérarchisés : niveaux, domaines, groupes, sous-groupes, métiers et descripteurs.

Le pipeline prévoit aussi une stratégie sémantique de secours : si la structure n'est pas détectée, le texte est découpé en fenêtres glissantes de 800 caractères avec un recouvrement de 120 caractères. Cette option n'a pas été nécessaire dans l'index courant : les 142 chunks effectivement présents dans `doc_chunks` ont tous été générés par la stratégie structurelle. C'est une information importante pour l'évaluation, car le retrieval documentaire ne récupère pas seulement des morceaux de taille fixe ; il récupère des unités qui correspondent autant que possible à des sections réglementaires interprétables.

Table 3.12 – Diagnostic statistique du chunking documentaire

Indicateur	Valeur observée
Chunks documentaires indexés	142
Stratégie réellement utilisée	100 % structurelle
Chunks NCF 2017	38
Chunks MEPC 2013	61
Chunks référentiel des diplômes	43
Longueur minimale d'un chunk	84 caractères
Longueur médiane	952,5 caractères
Longueur moyenne	1 329,0 caractères
Longueur p90	3 000 caractères
Chunks avec embedding	142 / 142
Chunks synchronisés avec Neo4j	142 / 142

Source : Table `doc_chunks` de la base PostgreSQL `kmer_ai` et pipeline `scripts/ingest_pdfs.py`

Ce diagnostic apporte deux garanties. Premièrement, la granularité est suffisamment fine pour permettre un top-K documentaire : 142 unités de recherche couvrent les trois référentiels sans transformer chaque page entière en bloc massif. Deuxièmement, les chunks restent assez longs pour conserver le contexte réglementaire : une médiane proche de 953 caractères évite de récupérer uniquement des intitulés isolés. La borne haute à 3 000 caractères limite cependant le risque inverse, c'est-à-dire des chunks trop larges qui diluent le signal sémantique dans l'embedding. En pratique, cette stratégie rend les résultats plus citables : le système peut retourner un passage, sa source, sa page approximative, son titre de section et son noeud Neo4j associé.

Table 3.13 – Indicateurs techniques pgvector

Indicateur	Valeur observée
Embeddings d'entités structurées	72 643
Chunks documentaires indexés	142
Dimension des vecteurs	384
Modèle d'encodage	all-MiniLM-L6-v2-ft-offres-cm
Index PostgreSQL sur embeddings et doc_chunks	19
Taille totale de la table embeddings	259,05 Mo
Taille de l'index emb_hnsw	81,30 Mo
Taille totale de doc_chunks	1,51 Mo
Latence moyenne top-10 HNSW sur 25 requêtes	34,31 ms
Latence médiane top-10 HNSW sur 25 requêtes	30,89 ms
Latence p90 top-10 HNSW sur 25 requêtes	64,41 ms

Source : outputs/memoire_stats/implementation_deep_current/06_pgvector_.csv*

La mesure de latence est suffisamment basse pour servir un retrieval interactif : une médiane de 30,89 ms pour un top-10 HNSW laisse de la marge pour les étapes suivantes, notamment le reranking graphe et la construction du contexte. Le maximum observé, 160,40 ms, rappelle toutefois qu'il faut raisonner en distribution et non seulement en moyenne. Pour un Agentic RAG, la stabilité du p90 est plus importante que le meilleur cas.

Table 3.14 – Index pgvector et index complémentaires

Famille d'index	Index observés	Usage dans le retrieval
Vectoriel HNSW	emb_hnsw, doc_chunks_hnsw vector_cosine_ops.	Recherche approximative des plus proches voisins en similarité cosinus.
B-tree métier	emb_offre_idx, emb_candidat_idx, emb_competence_idx, emb_metier_idx.	Filtrage par type d'entité et accès rapide aux objets métiers.
Synchronisation graphe	emb_neo4j_idx.	Passage d'un résultat vectoriel vers le noeud Neo4j correspondant.
Modèle et unicité	emb_model_idx, contrainte unique entity_kind, entity_id, model_id.	Éviter les doublons d'embeddings et gérer les futures versions de modèles.
Recherche lexicale	emb_fulltext_fr_idx, doc_chunks_fulltext_fr_:	Complément lexical français pour requêtes par mots exacts, acronymes ou intitulés rares.
Référentiels	doc_chunks_ncf_idx, doc_chunks_mepc_idx, doc_chunks_source_idx.	Recherche ciblée dans NCF, MEPC et référentiel des diplômes.

Source : `outputs/memoire_stats/implementation_deep_current/06_pgvector_indexes.csv`

Les documents référentiels ajoutent 142 chunks indexés : 43 pour les diplômes, 61 pour le MEPC 2013 et 38 pour la NCF 2017. Tous les chunks disposent d'un embedding et d'un identifiant Neo4j. Cette double indexation est importante pour le GraphRAG : pgvector retrouve le passage documentaire pertinent, puis Neo4j permet de le relier à une formation, un métier, un niveau ou un domaine.

Les recherches supportées sont de trois types. La première est la recherche sémantique top-K, fondée sur l'opérateur cosinus de pgvector. La deuxième est la recherche lexicale française, via les index GIN et to_tsvector. La troisième est la recherche hybride : présélection vectorielle, filtre métier par type d'entité, puis enrichissement par Neo4j.

-- Exemple 1 : top-10 sémantique sur les offres

```
SELECT entity_id, label_fr, 1 - (embedding <=> :query_embedding) AS score
FROM embeddings
WHERE entity_kind = 'OFFRE_EMPLOI'
ORDER BY embedding <=> :query_embedding
LIMIT 10;
```

```
-- Exemple 2 : recherche hybride sémantique + texte français
SELECT entity_id, label_fr,
       1 - (embedding <=> :query_embedding) AS score_vectoriel,
       ts_rank(
         to_tsvector('french', text_to_embed),
         plainto_tsquery('french', :q)
       ) AS score_lexical
FROM embeddings
WHERE entity_kind IN ('OFFRE_EMPLOI', 'COMPETENCE', 'METIER')
ORDER BY (embedding <=> :query_embedding) ASC
LIMIT 20;
```

Ces requêtes montrent que pgvector est prêt pour le retrieval de l'Agentic RAG : il peut produire un top-K sémantique, filtrer par nature d'entité, mobiliser un score lexical complémentaire et renvoyer des identifiants directement raccordables au graphe.

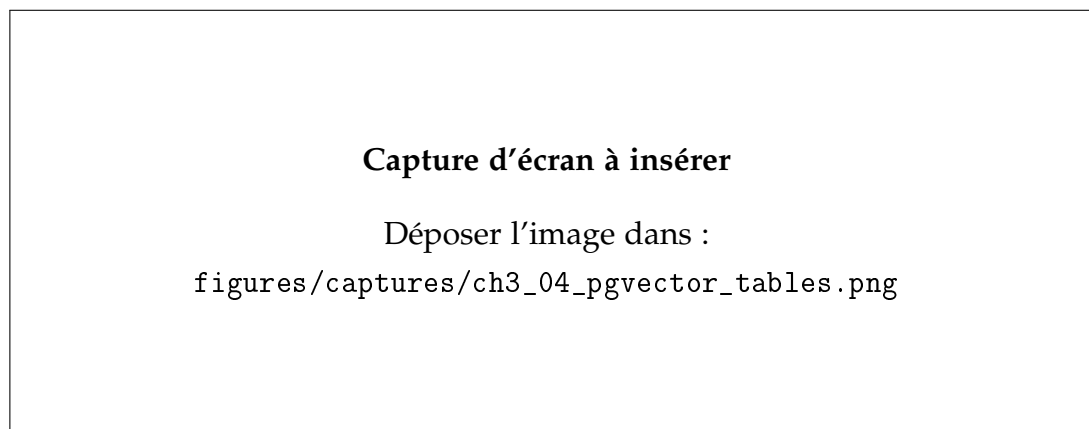


Figure 3.12 – Tables et index pgvector dans PostgreSQL

Source : Capture d'écran de pgAdmin, psql ou DBeaver

La capture pgvector doit montrer les tables embeddings et doc_chunks, ou les index associés. Son intérêt est de matérialiser la brique vectorielle : les embeddings ne sont

pas seulement calculés dans un script, ils sont persistés, indexés et exploitables par le moteur de recherche.

La conclusion technique est nette : pgvector remplit les conditions minimales d’une brique de retrieval. Il indexe toutes les entités utiles, conserve les liens Neo4j, dispose d’index vectoriels et lexicaux, et présente des latences compatibles avec une application interactive. Le point à surveiller pour l’évaluation n’est donc plus la disponibilité du stockage, mais la qualité du ranking produit par le retrieval et son amélioration par le graphe.

3.4 Implémentation du graphe Neo4j comme base de connaissances

3.4.1 Schéma du graphe de connaissances

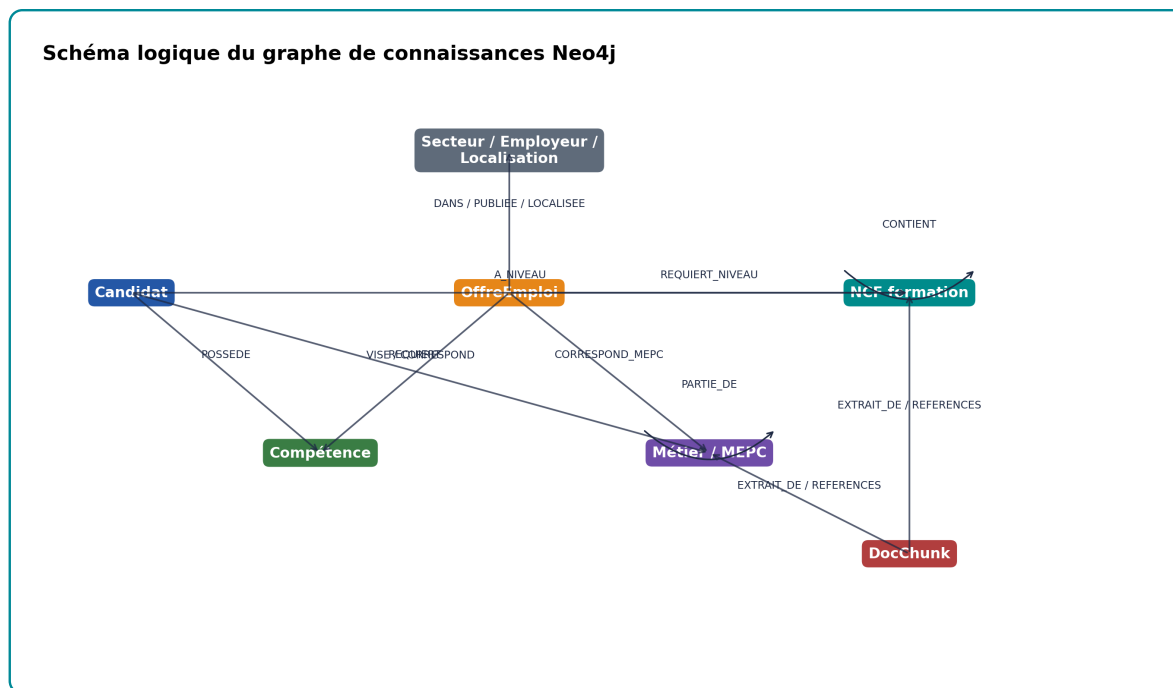


Figure 3.13 – Schéma logique du graphe de connaissances

Source : Synthèse des labels et relations implémentés dans Neo4j

Le graphe relie les candidats, offres, compétences, métiers, référentiels MEPC, niveaux et domaines NCF, secteurs, employeurs, localisations et chunks documentaires. Sa fonction principale n'est pas de remplacer l'embedding, mais de donner un contexte relationnel aux résultats vectoriels. Une offre proche vectoriellement peut ainsi être expliquée par ses compétences requises, son secteur, son niveau NCF ou son rattachement métier. Cette section ne décrit donc pas Neo4j en général ; elle interprète le graphe effectivement construit dans le projet.

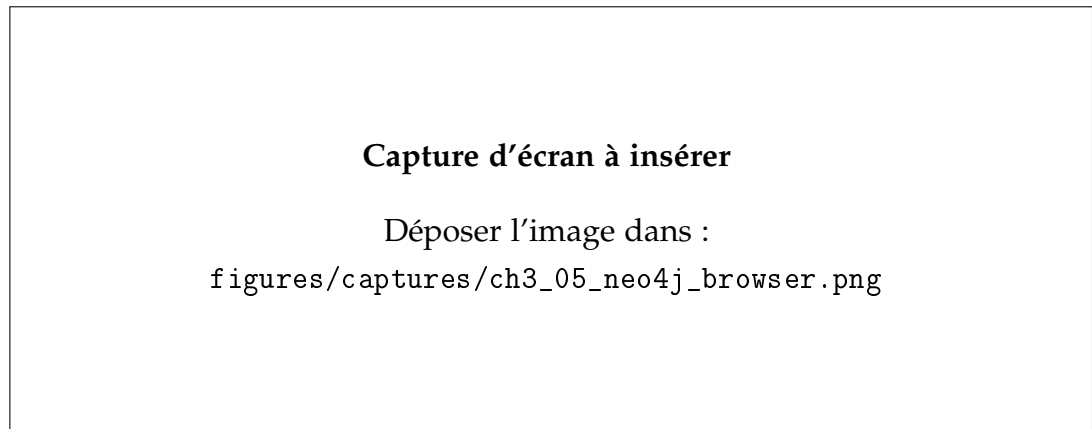


Figure 3.14 – Visualisation du graphe dans Neo4j Browser

Source : Capture d'écran de Neo4j Browser ou Neo4j Bloom

La capture Neo4j doit être utilisée comme preuve visuelle de la base de connaissances. Elle doit montrer des noeuds et relations réels du projet, par exemple un chemin candidat-compétence-offre ou offre-métier-référentiel. Son interprétation doit rester sobre : une capture ne prouve pas la qualité globale du graphe, mais elle illustre la manière dont les entités sont reliées.

3.4.2 Analyse topologique du graphe

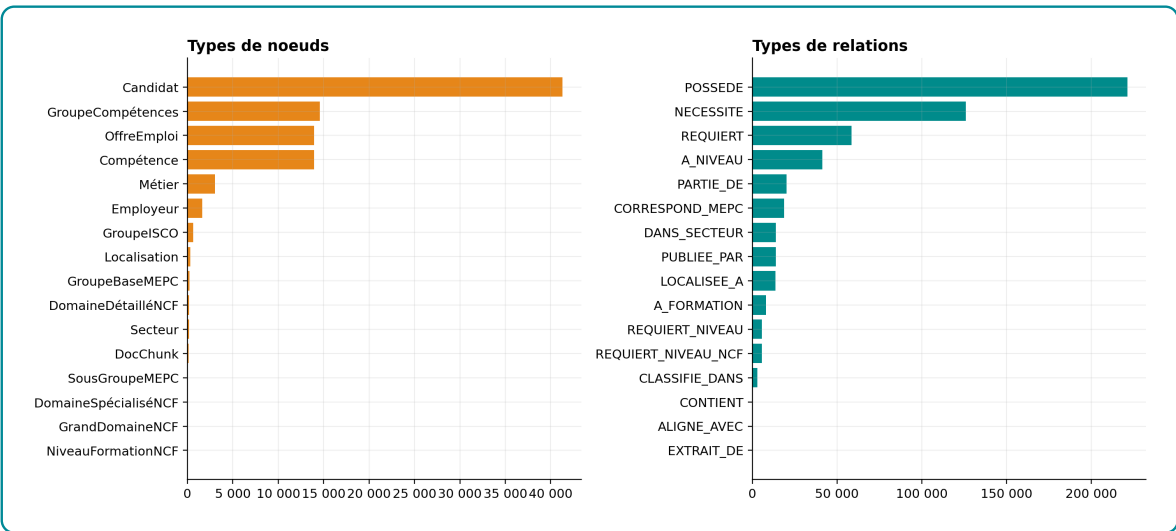


Figure 3.15 – Composition du graphe par types de noeuds et relations

Source : `outputs/memoire_stats/implementation_deep_current/08_neo4j_*.csv`

Le graphe contient 90 230 noeuds et 550 542 relations. Sa densité orientée globale est très faible, environ $6,76 \times 10^{-5}$, ce qui est normal pour un graphe hétérogène de connaissances : toutes les entités n'ont pas vocation à être reliées entre elles. Le degré moyen non orienté vaut 12,20 relations par noeud. Cette moyenne est cependant trompeuse, car le graphe est fortement déséquilibré : les cinq premiers types de noeuds concentrent 96,21 % des noeuds, et les trois premières relations concentrent 73,74 % des liens.

Table 3.15 – Indicateurs structuraux du graphe Neo4j

Indicateur	Valeur observée
Nombre total de noeuds	90 230
Nombre total de relations	550 542
Types de noeuds observés	18
Types de relations observés	18
Degré moyen non orienté	12,20
Densité orientée globale	0,0000676
Part des 5 premiers labels	96,21 %
Part des 3 premières relations	73,74 %
Noeuds isolés observés	12 487

Source : *08_neo4j_node_counts.csv*, *08_neo4j_relation_counts.csv* et requêtes Neo4j de diagnostic

Les principaux labels sont Candidat avec 41 298 noeuds, GroupeCompétences avec 14 579 noeuds, OffreEmploi avec 13 957 noeuds, Compétence avec 13 939 noeuds et Métier avec 3 039 noeuds. Cette structure est cohérente avec l'objectif métier : le graphe est d'abord un graphe d'appariement, non un graphe encyclopédique. Les candidats représentent 45,77 % des noeuds, les offres 15,47 %, les compétences 15,45 % et les groupes de compétences 16,16 %. Le risque statistique est clair : si le score de recommandation n'est pas normalisé, les familles très représentées peuvent dominer la recherche au détriment de signaux plus spécifiques comme le métier, le niveau ou le secteur.

Table 3.16 – Principaux types de noeuds et de relations

Type de noeud	Noeuds	Relation dominante	Relations
Candidat	41 298	POSSEDE	221 413
GroupeCompétences	14 579	NECESSITE	126 051
OffreEmploi	13 957	REQUIERT	58 519
Compétence	13 939	A_NIVEAU	41 254
Métier	3 039	PARTIE_DE	20 183
Employeur	1 638	CORRESPOND_MEPC	18 642
Localisation	312	DANS_SECTEUR	13 957
Secteur	192	PUBLIEE_PAR	13 935

Source : 08_neo4j_node_counts.csv et 08_neo4j_relation_counts.csv

Les relations dominantes sont POSSEDE, NECESSITE et REQUIERT. Elles forment le coeur du système : candidat-compétence, métier-compétence et offre-compétence. Les relations DANS_SECTEUR, PUBLIEE_PAR, LOCALISEE_A, A_NIVEAU et REQUIERT_NIVEAU ajoutent des contraintes d'interprétation. Le graphe n'est donc pas seulement volumineux ; il encode les chemins nécessaires à l'explication d'une recommandation.

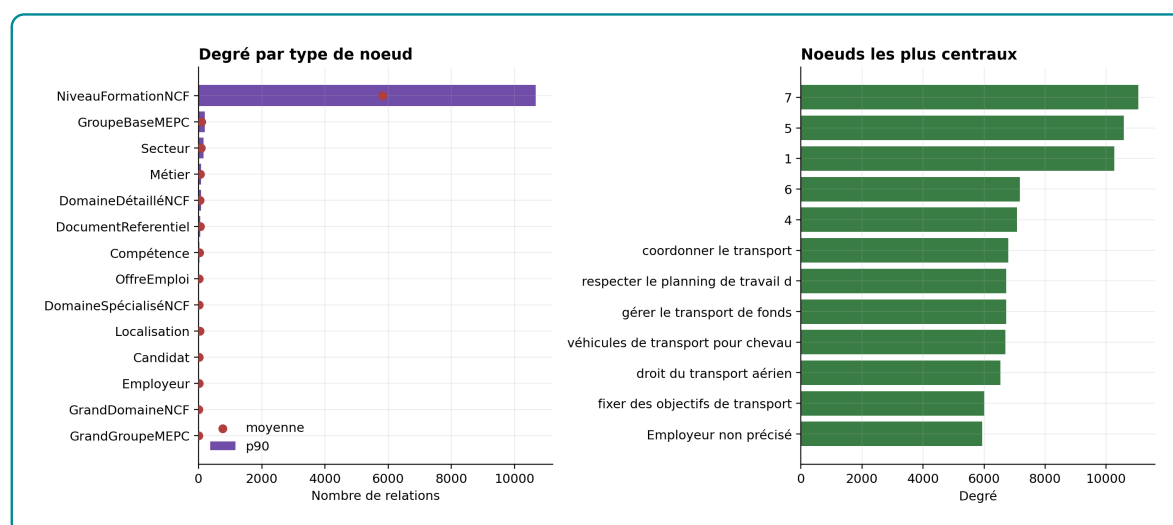


Figure 3.16 – Degrés et noeuds centraux du graphe

Source : outputs/memoire_stats/implementation_deep_current/09_neo4j_degree_stats.csv

Les degrés moyens montrent une forte hétérogénéité. Les noeuds NiveauFormationNCF

ont un degré moyen de 5 812,56 et un degré médian de 7 073, parce que neuf niveaux seulement absorbent une grande partie des relations candidat–niveau et offre–niveau. Les GroupeBaseMEPC ont un degré moyen de 90,22, les Secteur de 72,69, les Métier de 48,61 et les Localisation de 43,96. À l’inverse, DocChunk, SousGroupeMEPC et plusieurs noeuds référentiels restent faiblement connectés.

Table 3.17 – Hétérogénéité des degrés par famille de noeuds

Label	Moyenne	Médiane	p90	Max
NiveauFormationNCF	5 812,56	7 073	10 672,4	11 050
GroupeBaseMEPC	90,22	73	201	276
Secteur	72,69	5,5	151,6	1 548
Métier	48,61	44	77	186
Localisation	43,96	1	8	4 467
Compétence	30,57	5	28	6 789
OffreEmploi	7,97	9	11	11
Candidat	6,56	7	8	8

Source : 09_neo4j_degree_stats.csv

Cette hétérogénéité est une force et une faiblesse. Elle est une force car les hubs facilitent l’explication : le système peut rattacher rapidement une offre à un niveau, un secteur, une ville ou une famille MEPC. Elle est une faiblesse car certains hubs sont trop génériques. Les cinq plus grands hubs sont des niveaux NCF, puis apparaissent des compétences très larges liées au transport, à l’assurance ou au bâtiment. Le noeud Employeur non précisé atteint aussi un degré de 5 933. Ce dernier n’est pas un vrai signal métier ; il doit être pénalisé dans le reranking, sinon le graphe risque de récompenser une absence d’information.

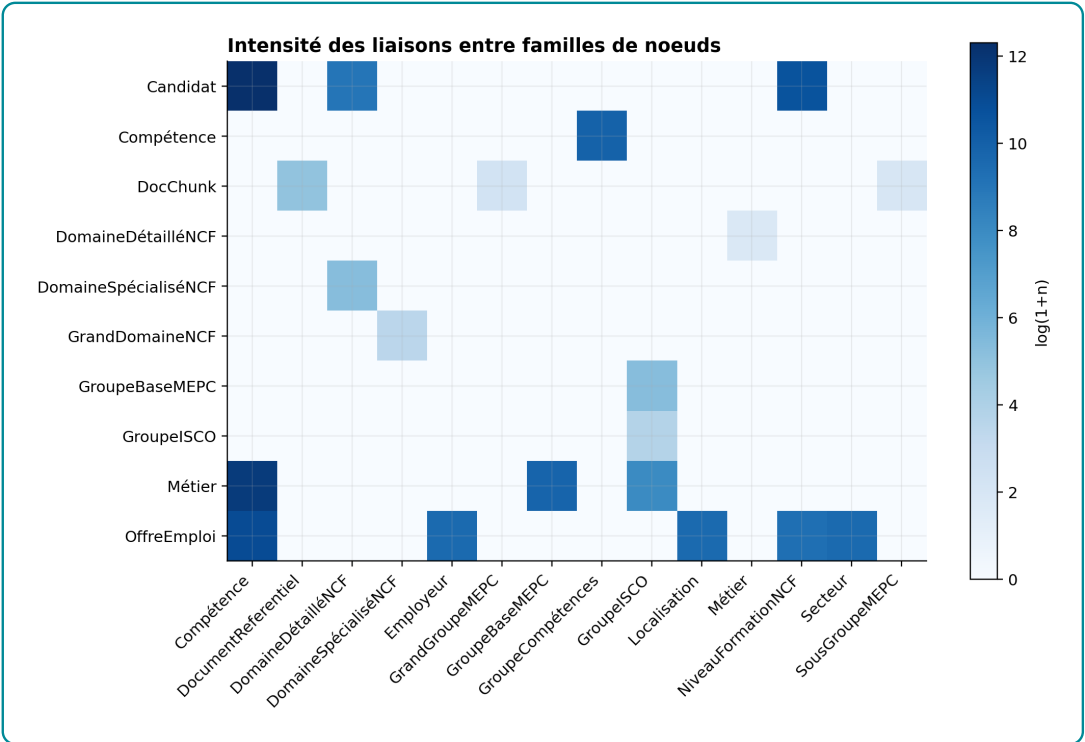


Figure 3.17 – Matrice des liaisons entre familles de noeuds

Source : `outputs/memoire_stats/implementation_deep_current/08_neo4j_patterns_top.csv`

La matrice des liaisons confirme que le graphe n’est pas seulement une collection de tables importées. Les relations structurantes relient candidats et compétences, offres et compétences, métiers et compétences, offres et secteurs, offres et employeurs, ainsi que les référentiels entre eux. Les motifs les plus importants sont les suivants : 221 413 chemins Candidat-POSSEDE-Compétence, 58 519 chemins OffreEmploi-REQUIERT-Compétence et 126 051 chemins Métier-NECESSITE-Compétence. En croisant les chemins, le graphe expose 72 832 868 triplets candidat-compétence-offre, 27 984 337 triplets candidat-niveau-offre et 963 063 triplets métier-compétence-offre. Ces chiffres expliquent pourquoi un simple top-K vectoriel est insuffisant : la base relationnelle offre un espace de justification beaucoup plus riche que la similarité textuelle seule.

Table 3.18 – Chemins métiers exploitables pour l’explication

Chemin relationnel	Occurrences	Usage dans le système
Candidat–Compétence–Offre	72 832 868	Mesure des compétences communes et base du <i>skill gap</i> .
Candidat–Niveau NCF–Offre	27 984 337	Contrôle de cohérence entre niveau acquis et niveau requis.
Métier–Compétence–Offre	963 063	Rattachement métier des offres et enrichissement MEPC/ISCO.
Offre–Secteur / Employeur / Localisation	41 607	Filtrage métier, géographique et contextualisation de la recommandation.
DocChunk–Référentiel	142	Citation et justification documentaire dans le GraphRAG.

Source : Requêtes Cypher de diagnostic sur Neo4j et `08_neo4j_patterns_top.csv`

Les communautés algorithmiques, de type Louvain, et les scores PageRank ne sont pas revendiqués dans ce chapitre. Le diagnostic d’exécution montre que les procédures GDS nécessaires n’étaient pas disponibles dans l’instance utilisée. C’est une limite méthodologique à assumer : parler de communautés sans calcul effectif serait fragile. En revanche, les composantes relationnelles métier sont déjà visibles par motifs : candidat–compétence–offre, métier–compétence–offre, offre–niveau–candidat et chunk–référentiel.

3.5 Implémentation du moteur hybride de recommandation

Le moteur hybride constitue le cœur décisionnel du système. Sans lui, l’application ne serait qu’un chatbot branché sur une recherche vectorielle. L’implémentation située dans `src/05_graphrag/context_builder.py` suit une logique en trois temps : pgvector présélectionne un ensemble d’offres candidates, Neo4j enrichit chaque couple candidat–offre avec des signaux relationnels, puis un score hybride réordonne les offres avant la génération. Le classement final n’est donc pas directement le score de similarité textuelle ; c’est une combinaison pondérée entre proximité sémantique,

couverture des compétences, niveau de formation et cohérence sectorielle.

3.5.1 Formalisation du score hybride

Pour chaque couple candidat-offre, le score implémenté est :

$$S_{\text{hybride}} = 0,35S_{\text{sémantique}} + 0,30S_{\text{compétences}} + 0,20S_{\text{niveau NCF}} + 0,15S_{\text{secteur}}$$

Le score sémantique provient du retrieval pgvector. Le score de compétences utilise le taux de compétences requises retrouvées dans le profil du candidat, puis applique une pénalité lorsque des compétences essentielles sont manquantes. Le score de niveau compare le niveau NCF du candidat au niveau NCF requis par l'offre. Le score sectoriel mesure le recouvrement lexical entre métier visé, secteur du candidat, formation et informations de l'offre. La localisation est utilisée comme information de contexte et d'affichage, notamment dans la sortie structurée, mais elle n'est pas encore intégrée comme composante pondérée du score. C'est une limite importante : le système peut expliquer la ville d'une offre, mais ne doit pas encore prétendre optimiser strictement la distance géographique.

Table 3.19 – Composantes du score hybride implémenté

Composante	Poids	Interprétation décisionnelle
Similarité sémantique	0,35	Mesure la proximité dense entre le profil/requête et l'offre présélectionnée par pgvector.
Compétences	0,30	Mesure la couverture des compétences requises ; les compétences essentielles manquantes réduisent le score.
Niveau NCF	0,20	Vérifie si le niveau du candidat est compatible avec le niveau requis par l'offre.
Secteur métier	0,15	Contrôle la cohérence entre secteur/métier visé du candidat et domaine de l'offre.
Localisation	Non pondérée	Conservée dans le contexte et l'explication, mais non utilisée comme poids explicite dans la version actuelle.

Source : `src/05_graphrag/context_builder.py`

Le traitement du niveau NCF est volontairement non linéaire. Si le niveau du candidat est supérieur ou égal au niveau requis, la composante vaut 1. Si le candidat est inférieur d'un niveau, elle vaut 0,72 ; inférieur de deux niveaux, 0,45 ; inférieur de trois niveaux ou plus, 0,20, avec pénalité supplémentaire sur le score final. Cette règle évite une erreur classique : traiter une différence d'un niveau et une différence de quatre niveaux comme de simples écarts numériques comparables.

3.5.2 Skill gap et verdicts de décision

Le *skill gap* est calculé à partir des relations du graphe : les compétences de l'offre sont séparées entre compétences acquises, compétences manquantes et compétences essentielles manquantes. Ce découpage est plus informatif qu'un score unique. Un candidat peut avoir un score moyen mais manquer une compétence essentielle ; inversement, un candidat légèrement sous le niveau requis peut être conservé si les compétences sont proches et si l'écart NCF est faible.

Table 3.20 – Règles de verdict du moteur hybride

Verdict implémenté	Condition principale	Lecture métier
Prêt à postuler	Score $\geq 0,72$, au plus une compétence essentielle manquante, pas d'écart NCF positif.	Profil immédiatement compatible.
Montée en compétence	Score $\geq 0,55$, au plus trois compétences essentielles manquantes, pas d'écart NCF bloquant.	Profil plausible, mais nécessitant un plan de montée en compétence.
Vivier à développer	Score $\geq 0,42$.	Profil à conserver, intéressant pour une trajectoire future ou une offre voisine.
Hors cible actuel	Score $< 0,42$.	Profil trop éloigné dans l'état actuel.

Source : `src/05_graphrag/context_builder.py`

Sur les 550 couples candidat-offre analysés dans le fichier d'ablation, les verdicts sont très majoritairement prudents : 259 couples, soit 47,09 %, relèvent du vivier à développer ; 144, soit 26,18 %, sont hors cible dans l'état actuel ; 118, soit 21,45 %, nécessitent une montée en compétence ; seuls 29 couples, soit 5,27 %, sont directement prêts à postuler. Cette distribution est rassurante : le moteur ne transforme pas mécaniquement toute proximité vectorielle en recommandation forte.

Table 3.21 – Distribution statistique des verdicts sur les sorties de reranking

Verdict	n	Part	Score moyen	Taux match moyen
Vivier à développer	259	47,09 %	0,4843	0,3050
Hors cible actuel	144	26,18 %	0,3073	0,2252
Montée en compétence	118	21,45 %	0,6870	0,9641
Prêt à postuler	29	5,27 %	0,7593	0,9712

Source : outputs/evaluation/pgvector_vs_graph/ablation_graph_rerank_details.csv

La corrélation entre le score sémantique et le score hybride vaut 0,6002, tandis que la corrélation entre le taux de match des compétences et le score hybride vaut 0,8023. Cela montre que le moteur ne se contente pas de recopier la similarité vectorielle : la compétence pèse fortement dans la décision finale. C'est cohérent avec l'objectif emploi-compétences. En revanche, cette forte dépendance aux compétences exige de surveiller la qualité des compétences extraites : si certaines compétences sont trop génériques ou mal typées, elles peuvent influencer fortement le score.

3.5.3 Effet du reranking hybride

Le reranking hybride modifie fortement les rangs. Sur les 550 couples étudiés, 82,73 % gagnent des positions après intégration du graphe et des règles métier, 10,18 % perdent des positions et 7,09 % restent au même rang. Le déplacement moyen est de 8,11 rangs. Ce résultat doit être lu avec nuance : un fort déplacement de rang n'est pas automatiquement une amélioration, mais il prouve que le moteur hybride apporte une décision supplémentaire par rapport au retrieval vectoriel.

Table 3.22 – Résumé statistique des scores et des rangs

Indicateur	Moyenne	Médiane	p75	Max
Score sémantique	0,4963	0,5087	0,5890	0,7661
Score hybride	0,4959	0,4863	0,6225	0,7836
Taux de match compétences	0,4606	0,2750	0,8330	1,0000
Rang pgvector initial	13,6073	12,0	21,0	30,0
Rang après graphe	5,5000	5,5	8,0	10,0
Déplacement de rang	8,1073	7,0	14,0	28,0

Source : `outputs/evaluation/pgvector_vs_graph/ablation_graph_rerank_details.csv`

L’ablation confirme l’intérêt du moteur hybride : l’ajout du graphe augmente le `recall@10` de 0,4599 à 0,5557 et le `NDCG@10` de 0,3889 à 0,4681. En contrepartie, la `précision@1` baisse de 0,4182 à 0,3636. Ce compromis est cohérent avec la finalité du système : pour explorer des opportunités et construire un plan de montée en compétence, élargir correctement le top 10 est plus utile que forcer une seule réponse immédiate. Si l’usage cible devient le recrutement automatique d’un seul candidat, les poids du score devront être recalibrés pour regagner de la `précision@1`.

3.5.4 Exemple de sortie intermédiaire

Le tableau suivant illustre une sortie intermédiaire avant génération. Il montre que le moteur expose les variables nécessaires à l’audit : score sémantique, score hybride, taux de match, verdict et déplacement de rang. Ce niveau de traçabilité est indispensable pour défendre le système comme recommandation robuste.

Table 3.23 – Exemple de reranking hybride pour un candidat

Offre	Sem.	Hybr.	Match	Rang	Verdict
Administrateur Finances et RH	0,6127	0,7033	0,833	2 → 1	Montée en compétence
Stage professionnel en cabinet	0,5468	0,6803	0,833	24 → 2	Montée en compétence
Directeur d'agence VIP	0,5411	0,6783	0,833	28 → 3	Montée en compétence
Contrôleur de gestion	0,5367	0,6767	0,833	29 → 4	Montée en compétence
Directeur Général Banque	0,5627	0,6298	0,833	3 → 5	Montée en compétence

Source : *outputs/evaluation/pgvector_vs_graph/ablation_graph_rerank_details.csv*, candidat PPBZV2501070016137

Cet exemple montre pourquoi le score hybride est nécessaire. Certaines offres initialement classées très bas par pgvector remontent fortement parce que leurs compétences et signaux métier correspondent mieux au profil. Mais le verdict reste « montée en compétence » et non « prêt à postuler » : le moteur ne se limite pas à reclasser, il qualifie la nature de la compatibilité. C'est précisément cette couche décisionnelle qui distingue le système d'un simple RAG conversationnel.

3.6 Orchestration LangGraph, génération contrôlée et interfaces

3.6.1 Workflow LangGraph et traces d'exécution

L'orchestration Agentic RAG est implémentée autour de plusieurs étapes : analyse de la requête, routage vers les outils, recherche pgvector, interrogation Neo4j, construction du contexte, génération finale et passage par le *critic*. Les outils disponibles couvrent la recherche sémantique, la recherche documentaire, les requêtes graphe, la recommandation hybride candidat-offre et les synthèses globales du graphe.

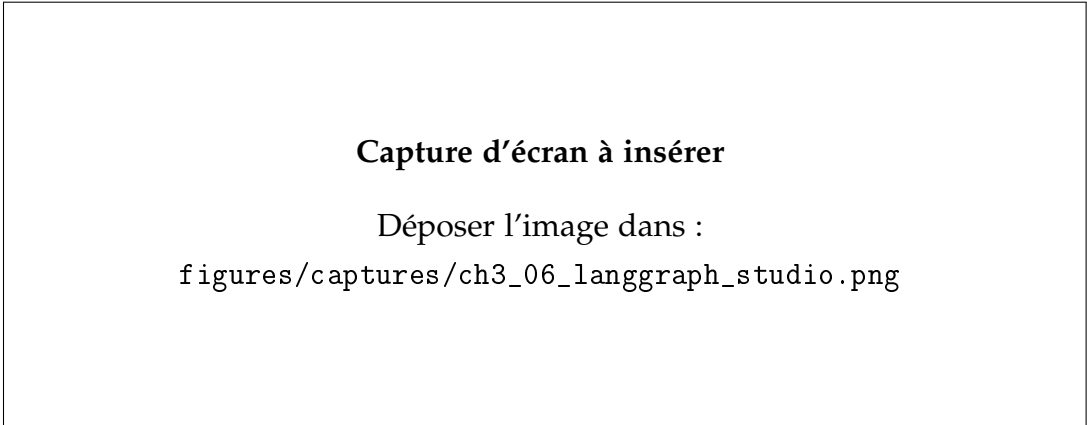


Figure 3.18 – Workflow LangGraph dans l'environnement de développement

Source : Capture d'écran de LangGraph Studio ou sortie `langgraph dev`

La capture LangGraph est la plus importante pour cette section. Elle doit montrer que l'agent n'est pas un simple chatbot, mais un workflow structuré en noeuds. L'intérêt méthodologique est la contrôlabilité : chaque étape peut être tracée, inspectée et corrigée. L'intérêt opérationnel est la modularité : ajouter un outil ou modifier le routage ne nécessite pas de réécrire tout le système.

Table 3.24 – Diagnostic d'implémentation de la génération contrôlée

Bloc	Preuve d'implémentation	Statut
Analyse requête	Noeud LangGraph <code>analyse_request</code>	Implémenté
Routage outils	Registre d'outils <code>pgvector</code> , <code>Neo4j</code> , recommandation hybride et synthèse globale	Implémenté
Construction contexte	Agrégation des résultats avant réponse finale	Implémenté
Génération	Client LLM <code>OpenRouter</code> avec fallback local	Implémenté
Critic	Vérification de l'appui contextuel et possibilité d'élargir la recherche	Implémenté
Score génération	À mesurer avec questions annotées et RAGAS dans le chapitre d'évaluation	Non mesuré ici

Source :

`outputs/memoire_stats/implementation_deep_current/13_agentic_generation_diagnostic.csv`

Cette table fixe une frontière importante. Côté génération, le chapitre d'implémentation prouve que les composants sont raccordés. Il ne prouve pas encore que les

réponses sont toujours factuelles, complètes ou utiles. Cette évaluation exige un jeu de questions, des critères annotés, RAGAS et une analyse statistique des seuils du *critic*.

3.6.2 Retrieval enrichi par le graphe

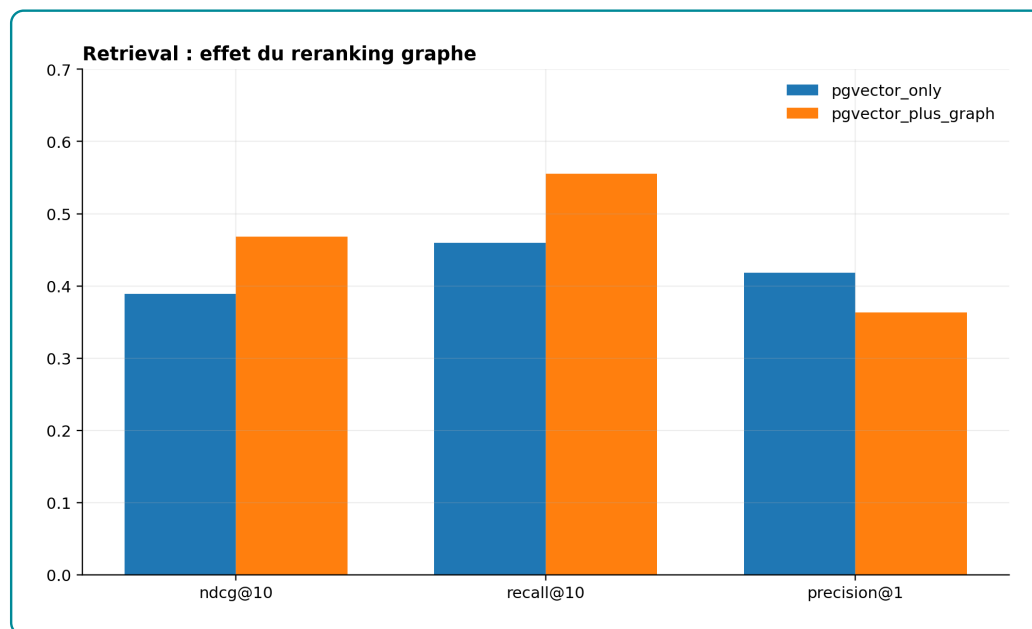


Figure 3.19 – Effet du reranking graphe sur le retrieval

Source : `outputs/evaluation/pgvector_vs_graph/ablation_metrics.csv`

L'ablation compare un retrieval pgvector seul à un retrieval pgvector enrichi par le graphe. Sur 55 requêtes évaluées, le passage à `pgvector_plus_graph` augmente le NDCG@10 de 0,3889 à 0,4681 et le recall@10 de 0,4599 à 0,5557. Le graphe améliore donc la couverture des éléments pertinents dans le top 10. En revanche, la précision@1 baisse de 0,4182 à 0,3636. Ce résultat doit être assumé : le graphe aide à élargir et contextualiser la liste, mais il peut déplacer la meilleure réponse immédiate. Le calibrage du reranking doit donc dépendre de l'usage cible : exploration d'opportunités ou recommandation unique.

3.6.3 Interfaces d'exploitation et captures à documenter

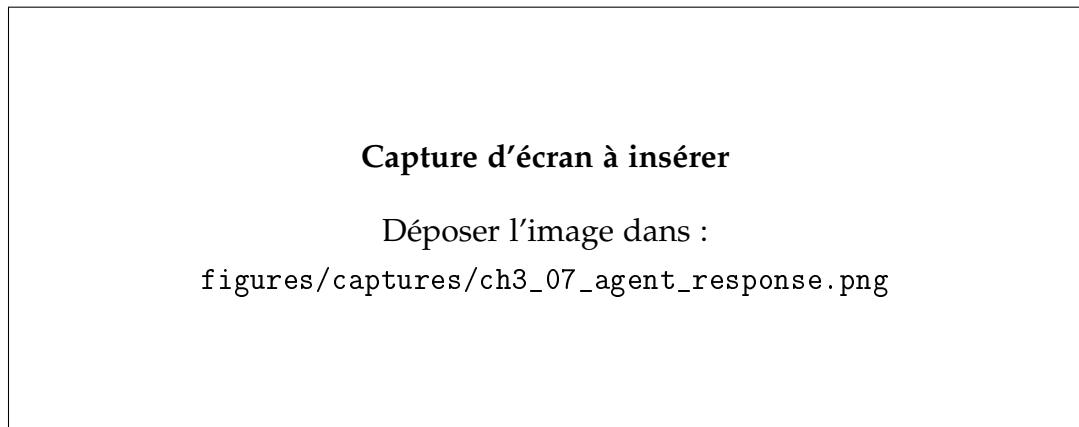


Figure 3.20 – Exemple de réponse générée avec sources et traces

Source : Capture d'écran de l'interface conversationnelle ou de la CLI

Cette capture doit illustrer une réponse complète : question utilisateur, outils mobilisés, contexte récupéré, réponse finale et, si possible, verdict du *critic*. Elle doit être choisie avec prudence. Un exemple spectaculaire mais non vérifiable affaiblirait le mémoire ; il vaut mieux sélectionner une requête simple, reproductible et bien tracée.

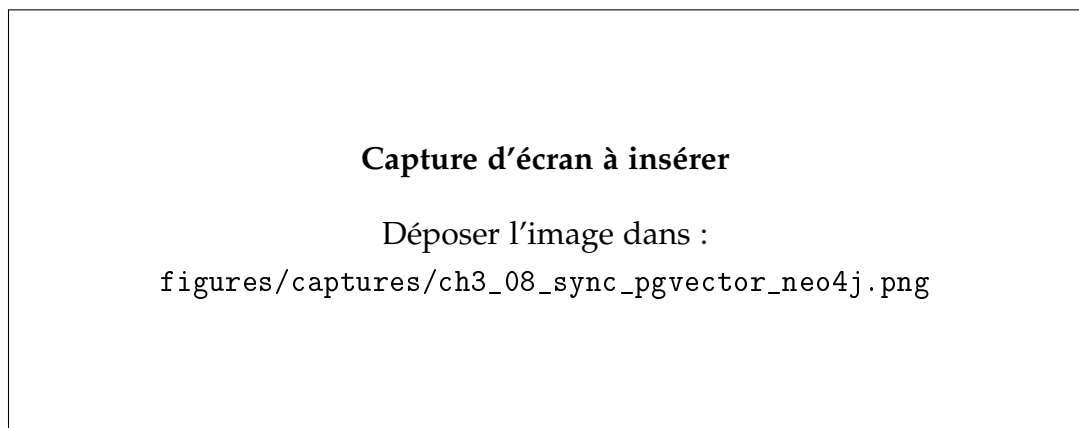


Figure 3.21 – Synchronisation entre pgvector et Neo4j

Source : Capture d'écran d'une requête SQL, Cypher ou d'un diagnostic terminal

Cette capture doit montrer la cohérence entre les deux bases : identifiants Neo4j présents dans pgvector, chunks documentaires reliés au graphe, ou exemple de passage d'un résultat vectoriel vers son noeud relationnel. Cette preuve est importante car le système hybride dépend précisément de cette articulation.

Synthèse du chapitre. L'implémentation aboutit à un système alimenté de bout en bout. Les données nettoyées couvrent 13 957 offres et 41 298 candidats. Le modèle SentenceTransformer fine-tuné domine les modèles de référence sur le classement sémantique. La base pgvector contient 72 643 embeddings synchronisés avec Neo4j et des index adaptés à la recherche hybride. Le graphe reconstruit contient 90 230 noeuds et 550 542 relations, principalement structurés autour des candidats, offres, compétences et référentiels. Enfin, l'orchestration LangGraph relie retrieval, graphe, contexte, génération contrôlée et critic.

Les limites doivent être conservées dans la lecture du chapitre. Les descriptions Louma-Jobs sont très peu renseignées, les compétences mélangent encore plusieurs natures d'information, les hubs du graphe doivent être normalisés pour ne pas dominer le score, et la génération n'a pas encore été évaluée comme réponse finale. Le chapitre suivant devra donc se concentrer sur les seuils, le compromis rappel-précision, la calibration du *critic*, et la mesure de factualité des réponses générées.

Évaluation de la performance du système

4.1 Objectif et logique générale de l'évaluation

Ce chapitre évalue empiriquement le système implémenté au chapitre précédent. L'objectif n'est pas seulement de produire des scores globaux, mais de répondre à une question centrale du mémoire : l'ajout d'un graphe de connaissances Neo4j apporte-t-il une amélioration mesurable par rapport à une recherche vectorielle fondée uniquement sur pgvector ? Cette question est décisive, car le graphe ajoute une complexité technique réelle : modélisation des nœuds et relations, chargement Neo4j, requêtes Cypher, enrichissement des résultats, maintenance d'une base supplémentaire. Son intérêt doit donc être justifié par un gain observable en qualité de classement, en explicabilité ou en contrôle métier.

L'évaluation a été conduite en trois niveaux. Le premier niveau porte sur le modèle d'embeddings, c'est-à-dire la capacité du SentenceTransformer fine-tuné à retrouver les paires offre-profil pertinentes dans un corpus de test. Le deuxième niveau porte sur l'ablation du moteur de recommandation : une configuration *pgvector seul* est comparée à une configuration *pgvector + graphe*. Le troisième niveau porte sur les limites opérationnelles observées, notamment la stabilité de la connexion Neo4j et la portée réelle des métriques utilisées.

Les résultats présentés dans ce chapitre proviennent de deux artefacts exécutables : le benchmark d'embeddings sauvegardé dans `outputs/evaluation/embedding_benchmark.json` et le notebook d'ablation `notebooks/10_Evaluation_pgvector_vs_graph.ipynb`. Les sorties de ce notebook sont exportées dans `outputs/evaluation/pgvector_vs_graph` et les figures dans `rapport/figures/generated/evaluation`. Cette traçabilité est importante : les nombres présentés ici ne sont pas des valeurs théoriques, mais des sorties produites par le pipeline local.

4.2 Données et protocole expérimental

Le système dispose de 7 861 offres normalisées et de 1 105 profils candidats. Le benchmark d’embeddings utilise le jeu de test issu de la construction des paires contrastives offre–profil, soit 473 requêtes de test. Chaque requête correspond à une description structurée d’offre ou de profil, et le corpus contient les textes enrichis utilisés pour l’apprentissage contrastif. Les métriques calculées sont celles de la recherche d’information : Precision@K, Recall@K, NDCG@K et MRR@K.

Pour l’ablation *pgvector* contre *pgvector + graphe*, le protocole est différent. Un échantillon de 60 candidats est tiré aléatoirement avec la graine 42. Pour chaque candidat, *pgvector* retourne un pool initial de 30 offres candidates. Deux classements sont alors comparés :

- **pgvector seul** : l’ordre des offres est conservé tel qu’il est retourné par la recherche sémantique et lexicale dans PostgreSQL/pgvector ;
- **pgvector + graphe** : le même pool initial est enrichi par Neo4j, puis rerangé par le score hybride combinant similarité sémantique, compatibilité de compétences, compatibilité de niveau NCF et alignement sectoriel.

Cette comparaison est volontairement réalisée sans génération LLM. Le modèle génératif OpenRouter intervient dans le chatbot pour formuler la réponse finale, mais il n’est pas mobilisé dans l’ablation. Cette séparation évite de confondre deux phénomènes : la qualité du classement des offres et la qualité rédactionnelle de la réponse.

La pertinence utilisée pour calculer les métriques d’ablation est un proxy construit à partir des métadonnées normalisées : chevauchement lexical entre le profil et l’offre, proximité sectorielle et compatibilité de niveau NCF. Cette approximation est acceptable pour comparer deux variantes internes sur le même échantillon, mais elle ne remplace pas une annotation humaine. Les résultats doivent donc être interprétés comme une mesure comparative de l’effet du graphe, non comme une mesure définitive de satisfaction utilisateur.

4.3 Évaluation du modèle d'embeddings

Le premier résultat concerne l'intérêt du fine-tuning du SentenceTransformer. Le tableau 4.1 présente les performances du modèle fine-tuné par rapport à plusieurs modèles généralistes ou multilingues. Le modèle spécialisé domine nettement les alternatives sur les métriques de retrieval.

Table 4.1 – Benchmark des modèles d'embeddings sur 473 requêtes de test

Modèle	NDCG@10	MRR@10	Recall@10	Latence ms/phrased
ST fine-tuné emploi-compétences	0.6336	0.5653	0.8520	32.77
MiniLM-L6 généraliste	0.1980	0.1603	0.3192	26.03
DistilUSE multilingue	0.1773	0.1434	0.2896	57.96
mE5-base multilingue	0.1505	0.1274	0.2262	295.76
ML-MiniLM-L12	0.1115	0.0922	0.1734	42.82
CamemBERT sentence	0.0904	0.0771	0.1332	144.15

Le modèle fine-tuné atteint un Recall@10 de 0.8520, contre 0.3192 pour MiniLM-L6 généraliste. L'écart est substantiel : le modèle spécialisé retrouve beaucoup plus souvent le document pertinent dans les dix premiers résultats. Le NDCG@10 passe également de 0.1980 à 0.6336, ce qui indique non seulement une meilleure récupération, mais aussi un meilleur ordre des résultats pertinents. La latence moyenne du modèle spécialisé reste contenue à 32.77 ms par phrase, ce qui est compatible avec une indexation ou une recherche interactive locale.

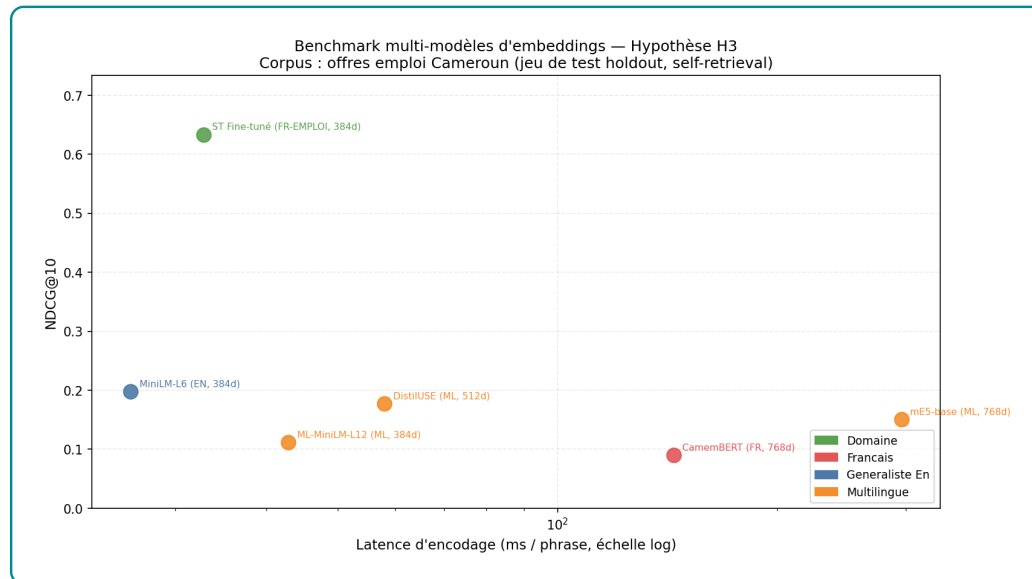


Figure 4.1 – Comparaison des modèles d’embeddings sur le jeu de test

Ce résultat valide le choix d’un encodeur adapté au domaine. Il montre aussi pourquoi une base vectorielle généraliste ne suffit pas : le vocabulaire emploi-compétences contient des formulations longues, des intitulés hybrides, des niveaux de formation et des compétences implicites. Le fine-tuning contrastif rapproche ces formulations dans l’espace vectoriel et améliore directement le retrieval.

4.4 Ablation pgvector seul contre pgvector + graphe

Le coeur de l’évaluation concerne l’apport de Neo4j. Le notebook d’ablation a demandé l’évaluation de 60 candidats. Sur cet échantillon, 55 requêtes ont été évaluées avec succès ; 5 requêtes ont échoué en raison d’une erreur interne Neo4j de type `CursorException` sur certaines relations. Cette information doit être conservée dans l’analyse : elle montre que la qualité d’un système GraphRAG ne dépend pas seulement du raisonnement algorithmique, mais aussi de l’intégrité physique et logique de la base graphe.

Table 4.2 – Comparaison retrieval : pgvector seul versus pgvector + graphe

Configuration	Precision@5	Recall@10	NDCG@10	MRR@10	HitRate@10
pgvector seul	0.2509	0.4599	0.3889	0.5602	0.9273
pgvector + graphe	0.3091	0.5557	0.4681	0.4647	0.7818
Différence	+0.0582	+0.0958	+0.0792	-0.0955	-0.1455

Les résultats sont instructifs parce qu'ils ne sont pas uniformément favorables au graphe. L'ajout de Neo4j améliore Precision@5, Recall@10 et NDCG@10. Autrement dit, lorsque l'on considère les cinq ou dix premiers résultats, l'enrichissement graphe permet de remonter davantage d'offres jugées pertinentes par le proxy métier. Le Recall@10 progresse de 0.4599 à 0.5557, soit un gain de 0.0958. Le NDCG@10 augmente également de 0.3889 à 0.4681, ce qui indique une meilleure organisation globale du top 10.

En revanche, le MRR@10 diminue de 0.5602 à 0.4647 et la Precision@1 passe de 0.4182 à 0.3636. Cela signifie que le graphe ne place pas toujours la première offre pertinente aussi haut que le classement vectoriel initial. Ce résultat est important : le graphe améliore la couverture et la cohérence globale du classement, mais il peut dégrader le tout premier rang lorsque ses signaux structurels sont imparfaits, incomplets ou trop fortement pondérés.

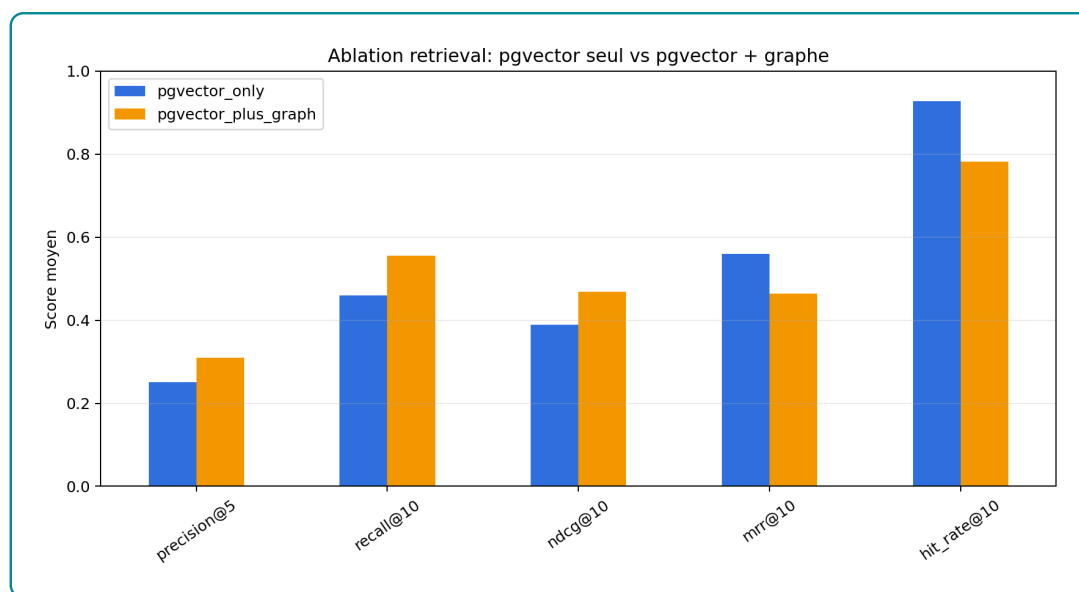


Figure 4.2 – Effet de l'enrichissement graphe sur les métriques de retrieval

La figure 4.2 confirme cette lecture. La configuration *pgvector* + *graphe* est supérieure sur les métriques orientées couverture, mais inférieure sur les métriques sensibles au premier rang. Dans un système de recommandation destiné au recrutement, cette distinction est opérationnellement importante. Si l'interface n'affiche qu'une seule offre, *pgvector* seul peut être plus robuste. Si l'interface affiche une liste courte accompagnée d'explications, le graphe devient plus utile car il enrichit le top 5 ou le top 10 avec des critères métier explicables.

4.5 Analyse du reranking par le graphe

Le reranking graphe déplace les offres dans le classement initial. Une offre qui était par exemple au rang 8 dans *pgvector* peut être remontée au rang 3 si elle présente une meilleure compatibilité NCF, un meilleur alignement sectoriel ou un meilleur score de compétences. Inversement, une offre très proche sémantiquement peut être rétrogradée si le niveau requis ou les signaux métier sont moins compatibles.

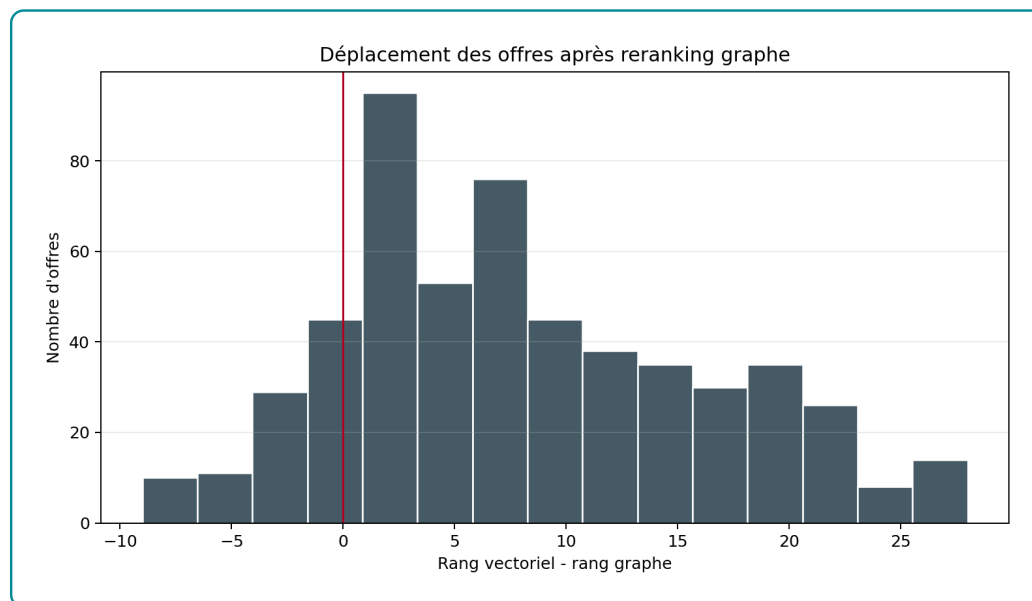


Figure 4.3 – Distribution des déplacements de rang après reranking par le graphe

Cette redistribution a deux effets. Le premier est positif : le système ne se limite plus à une proximité textuelle. Il peut corriger des cas où une offre ressemble lexicalement au profil mais ne correspond pas au niveau de formation ou au secteur recherché. Le

second est plus problématique : si les relations du graphe sont incomplètes, le score structurel peut pénaliser de bonnes offres ou favoriser des offres seulement parce que leurs métadonnées sont mieux renseignées. Le graphe ajoute donc de l'information, mais il ajoute aussi une source d'erreur potentielle.

Ce résultat appelle une conclusion nuancée. Le graphe n'est pas une garantie automatique d'amélioration. Il améliore le système si les relations sont correctes, si les poids du score hybride sont bien calibrés et si l'objectif est de produire une liste explicable plutôt qu'un unique meilleur résultat. Dans le système actuel, l'apport est visible sur Recall@10 et NDCG@10, mais il reste insuffisamment stabilisé sur le rang 1.

4.6 Calibration bayésienne des pondérations du score hybride

Afin de ne pas conserver des pondérations arbitraires dans le moteur hybride, un module logiciel spécifique a été ajouté : `src/07_evaluation/optimize_hybrid_weights.py`. Ce module lit les sorties détaillées de l'ablation, optimise les pondérations du score hybride avec Optuna et retient la combinaison qui maximise le NDCG@10. Les sorties sont sauvegardées dans `outputs/evaluation/hybrid_weight_optimization` et les figures correspondantes dans `rapport/figures/generated/evaluation`.

L'expérience a été conduite sur 550 couples candidat-offre correspondant à 55 candidats évalués. La pertinence graduée utilisée pour le NDCG@10 est `proxy_relevance`, c'est-à-dire le proxy métier construit à partir des métadonnées normalisées. Cette décision est préférable à l'utilisation directe des verdicts du moteur, car elle limite la circularité : on ne calibre pas les poids uniquement sur une étiquette déjà produite par le score hybride. Cependant, cette pertinence reste un label faible ; elle ne remplace pas une annotation humaine.

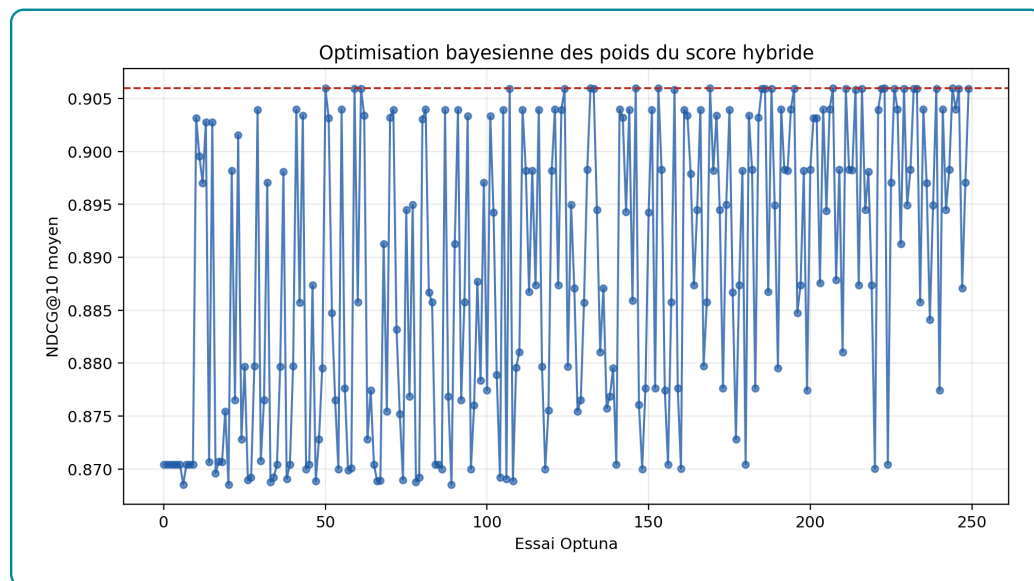
Table 4.3 – Résultat de l’optimisation bayésienne des pondérations du score hybride

Configuration testée	NDCG@10 moyen	Lecture statistique
Score sémantique seul	0.9087	Meilleure solution observée
Meilleur essai Optuna	0.9060	Très proche, mais inférieur au bord sémantique
Score Neo4j seul	0.8735	Moins performant que le signal sémantique
Poids égaux	0.8704	Combinaison naïve défavorable
Score hybride actuel	0.8829	Inférieur au score sémantique seul

Le résultat est sévère pour le score hybride actuel. Sur ce protocole, la meilleure pondération retenue est :

$$w_{\text{sémantique}} = 1,0000 \quad ; \quad w_{\text{Neo4j}} = 0,0000.$$

Autrement dit, avec les composantes actuellement exportées dans le fichier d’évaluation, le NDCG@10 est maximisé par le score sémantique seul. Ce résultat ne doit pas être maquillé : il indique que le signal Neo4j disponible dans cette expérience, exporté sous la forme `taux_match`, n’améliore pas le classement selon le proxy de pertinence. Il peut rester utile pour expliquer le *skill gap*, produire les verdicts ou filtrer des profils, mais il ne doit pas être surpondéré dans le classement si l’objectif unique est le NDCG@10.

**Figure 4.4** – Évolution du NDCG@10 au cours des essais Optuna

La figure 4.4 montre que les meilleurs essais apparaissent lorsque le poids de la composante Neo4j devient presque nul. Cette concentration des bons essais près de la frontière confirme que le problème n'est pas seulement un mauvais choix initial de pondérations : dans l'échantillon disponible, le signal graphe agrégé par `taux_match` n'apporte pas d'amélioration marginale au NDCG@10.

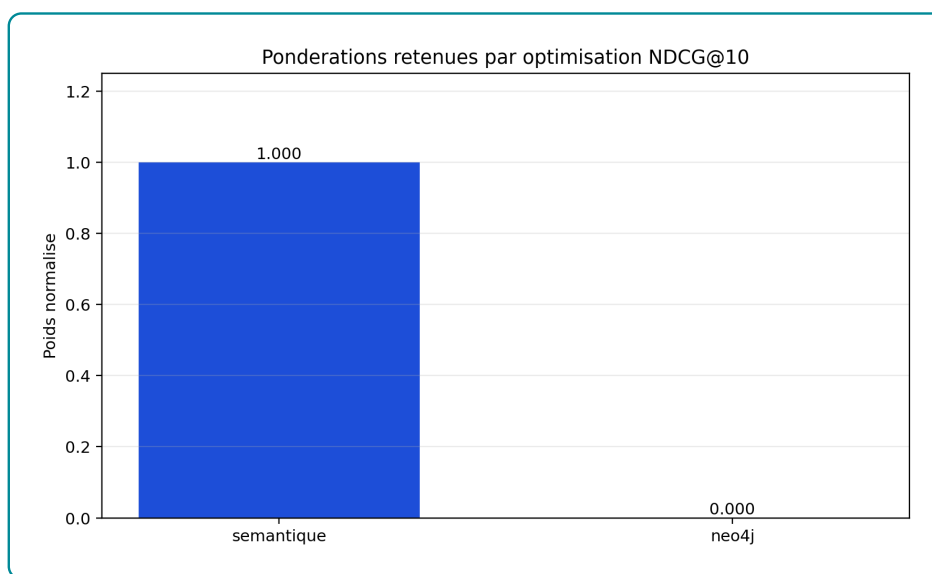


Figure 4.5 – Pondérations retenues après optimisation NDCG@10

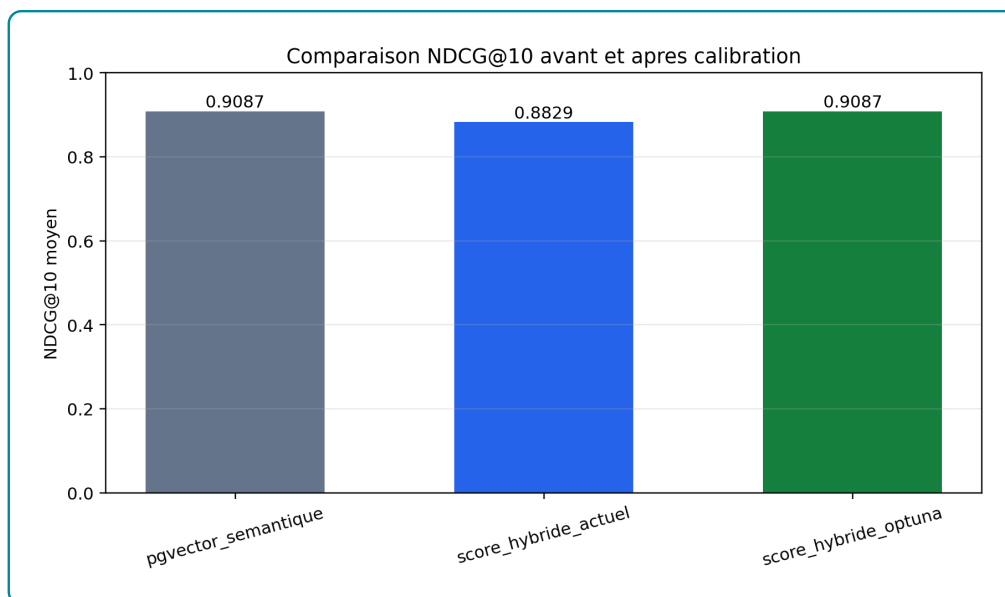


Figure 4.6 – Comparaison du NDCG@10 avant et après calibration

La figure 4.6 confirme que la calibration améliore le score hybride actuel, qui passe

de 0.8829 à 0.9087. Mais cette amélioration vient d'un choix radical : revenir au score sémantique pur. La conclusion d'ingénierie est donc claire. Le moteur hybride ne doit pas être abandonné, mais le score Neo4j doit être affiné avant d'être utilisé comme pondération de ranking : distinguer compétences essentielles et secondaires, intégrer explicitement le niveau NCF, exporter le score sectoriel, ajouter une composante géographique mesurable et calibrer ces signaux sur un jeu annoté. Dans l'état actuel des sorties disponibles, la stratégie optimale pour le NDCG@10 est de conserver Neo4j pour l'explication et le *skill gap*, mais de ne pas laisser le signal graphe brut dominer le classement.

4.7 Performance opérationnelle et stabilité

L'évaluation d'ablation a pris 38.7 secondes pour le coeur retrieval-reranking, hors coût complet de démarrage du notebook et de chargement du modèle. Le notebook exécuté par `nbconvert` a produit les résultats en environ 85 secondes en incluant le chargement du modèle SentenceTransformer, les connexions aux bases et l'export des figures. Cette durée reste acceptable pour une évaluation hors ligne, mais elle ne représente pas encore une latence utilisateur finale, car le chatbot complet ajoute la génération LLM via OpenRouter.

Le point critique est la stabilité Neo4j. Cinq candidats sur soixante ont échoué à cause d'une erreur interne `Neo.DatabaseError.Statement.ExecutionFailed` liée au page cache et à des relations corrompues. Le système a donc évalué 55 requêtes valides. Cette fragilité doit être traitée avant toute démonstration en production : il faut reconstruire ou vérifier la base Neo4j, contrôler les imports, et ajouter des tests d'intégrité sur les relations candidat-offre-compétence.

4.8 Interprétation par rapport aux hypothèses

Les résultats soutiennent partiellement l'hypothèse selon laquelle l'intégration d'un graphe améliore la recommandation. L'hypothèse est confirmée pour les métriques de couverture du top 10 : Recall@10 et NDCG@10 progressent lorsque Neo4j est uti-

lisé pour enrichir et reranger les offres. En revanche, elle n'est pas confirmée pour le premier rang : Precision@1, MRR@10 et HitRate@10 diminuent. Le graphe apporte donc un gain structurel, mais ce gain n'est pas encore parfaitement aligné avec l'optimisation du meilleur résultat immédiat.

Sur le plan méthodologique, cette conclusion est plus utile qu'une affirmation générale. Elle montre où le graphe apporte de la valeur : explicabilité, couverture, prise en compte des niveaux et secteurs. Elle montre aussi où le système doit être amélioré : calibration des poids, complétude des relations, robustesse de Neo4j, et validation par annotation humaine.

4.9 Lecture des sorties graphiques de performance

Les figures du chapitre convergent vers une lecture prudente. La première sortie graphique, consacrée au benchmark d'embeddings, montre que le fine-tuning apporte le gain le plus net de toute l'évaluation : le modèle spécialisé domine les modèles généralistes sur NDCG@10, MRR@10 et Recall@10. Ce résultat indique que l'adaptation au vocabulaire emploi-compétences améliore directement la capacité du système à retrouver les correspondances pertinentes.

La deuxième sortie graphique, issue de l'ablation pgvector contre pgvector + graphe, confirme un effet plus nuancé. Le graphe améliore les métriques de couverture du top 5 et du top 10, mais il ne sécurise pas le premier rang. Cette différence est centrale pour l'interprétation métier : le graphe est utile lorsque l'utilisateur consulte une liste courte d'offres accompagnée d'explications, mais il reste insuffisamment calibré si l'objectif est de produire une unique meilleure offre.

La troisième sortie graphique, centrée sur les déplacements de rang, explique ce résultat. Neo4j ne se contente pas d'ajouter un score marginal ; il réordonne réellement le pool initial retourné par pgvector. Cette intervention peut corriger des proximités purement lexicales, mais elle peut aussi rétrograder une bonne offre lorsque les relations de compétences, de niveau ou de secteur sont incomplètes. Le gain du graphe dépend donc de la qualité des relations et du réglage des poids du score hybride.

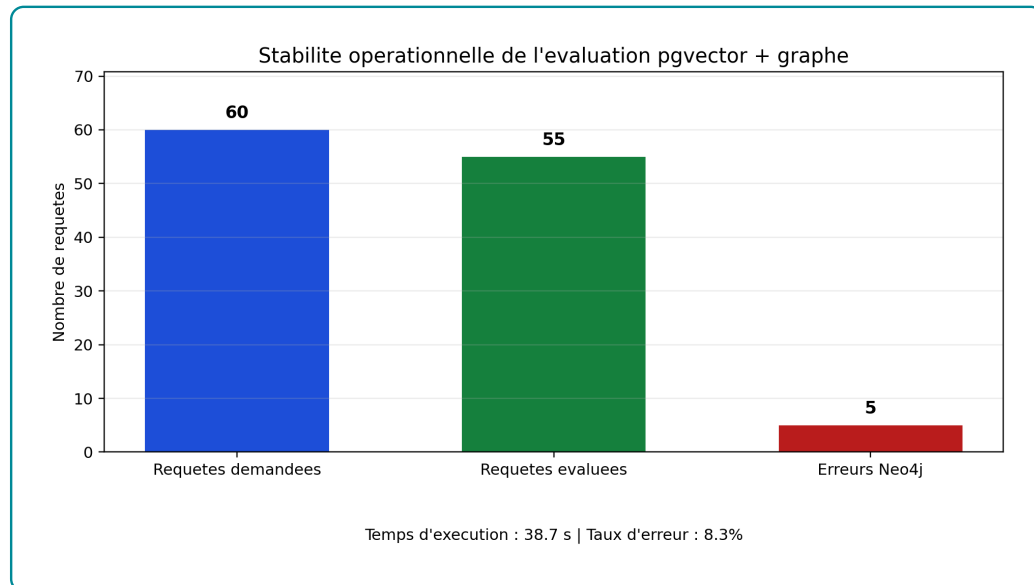


Figure 4.7 – Stabilité opérationnelle observée pendant l'évaluation d'ablation

La figure 4.7 complète cette lecture par la stabilité d'exécution. Sur 60 requêtes demandées, 55 ont été évaluées et 5 ont échoué à cause de Neo4j. Le taux d'erreur observé atteint donc 8.3%. Ce résultat n'annule pas les gains mesurés sur Recall@10 et NDCG@10, mais il empêche de conclure à une robustesse de production. Dans l'état actuel, la performance algorithmique du graphe existe, mais elle est conditionnée par un assainissement de la base Neo4j.

Enfin, aucune figure de calibration du critic n'est interprétée dans ce chapitre, car le dépôt ne contient pas encore de jeu réel `critic_calibration_cases.csv`. Le fichier disponible est seulement un gabarit, avec des lignes fictives. L'utiliser pour produire des courbes RAGAS donnerait des graphiques visibles, mais méthodologiquement faux. La calibration du critic doit donc rester une étape d'évaluation complémentaire, à exécuter uniquement après constitution d'un échantillon réel de questions, réponses, contextes récupérés et labels RAGAS ou humains.

4.10 Limites de l'évaluation

La principale limite est l'absence d'un jeu d'annotations humaines indiquant quelles offres sont réellement pertinentes pour chaque candidat. Le proxy utilisé combine

les métadonnées disponibles, mais il ne capture pas toutes les dimensions du recrutement : motivation, expérience qualitative, disponibilité, préférences salariales, soft skills ou contraintes géographiques fines. Les métriques doivent donc être lues comme des indicateurs comparatifs internes.

La deuxième limite concerne le graphe lui-même. Les erreurs Neo4j observées indiquent que l'instance locale doit être audité. Une base graphe partiellement corrompue ou incomplète peut biaiser l'évaluation du système hybride. Avant une évaluation finale à plus grande échelle, il faudra reconstruire la base, vérifier les nombres de nœuds et de relations, puis contrôler des chemins représentatifs : candidat-compétence, offre-compétence, offre-niveau, candidat-formation et métier-compétence.

La troisième limite concerne la génération LLM. Ce chapitre a volontairement isolé le retrieval et le reranking. La fidélité des réponses générées, la qualité des citations, la robustesse du critic et l'expérience utilisateur nécessitent une évaluation complémentaire, idéalement avec un protocole d'annotation humaine ou un juge LLM contrôlé par des critères explicites.

4.11 Synthèse du chapitre

L'évaluation confirme d'abord l'intérêt du SentenceTransformer fine-tuné : il dépasse nettement les modèles généralistes sur le jeu de test emploi-compétences. Elle montre ensuite que Neo4j apporte un gain réel sur les métriques de couverture et d'ordre global du top 10, mais qu'il dégrade le tout premier rang dans l'état actuel du système. La conclusion n'est donc pas que le graphe est automatiquement supérieur à pgvector. La conclusion correcte est que le graphe enrichit le classement lorsqu'on cherche une liste explicable et structurée, mais qu'il exige une meilleure qualité de relations et une calibration plus fine pour optimiser le top 1.

Ces résultats orientent directement les recommandations du chapitre suivant : renforcer l'intégrité Neo4j, construire un jeu d'évaluation annoté, ajuster les poids du score hybride, et distinguer dans l'interface les cas où le système privilégie la proximité sémantique immédiate et ceux où il privilégie la compatibilité métier structurée.

Discussion et recommandations

Conclusion générale

Bibliographie

- [1] Francesco RICCI et al., éd. *Recommender Systems Handbook*. Springer, 2011. DOI : [10.1007/978-0-387-85820-3](https://doi.org/10.1007/978-0-387-85820-3).
- [2] PGVECTOR CONTRIBUTORS. *pgvector : Open-source Vector Similarity Search for Postgres*. 2024. URL : <https://github.com/pgvector/pgvector> (visité le 04/05/2026).
- [3] Aidan HOGAN et al. "Knowledge Graphs". In : *ACM Computing Surveys* 54.4 (2021), p. 1-37. DOI : [10.1145/3447772](https://doi.org/10.1145/3447772).
- [4] Patrick LEWIS et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474. URL : <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [5] Dale T. MORTENSEN et Christopher A. PISSARIDES. "Job Creation and Job Destruction in the Theory of Unemployment". In : *The Review of Economic Studies* 61.3 (1994), p. 397-415. DOI : [10.2307/2297896](https://doi.org/10.2307/2297896).
- [6] Christopher A. PISSARIDES. *Equilibrium Unemployment Theory*. 2^e éd. Cambridge, MA : MIT Press, 2000.
- [7] George A. AKERLOF. "The Market for "Lemons" : Quality Uncertainty and the Market Mechanism". In : *The Quarterly Journal of Economics* 84.3 (1970), p. 488-500. DOI : [10.2307/1879431](https://doi.org/10.2307/1879431).
- [8] Michael SPENCE. "Job Market Signaling". In : *The Quarterly Journal of Economics* 87.3 (1973), p. 355-374. DOI : [10.2307/1882010](https://doi.org/10.2307/1882010).
- [9] Gary S. BECKER. *Human Capital : A Theoretical and Empirical Analysis, with Special Reference to Education*. New York : Columbia University Press, 1964.
- [10] David H. AUTOR. "The "Task Approach" to Labor Markets : An Overview". In : *Journal for Labour Market Research* 46.3 (2013), p. 185-199. DOI : [10.1007/s12651-013-0128-z](https://doi.org/10.1007/s12651-013-0128-z).

- [11] Gediminas ADOMAVICIUS et Alexander TUZHILIN. "Toward the Next Generation of Recommender Systems : A Survey of the State-of-the-Art and Possible Extensions". In : *IEEE Transactions on Knowledge and Data Engineering* 17.6 (2005), p. 734-749. DOI : [10.1109/TKDE.2005.99](https://doi.org/10.1109/TKDE.2005.99).
- [12] Michael J. PAZZANI et Daniel BILLSUS. "Content-Based Recommendation Systems". In : *The Adaptive Web* (2007), p. 325-341. DOI : [10.1007/978-3-540-72079-9_10](https://doi.org/10.1007/978-3-540-72079-9_10).
- [13] Yehuda KOREN, Robert BELL et Chris VOLINSKY. "Matrix Factorization Techniques for Recommender Systems". In : *Computer* 42.8 (2009), p. 30-37. DOI : [10.1109/MC.2009.263](https://doi.org/10.1109/MC.2009.263).
- [14] Xiangnan HE et al. "Neural Collaborative Filtering". In : *Proceedings of the 26th International Conference on World Wide Web*. 2017, p. 173-182.
- [15] Robin BURKE. "Hybrid Recommender Systems : Survey and Experiments". In : *User Modeling and User-Adapted Interaction* 12 (2002), p. 331-370. DOI : [10.1023/A:1021240730564](https://doi.org/10.1023/A:1021240730564).
- [16] Chen YANG et al. "Modeling Two-way Selection Preference for Person-Job Fit". In : *Proceedings of the 16th ACM Conference on Recommender Systems*. 2022, p. 102-112.
- [17] A. T. WASI. "HRGraph : Leveraging LLMs for HR Data Knowledge Graphs with Information Propagation-based Job Recommendation". In : *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models*. 2024, p. 56-62.
- [18] B. YANG et Z. SHEN. "Knowledge Graph Construction and Talent Competency Prediction for Human Resource Management". In : *Alexandria Engineering Journal* 121 (2025), p. 223-235.
- [19] Ashish VASWANI et al. "Attention Is All You Need". In : *Advances in Neural Information Processing Systems*. 2017, p. 5998-6008.
- [20] Tomas MIKOLOV et al. "Distributed Representations of Words and Phrases and their Compositionality". In : *Advances in Neural Information Processing Systems*. T. 26. 2013. URL : <https://proceedings.neurips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality>.

- [21] Dzmitry BAHDANAU, Kyunghyun CHO et Yoshua BENGIO. "Neural Machine Translation by Jointly Learning to Align and Translate". In : *International Conference on Learning Representations*. 2015. URL : <https://arxiv.org/abs/1409.0473>.
- [22] Jacob DEVLIN et al. "BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding". In : *Proceedings of NAACL-HLT*. 2019, p. 4171-4186. URL : <https://aclanthology.org/N19-1423/>.
- [23] Nils REIMERS et Iryna GUREVYCH. "Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks". In : *Proceedings of EMNLP-IJCNLP*. 2019, p. 3982-3992. URL : <https://aclanthology.org/D19-1410/>.
- [24] Tom B. BROWN et al. "Language Models are Few-Shot Learners". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 1877-1901. URL : <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html>.
- [25] Colin RAFFEL et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In : *Journal of Machine Learning Research* 21.140 (2020), p. 1-67. URL : <https://www.jmlr.org/papers/v21/20-074.html>.
- [26] Long OUYANG et al. "Training Language Models to Follow Instructions with Human Feedback". In : *Advances in Neural Information Processing Systems* 35 (2022), p. 27730-27744. URL : https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- [27] Nandan THAKUR et al. "BEIR : A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models". In : *Advances in Neural Information Processing Systems*. T. 34. 2021, p. 28974-28989. URL : <https://arxiv.org/abs/2104.08663>.
- [28] Niklas MUENNIGHOFF et al. "MTEB : Massive Text Embedding Benchmark". In : *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, p. 2014-2037. URL : <https://aclanthology.org/2023.eacl-main.148/>.

- [29] Yu A. MALKOV et D. A. YASHUNIN. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), p. 824-836. DOI : [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [30] Jeff JOHNSON, Matthijs DOUZE et Hervé JÉGOU. "Billion-scale Similarity Search with GPUs". In : *IEEE Transactions on Big Data* 7.3 (2019), p. 535-547. DOI : [10.1109/TBDATA.2019.2921572](https://doi.org/10.1109/TBDATA.2019.2921572).
- [31] Thomas N. KIPF et Max WELLING. "Semi-Supervised Classification with Graph Convolutional Networks". In : *International Conference on Learning Representations*. 2017.
- [32] Petar VELIČKOVIĆ et al. "Graph Attention Networks". In : *International Conference on Learning Representations*. 2018.
- [33] Xiangnan HE et al. "LightGCN : Simplifying and Powering Graph Convolution Network for Recommendation". In : *Proceedings of SIGIR*. 2020, p. 639-648.
- [34] Yunfan GAO et al. *Retrieval-Augmented Generation for Large Language Models : A Survey*. 2023. arXiv : [2312.10997](https://arxiv.org/abs/2312.10997) [cs.CL]. URL : <https://arxiv.org/abs/2312.10997>.
- [35] Shunyu YAO et al. "ReAct : Synergizing Reasoning and Acting in Language Models". In : *International Conference on Learning Representations*. 2023. URL : https://openreview.net/forum?id=WE_vluYUL-X.
- [36] Akari ASAI et al. "Self-RAG : Learning to Retrieve, Generate, and Critique through Self-Reflection". In : *International Conference on Learning Representations*. 2024. URL : <https://openreview.net/forum?id=hSyW5go0v8>.
- [37] Shahul Es et al. "RAGAS : Automated Evaluation of Retrieval Augmented Generation". In : *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics : System Demonstrations*. 2024, p. 150-158. URL : <https://aclanthology.org/2024.eacl-demo.16/>.
- [38] Yang LIU et al. "G-Eval : NLG Evaluation using GPT-4 with Better Human Alignment". In : *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, p. 2511-2522. URL : <https://aclanthology.org/2023.emnlp-main.153/>.

- [39] LANGCHAIN. *LangGraph Documentation*. Documentation officielle du framework LangGraph. 2026. URL : <https://langchain-ai.github.io/langgraph/> (visité le 06/06/2026).
- [40] William L. HAMILTON. *Graph Representation Learning*. Morgan & Claypool Publishers, 2020. DOI : [10.2200/S01045ED1V01Y202009AIM046](https://doi.org/10.2200/S01045ED1V01Y202009AIM046).
- [41] Takuya AKIBA et al. "Optuna : A Next-generation Hyperparameter Optimization Framework". In : *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, p. 2623-2631. DOI : [10.1145/3292500.3330701](https://doi.org/10.1145/3292500.3330701).

Annexes

Table A.1 – Labels de nœuds et relations structurantes du graphe

Bloc	Labels de nœuds	Relations principales et interprétation
Données d'emploi	OffreEmploi, Candidat, Employeur, Secteur, Localisation	PUBLIEE_PAR relie une offre à son employeur; DANS_SECTEUR la rattache à un secteur; LOCALISEE_A indique la ville ou zone d'exercice; A_NIVEAU relie le candidat à son niveau de formation.
Compétences et métiers	Compétence, Métier, GroupeCompétences, GroupeISCO	NECESSITE relie un métier aux compétences attendues; PARTIE_DE rattache une compétence à son groupe; CLASSIFIE_DANS relie un métier à un groupe ISCO; PLUS_LARGE_QUE représente les relations hiérarchiques entre compétences.
Référentiel MEPC	GrandGroupeMEPC, SousGroupeMEPC, GroupeBaseMEPC	CONTIENT représente la hiérarchie interne de la nomenclature; ALIGNE_AVEC rapproche les groupes MEPC des groupes ISCO; CORRESPOND_MEPC rattache un métier ESCO à un groupe de base MEPC.
Référentiel NCF	NiveauFormationNCF, GrandDomaineNCF, DomaineSpécialiséNCF, DomaineDétailléNCF	CONTIENT représente la hiérarchie formation; REQUIERT_NIVEAU_NCF relie une offre au niveau demandé; A_FORMATION rattache un candidat à son domaine de formation; PREPARE_POUR relie un domaine de formation aux métiers visés.
Recommandation	OffreEmploi, Candidat, Compétence, Métier, DomaineDétailléNCF	Les chemins candidat–compétence–offre permettent d'identifier les compétences acquises et manquantes; les chemins offre–niveau–formation servent à contrôler la compatibilité de qualification; les chemins formation–métier soutiennent la proposition de montée en compétences.

Source : Auteur

Table A.2 – Volume d'entités indexées dans la base vectorielle pgvector

Type d'entité	Nombre	Source principale
OFFRE_EMPLOI	15 722	Scraping KmerAI
COMPETENCE	13 939	Référentiel ESCO v1.1
METIER	3 039	Référentiel ESCO v1.1
CANDIDAT	1 105	Base locale de profils demandeurs
GROUPE_BASE_MEPC	209	INS Cameroun - MEPC 2013
DOMAINE_DETAILLE_NCF	201	INS Cameroun - NCF 2017
Total	34 215	

Source : Auteur

Table des matières

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	vi
Liste des tableaux	viii
Liste des figures	x
Avant-propos	xi
Résumé	xiii
Abstract	xiv
Introduction générale	1
I Cadre théorique et conceptuel	6
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	7
1.1 Concepts clés mobilisés dans le mémoire	7
1.2 Fondements économiques et problématique d'appariement	10

Table des matières

1.2.1	Marché du travail, recherche d'emploi et frictions	10
1.2.2	Capital humain, tâches et inadéquation des compétences	10
1.3	Systèmes de recommandation et person-job fit	11
1.3.1	Approches par contenu, collaboratives et hybrides	11
1.3.2	Spécificité du person-job fit	12
1.4	IA générative et connaissances	13
1.4.1	Grands modèles de langage et génération contrôlée	13
1.4.2	Embeddings, recherche sémantique et bases vectorielles	14
1.4.3	Graphes de connaissances et recommandation relationnelle	15
1.5	RAG, GraphRAG et orchestration agentique	15
1.5.1	De la RAG au GraphRAG	15
1.5.2	Agentic RAG, critic et traçabilité	17
1.6	Évaluation, acquis de la littérature et positionnement du mémoire	17
1.6.1	Évaluer la performance et la factualité	17
1.6.2	Acquis, manques et contribution du mémoire	20
2	Source de données et Approche méthodologique	22
2.1	Cadre méthodologique et sources de données	22
2.1.1	Positionnement méthodologique	22
2.1.2	Sources de données et leur rôle dans le système	26
2.1.3	Préparation et normalisation des données	28
2.1.4	Alignement des référentiels métiers et formations	29
2.2	Modélisation sémantique et structuration des connaissances	29
2.2.1	Choix du modèle d'embeddings et fine-tuning	29
2.2.2	Indexation vectorielle dans PostgreSQL et pgvector	30

2.2.3	Construction du graphe de connaissances sous Neo4j	31
2.2.4	Score hybride et logique de skill gap	31
2.3	Orchestration agentique et génération contrôlée	33
2.3.1	Architecture Agentic RAG et orchestration LangGraph	33
2.3.2	Text2Cypher et rôle du LLM	34
2.3.3	Interfaces et traçabilité	34
2.4	Évaluation prévue, limites et synthèse méthodologique	35
2.4.1	Protocole d'évaluation prévu	35
2.4.2	Limites méthodologiques et précautions d'interprétation	36
2.4.3	Synthèse de l'approche méthodologique	36
II	Présentation des résultats	37
3	Implémentation du système de recommandation	38
3.1	Architecture opérationnelle et données exploitées	38
3.1.1	Briques effectivement implémentées	38
3.1.2	Diagnostic du corpus après nettoyage	43
3.1.3	Qualité des variables utiles au matching	45
3.1.4	Nettoyage réalisé et limites conservées	49
3.2	Implémentation du modèle d'embeddings et indexation pgvector	50
3.2.1	Choix du modèle baseline et construction des paires	51
3.2.2	Courbes d'entraînement et comparaison des modèles	54
3.2.3	Interprétation de l'espace vectoriel	58
3.3	Implémentation de pgvector comme brique de retrieval	59
3.3.1	Stratégie de chunking des référentiels	61

3.4	Implémentation du graphe Neo4j comme base de connaissances	66
3.4.1	Schéma du graphe de connaissances	66
3.4.2	Analyse topologique du graphe	68
3.5	Implémentation du moteur hybride de recommandation	73
3.5.1	Formalisation du score hybride	74
3.5.2	Skill gap et verdicts de décision	75
3.5.3	Effet du reranking hybride	77
3.5.4	Exemple de sortie intermédiaire	78
3.6	Orchestration LangGraph, génération contrôlée et interfaces	79
3.6.1	Workflow LangGraph et traces d'exécution	79
3.6.2	Retrieval enrichi par le graphe	81
3.6.3	Interfaces d'exploitation et captures à documenter	82
4	Évaluation de la performance du système	84
4.1	Objectif et logique générale de l'évaluation	84
4.2	Données et protocole expérimental	85
4.3	Évaluation du modèle d'embeddings	86
4.4	Ablation pgvector seul contre pgvector + graphe	87
4.5	Analyse du reranking par le graphe	89
4.6	Calibration bayésienne des pondérations du score hybride	90
4.7	Performance opérationnelle et stabilité	93
4.8	Interprétation par rapport aux hypothèses	93
4.9	Lecture des sorties graphiques de performance	94
4.10	Limites de l'évaluation	95
4.11	Synthèse du chapitre	96

Table des matières

5 Discussion et recommandations	97
Conclusion générale	I
Bibliographie	I
Bibliographie	III
A Annexes	VIII